Project HiveMind: A System Design for Decentralized Sparse Mixture-of-Experts Training with DiLoCo-Style Outer Optimization on 8 GB Consumer GPUs

Integration of Low-Communication Training Methods, Controlled Experimental Evaluations, Live WAN Swarm Validation, the Bi-Layer Honeybee Agent Mesh, Multi-Division Vision-MoE, Pre-Registered Experiment E20, and the Literature-Grounded > 200-Node International Launch Forecast

Xavier Callens*
Project HiveMind Open-Source Contributors

November 11, 2026 — Version 4.4.0 (adds Section 14: analysis of LLM-JEPA [64] and controlled RTX 2070 experiments `E28` [5 seeds, constant LR: NTP 75.90 %, LLM-JEPA 80.00 %, shuffled-view 44.55 %; `P28a` not supported], `E28b` [`IP-JEPA-02`, 10 seeds, cosine LR + loss-dropout $\alpha \in \{0.50, 0.75\}$: `P28b_gain` supported at +16.35 pts, $p = 0.004761$ and +38.97 pts vs. shuffled; `P28b_dropout` refuted for $\lambda = 4.0$ co-tuning at 61.10 %, whereas unscaled $\lambda = 1.0$ at $\alpha = 0.75$ reaches 84.72 %, +12.05 pts, $p = 0.003224$ at 1.390× train time], and `E29` [`IP-JEPA-03/04`, 5-seed Stage-2 4-bit `NF4` `GraniteMoE` $r = 4$ QLoRA DiLoCo + `B3` gate: all JEPA arms refused promotion at −0.106741 and −0.140903 nats, ≤ 37.03 MiB peak VRAM, proving `view_gap_sv1` must remain non-gating]; v4.3.0: adds Section 13: live two-host `E8` Stage 1 RDA verification across `socreateai-agora-hermes-node1` T4 and `XAVGAMSRV` RTX 2070 [3 seeds, $K = 2$, paired diff $+4.8 \times 10^{-7} \pm 8.3 \times 10^{-7}$, post-join $p_{50} = 1.20$ s, $p_{95} = 3.79$ s], Stage 2 two-host AMCP Kafka QLoRA MoE [`crossA.json`: 4.1709 → 1.4808, `night-002.json`: 211 rounds to 1.4525, `chaos-1.json`: 70 rounds to 1.4649] and live RTX 2070 CUDA NF4 QLoRA MoE [4.1721 → 1.3395 at 27.75 MiB peak VRAM vs. shuffled-label control 4.1563]; live `A1` production Cloud Run gateway verification over WireGuard [48/48 HTTP 200 on `hivemind/granite-moe` at $p_{50} = 0.41$ s / 0.39 s and WRR 12 + 12 → 24/24 failover with exact HMT conservation]; negative `B3` train-to-serve gate result on `hmq_00147` [−0.00403 nats] and `E24` 2 × 2 factorial attribution + 20-round remediated window `hmq_00216..hmq_00235` [median −0.01275 nats, share worse 0.2667 ≤ 0.30]; planetary-scale verification suite `PS-01..PS-11` [`E21` canary 0.0977 FLOP ratio, `E22` `did:key` discovery $p_{95} = 4.18$ s, `E23` AMCP NAT mobility $p_{95} = 1.48$ s, `E26` 186.18× residential compression, `E27` 1,000-node Byzantine testbed, $120.62/mo ≤ $150/mo GCP cost receipt]; and reconciled **166** Lean 4 lemmas + 3 Mathlib theorems; Version 4.2.0: adds Section 12: pre-registered Experiment `E20` validating the ≤ 1,536 MiB Adaptive Multi-Split ML background guard at 680.0 MiB [94.34%–98.29% latency reduction], Top-1% 2D-DCT + EF21 outer-gradient compression at 225.4× with 3×10^{-8} drift, and 6-region Super-Peer fan-out cutting 10,000-node coordinator ingress by 31.45× plus 62.5% SQLite write-IOPS coalescing; 3-plan-class BOINC workunit assimilation and nocturnal Pareto-ratchet V-MoE evolution; and the literature-grounded > 200-node [265-node active swarm + 2,400 visitors] International Launch Traffic, Throughput & Balancing Forecast combining Kleinrock/Halfin-Whitt $M/M/c(t)$ Erlang-C, Kingman $G/G/c$, Mitzenmacher P2C, Kelly shadow pricing, and Bass prosumer conversion across 6 global Super-Peer hubs and a 5-wave roadmap to 10,000+ nodes; v4.1.0: adds Section 11: "La Ruche" sub-1.5 GB Vision-MoE across 4 visual backbones at 972.0–1,490.0 MiB, Exoplanet & Astrophysics CWT/Focal-Loss Vision-MoE PoC at 910.0–1,410.0 MiB, 7-Division 15-dataset 42,870,000-sample Swarm Image-MoE Orchestrator across 104

Preprint (scripted audits + external review records in `paper/review/`) · Zenodo DOI (v3.8.0–v4.4.0): 10.5281/zenodo.23776302 · https://hivemind-at-home-ruche-cfhmhmvv5a-uc.a.run.app/

Bees, and the 18-Month, 4-Phase Research Roadmap for Overcoming Scale and Security Bottlenecks in Decentralized MoE Architectures via Top-1% Quantized Split Learning, `zk-SNARK` Verifiable Computation, and Geo-Cryptographic Mesh Routing across Seasons 29–32; v4.0.0: adds Section 10: the Bi-Layer Honeybee Agent Mesh architecture with Kafka AMCP Quorum Sensing and ABC MoE loop, 20 nocturnal Hugging Face domain MoE adapters (+277.06 TFLOPs compute contribution, 35 models total), 4 Macro-Cluster Super-MoEs, Timezone-Aware Diurnal Scheduling with Long-Tail Resident Intent Scoring, Bring Your Own Bee formal lifecycle with 3 live exemplars, 4 edge models, Swarm Nectar Cache P2P bus with HMT royalties, and S20–S24 Bio-Inspired Swarm Roadmap; v3.9.0: live 47-node heterogeneous WAN swarm validation, 100 assimilated BOINC 6-domain MoE workunits + 24 nocturnal multi-producer MoE workunits, 5,632.11 Cobblestones / 2.433 PFLOPs attested compute, 3,409–4,838 MiB QLoRA MoE VRAM footprint, 0.0% execution-class-matched false rejection rate, and 51.24 GFLOPS custom Turing kernels at 196.8 ms mean p50 latency; v3.8.0: withdraws the invalid E11 Qwen2.5-Coder-7B row and adds a likely-cause note to E10b; v3.7.2: adds pre-registered E10b, E11, E12 results; 141 Lean 4 lemmas + 3 Mathlib theorems)

Abstract

Training and serving frontier Large Language Models (LLMs) are increasingly monopolized by hyperscale datacenters due to two fundamental systems bottlenecks: the **VRAM Wall** (a single 8 GB–12 GB consumer GPU cannot hold a 70B+ dense model or its FP32 AdamW optimizer states) and the **Bandwidth Wall** (standard synchronous All-Reduce Distributed Data Parallelism saturates 10–50 Mbps residential internet links instantly). Yet the global PC gaming and open-source developer community possesses millions of idle NVIDIA RTX 20/30/40-series, AMD ROCm, and Apple Metal GPUs alongside terabytes of NVMe flash storage.

We describe **Project HiveMind**, an open-source decentralized training and inference system, and report what has and has not yet been validated. The system combines **Branch-Train-Mix (BTM) sparse MoE upcycling**, **weight-tied looped recurrence**, and **DiLoCo-style outer optimization**, with an emphasis on fitting a strict 8 GB GPU memory budget. Workers stream open-weight backbones (`Qwen2.5`) into 4-bit NF4 and train QLoRA adapters; outer pseudo-gradients can be compressed with a DCT + 2-bit centroid scheme adapted from DeMo [28] and SparseLoCo [29]; the outer optimizer supports a CPU-resident flat-bucket state (after `prime-diloco` [17]) and the staleness gate of CGAD [24]; converged branches are upcycled into a sparse MoE with optional Iso-CTS [25] or TSV [27] attention merging and FoMoE-style partial expert replication [20].

Empirical and formal evaluation. We evaluate the system at three levels, separating verified properties from open multi-node questions: (i) Ten *unit-level implementation checks* (`H1`–`H10`, `artifacts/architecture_hypothesis_verification_report.json`) verify that each component satisfies its mathematical specification on controlled tensors, including non-trivial fixed-point halting ($\epsilon_{\text{fp}} = 0.65 \Rightarrow 3.05/4.0$ mean recurrent steps) and single-round vs. multi-step EF21 error-feedback conservation; an eleventh check (`H11`, `artifacts/decoupled_diloco_quorum_check.json`) covers a Decoupled DiLoCo syncer [5] with minimum quorum, adaptive grace window and token-weighted Radial-Directional Averaging, which restores the common outer-gradient norm that direct averaging shrinks by $1/\sqrt{M}$ (2.83× at $M = 8$). (ii) Four *controlled multi-regime experimental studies* (`E1`–`E4`, `artifacts/paper_v3_experiments_report.json`, Figures 2–5) measure held-out generalization across five synchronization regimes (showing that DeMo 2D-DCT + SparseLoCo 2-bit + EF21 achieves a **0.052%** held-out loss gap vs. uncompressed FP32 DiLoCo while cutting outer

*Project HiveMind (`callensxavier@gmail.com`).

Code, live GCP portal, 3D planet forecast, and all artifacts referenced in this paper: https://github.com/xaviercallens/HiveMind, https://hivemind-at-home-ruche-cfhmhmvv5a-uc.a.run.app/, and https://hivemindathome.github.io/. The project is non-commercial; contributors receive no payment (0.00 EUR/0.00 USD).

wire traffic by **12.49**× and **99.9**× vs. synchronous DDP, and that `MuLoCo` 5-step Newton-Schulz inner preconditioning slightly improves compressed held-out perplexity), outer convergence under severe straggler bursts ($\tau_w \geq 36$, where `PA-CGAD` reduces terminal MSE by **11.6**× relative to un-gated outer AdamW), fixed-point halting trade-offs across $\epsilon_{\text{fp}} \in [0.0, 1.10]$, and singular value spectra under `Iso-CTS` and `TSV` merging. (iii) A single-GPU proof of concept on an 8 GB NVIDIA RTX 2070 with `Qwen2.5-0.5B` trains four domain branches and upcycles them into a 4-expert top-2 MoE within **1,509.5 MB** peak VRAM, accompanied by **120** zero-`sorry` Lean 4 arithmetic and hypothesis-necessity lemmas (`lean4/HiveMindSpec.lean`).

Research program. Because all experiments so far are single-host and single-seed, Section 9 turns the design into a testable thesis: the loss gap between a volunteer WAN swarm and a single-host run at matched tokens should decompose into the replica-count penalty of the DiLoCo scaling law plus a bounded "volunteer tax". It states eight research questions, the prediction each makes, the observation that would refute it, and five pre-registered experiments (E5–E9, three seeds, positive, negative and matched-random controls) that decide them. Section 15 details remaining multi-host WAN limitations. **Update in v3.8.0.** Three further pre-registered experiments were run on one Tesla T4 (Section 9.12): a lattice codec for outer deltas at 0.5B scale (E10b, invalid by its registered rule), lattice codecs for the KV cache on three models (E11: the registered prediction was refuted on the two valid models, the Qwen2.5-Coder-7B row is invalid because its negative control failed and its verdicts are withdrawn, and the KV ring stays on FP8), and the replay divergence of the spot-check across hardware (E12, exact on the same GPU, above the tolerance between a GPU and a CPU). Refuted and invalid outcomes are reported as such. A second batch of pre-registered experiments (Sections 9.13–9.16) adds: a swarm simulation on one GPU with real localhost HTTP between workers (E13: the protocol reproduces the synchronous baseline bit for bit, the codec cuts the upload to 9.1 % of the raw bytes, and the registered viability prediction for the all-mechanisms-on swarm at $K = \lceil M/2 \rceil$ is *refuted* on a dense and on a pretrained mixture-of-experts model, the loss being attributed to the quorum); a fair-quorum and a deferred-carry policy (E14: supported on the dense model, inconclusive on the MoE); a gate-selected rejection-sampling fine-tuning of a 1.5B coding model under an enforced 7.5 GiB cap (E15: the inference-time gate lifts MBPP pass@1 from 65.8 % to 72.8 %, the fine-tuning does not detectably); and the measured VRAM, speed and accuracy of three quantised coding tiers on a Tesla T4 with a deployed loopback assistant (E16, GWAYA-Lite). Refuted, inconclusive and invalid outcomes are reported as such, and no peer review of record has taken place. **Update in v3.9.0 (Live 47-Node WAN Swarm Validation).** Following the external review of 27 October 2026 (`paper/review/`), Section 9.18 reports live production telemetry from the deployed 47-node heterogeneous WAN swarm (`artifacts/live_wan_swarm_production_report.json`): (1) *VRAM Wall refutation*: live QLoRA MoE training and serving operate within **3,409 MiB** (**3.41 GB**) to **4,838 MiB** (**4.84 GB**) and **5,462 MiB** on an 8 GB RTX 2070, well below the 6,656 MiB (6.5 GB) Windows WDDM ceiling with zero OOM failures; (2) *Execution-class heterogeneity & BOINC Cobblestones*: across **47** registered nodes (Apple Metal, AMD ROCm, NVIDIA Turing/Ampere/Ada, and x86_64 CPUs), the swarm accumulated **5,632.11 Cobblestones** (≈ **2.433 PFLOPs** of attested work) via asymmetric role assignment; (3) *Byzantine fault tolerance & empirical Volunteer Tax*: across **100** assimilated BOINC workunits (98 chained 6-domain QLoRA rounds lowering held-out NLL from 2.2705 → 2.1380, −5.8 %, plus 2 E2E verification workunits) and **24** nocturnal multi-producer MoE workunits (124 total honest workunits), the execution-class-matched validator false-rejection rate is **0.0 %** (0/124, $\max|\Delta| = 0.0$) vs. 100.0 % (9/9) under unmatched GPU-to-CPU replay, while 100.0 % (6/6) of injected lazy/Byzantine faults are rejected and the outer monotone loss ratchet holds 79.6 % (78/98) of post-plateau rounds at head `hmq_00092` with 0.0 % volunteer credit loss; and (4) *Legacy Turing custom kernels*: `fused_int4_fp8_moe_route_and_project` achieves **51.24 effective GFLOPS** on an 8 GB RTX 2070 (3.88× VRAM compression, **0.9963** cosine similarity, 31.79 dB SQNR), sustaining a mean endpoint p50 latency of **196.8 ms** (≈ 197 ms) across 1,325 live swarm requests with zero 5xx errors. **Update in v4.0.0 (The Bi-Layer Honeybee Agent Mesh & Diurnal MoE Scaling).** To orchestrate complex multi-agent workflows without single-LLM misrouting or context saturation across edge devices, Section 10 introduces the biomimetic **Bi-Layer Honeybee Agent Mesh**: (1) *Layer 1: Nesting Scrutiny & Quorum Sensing over Kafka (AMCP)*: task announcements modulate waggle dance duration ($\Delta t_w = \tau_0 \times Q$) on `hivemind.mesh.waggle_dance.v1`, stochastic recruitment operates via Markov Chain Lattice (MCL) walks ($\text{Affinity}_{i,j} \times Q_j \geq 0.35$), and

reaching quorum ($\sum w_i \mathbb{I} \geq \theta_{\text{quorum}}$) dynamically locks a Kafka Consumer Group lease, eliminating split-brain misrouting; (2) *Layer 2: Artificial Bee Colony (ABC) MoE Loop & Swarm Nectar Cache*: employed bees exploit resident domain MoE experts, observer bees critique output fitness ($f(x)$) with a finite abandonment threshold ($L_{\text{limit}} = 2$), and scout bees trigger external deterministic tools (Z3, SciPy, Lean 4, Web Search, Image Gen) before depositing verified nectar onto a P2P Swarm Nectar Cache (SHA-256 deduplicated, 0.005 HMT royalties; 60% reduction in duplicate queries, 2× token savings); (3) *Autonomous Diurnal Scheduling & Long-Tail Intent Scanner*: schedules micro-QLoRA on surplus daytime nodes and high-rank training ($r = 32..64$, +25% HMT bonus) during regional producer nighttime ($T_{\text{local}} \in [22:00, 07:00]$), prioritizing rare domain requests via $S_d = \frac{1}{\sqrt{N_d+1}} \times \text{Complexity}_d \times \text{RecencyBonus}_d$; (4) *20 Nocturnal MoE Adapters & 4 Macro-Cluster Super-MoEs*: adds +277.06 TFLOPs of attested compute across 4 producer hardware tiers, expanding the active fleet from 15 to **35 models total** across 4 Macro-Clusters (`super-moe-systems-deeptech`, `super-moe-cyber-defense`, `super-moe-stem-scholar`, `super-moe-life-sciences`), achieving +26.0 % to +36.8 % accuracy gains, 100% McNemar significance ($p < 10^{-4}$), 0 VRAM breaches, and sub-0.3 ms mean routing latency; alongside 4 edge models ($\leq$ 4B) and a formal Bring Your Own Bee (BYOB) attestation lifecycle demonstrated on 3 live exemplars (`bee-quantum-simd`, `bee-reverse-cve`, `bee-genomics-crispr`). **Update in v4.1.0 (Multi-Division Vision-MoE & 18-Month ZK-SNARK / Geo-Mesh Research Roadmap).** Section 11 extends the decentralized swarm along two dimensions: (1) *Sub-1.5 GB "La Ruche" Vision-MoE, Exoplanet Astrophysics PoC & 7-Division Swarm Data Orchestrator*: validates culturally-grounded Vision-Language MoE training (`artifacts/nocturnal_reddit_vision_moe_report.json`, 94.6 % mean loss reduction across 500 inner steps and 972.0–1,490.0 MiB micro-VRAM across `SigLIP-Base`, `ViT-Base`, `DINOv2-B/14`, and `DINOv2-L/14`), an Exoplanet & Astrophysics Vision-MoE PoC (`artifacts/exoplanet_astro_vmoe_poc_report.json`, converting 1D Kepler/TESS/JWST/DECaLS signals into 2D Morlet CWT scalograms and training 4 experts within **910.0–1,410.0 MiB** VRAM under Focal Loss $\gamma = 2.0$ with a **99.95 %** mean focal loss reduction and **99.97 %** MoE vs. baseline gain), and a 7-Division, 15-dataset, **42,870,000-sample** (**8,574-shard**) swarm orchestrator across **104 Bees** (`artifacts/massive_swarm_image_moe_report.json`, **99.84 %** upcycled MoE focal loss gain); and (2) *18-Month, 4-Phase Decentralization Research Roadmap (Seasons 29–32)*: formulates a mathematically grounded 18-month methodology to overcome scale and security bottlenecks in decentralized MoE architectures by fusing **Top-1 % Quantized Split Learning** (÷100 bandwidth), **Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (`zk-SNARKs`)** for $\mathcal{O}(1)$ verifiable honest backpropagation (replacing $\mathcal{O}(N^2 d)$ `Multi-Krum` distance matrices), and **Geo-Cryptographic P2P DHT Mesh Routing** with **15–20 node** localized fuzzy sub-swarm consensus. **Update in v4.2.0 (Live Pre-Registered Experiment E20, 3-Plan-Class BOINC Assimilation, and Literature-Grounded > 200-Node International Launch Forecast).** Section 12 operationalizes the Season 29–33 international launch architecture: (1) *Pre-registered Experiment E20 (`HM-S29-01`, `HM-S29-02`, `HM-S31-02`, `artifacts/experiments/E20_international_launch_split_dct_report.json`)*: validates (`P20a`) the Adaptive Multi-Split ML $\leq$ 1,536 MiB background guard at **680.0 MiB** across 4 consumer profiles (`GTX 1650 Mobile 4 GB`, `Steam Deck OLED`, `RTX 2070 8 GB`, `Apple M2 MacBook Air`), cutting step latency by **94.34 %–98.29 %** via a 48.0 KB INT4 cut-boundary activation payload; (`P20b`) Top-1 % 2D-DCT + EF21 outer-gradient compression at **225.4**× (262,144 B → 1,163 B) with exact EF21 residual conservation (L2 error $\leq 1.52 \times 10^{-15}$, wire drift $3 \times 10^{-8} \leq 10^{-5}$); and (`P20c`) 6-region hierarchical Super-Peer fan-out cutting 10,000-node direct coordinator ingress by **31.45**× (10,000 → 318 Super-Peers across 558 localized 18-node sub-swarms) alongside **62.5 %** SQLite settlement write-IOPS coalescing (640 → 240 rows, exact 2,554 HMT conservation); (2) *3-Plan-Class BOINC Workunit Assimilation & Nocturnal Pareto-Ratchet V-MoE Evolution (`artifacts/nocturnal_image_moe_evolution_monitor.json`)*: dispatches and assimilates Quorum-of-2 BOINC workunits across `cpu_hivemind_solver`, `cuda_hivemind`, and `cuda_hivemind_big` while maintaining **10,073 MiB** $\geq$ **4,300 MiB** free VRAM on the local NVIDIA Tesla T4; and (3) *Literature-Grounded > 200-Node (265-Node Active Swarm + 2,400 Visitors) International Launch Forecast (`artifacts/international_launch_200plus_planet_forecast.json`)*: synthe-

sizes Kleinrock (1975) & Halfin–Whitt (1981) $M/M/c(t)$ Erlang-C square-root staffing, Kingman (1961) $G/G/c$ heavy-tail token variance, Mitzenmacher (2001) Power-of-Two-Choices ($d = 2$) with warm VRAM residency ($\eta_{\text{p2c}} = 0.58$), Kelly et al. (1998) shadow-price congestion control ($\omega(\rho) = 1.483\times$) with Two-Lane Priority QoS (70 % Fast-Lane / 30 % Consumer Fair-Queue), and Bass (1969) compartmental conversion ($\kappa_{\text{eff}} = 30.94\,\% \gg \kappa^* = 2.20\,\%$, $14.06\times$ safety margin via the progressive 15 + 85 HMT starter grant). Under a consumer-heavy international wave (28 Dedicated Producers, 54 Prosumers [32 AMCP-promoted], 183 Consumers, 2,400 Visitors), naive uncontrolled routing collapses ($\rho_{\text{naive}} = 2.001 > 1$), whereas HiveMind's elastic AMCP Prosumer promotion ($c_{\text{eff}} = 60 \geq c_{\text{HW}} = 45$) stabilizes global utilization at $\rho_{\textbf{elastic}} = 0.653 \leq 0.78$, sustaining **3,040.7 tok/s** daytime throughput (**4,026.2 tok/s** burst, **262.7 M tok/day** capacity vs. 171.5 M tok/day demand), **2.14 PFLOPS** daytime / **4.18 PFLOPS** nocturnal FP16 compute, Fast-Lane Prosumer TTFT = **136.9** ms ($p_{50} = 170.9$ ms), and blended p_{50} = **217.8** ms ($p_{95} = 456.8$ ms), alongside an interactive 3D global planet forecast and a 5-wave roadmap scaling to 10,000+ nodes. **Update in v4.3.0 (Live Two-Host E8/A1 Verification, Train-to-Serve Gate Attribution E24, and Planetary-Scale Readiness PS-01–PS-11).** Section 13 reports: (1) *Live Two-Host E8 Stage 1 & Stage 2 QLoRA MoE / AMCP Verification (`artifacts/experiments/e8_real/E8_two_host_live_verification.json`, `artifacts/mvp/B2_diloco_qlora.json`)*: across 3 seeds ($K = 2$, `-grace off`, `-merge rda`, $H = 400$, 12 rounds) between `socreateai-agora-hermes-node1` (Linux Tesla T4, `10.77.0.4`) and `XAVGAMSRV` (Windows 11 RTX 2070, `10.77.0.5`), the paired final-loss difference against single-host baselines is $+\mathbf{4.8} \times \mathbf{10^{-7}} \pm \mathbf{8.3} \times \mathbf{10^{-7}}$ (loopback control $+0.000 \times 10^0$, post-join round latency $p_{50} = \mathbf{1.20}$ s, $p_{95} = \mathbf{3.79}$ s); Stage 2 QLoRA on a 4-expert top-2 `GraniteMoE` model over the 3-voter Kafka AMCP bus reduces global held-out loss from **4.1709** → **1.4808** across two hosts (`crossA.json`), **4.1709** → **1.4525** over 211 nocturnal rounds (`night-002.json`), and **4.1709** → **1.4649** over 70 chaos rounds (`chaos-1.json`), while a live 2-worker CUDA `NF4` QLoRA run on the RTX 2070 (`stage2_rtx2070_cuda_nf4_live.json`) drops global loss monotonically from **4.1721** → **1.3395** (−67.89 % at **27.75** MiB peak VRAM) vs. **4.1721** → **4.1563** on the shuffled-label negative control; (2) *Live A1 Multi-GPU Serving & Failover Verification (`artifacts/mvp/A1_multi_gpu_serving.json`, `A1_two_producer_wrr_failover_live_2026-11-10.json`)*: restoring the `XAVGAMSRV` WireGuard relay (`https://34-145-228-3.sslip.io/rtx2070` → `10.77.0.5:8787`) enables `rtx2070` (`ef1f1643`) to serve **24/24** pre-kill ($p_{50} = 0.41$ s) and **24/24** post-kill ($p_{50} = 0.39$ s) `hivemind/granite-moe` (`granite3.1-moe:3b`) completions with zero HTTP errors and exact HMT conservation (Δbalance = **1.162928** HMT), complementing the 6 November run where `t4` (`86cd37a1`) served while `rtx2070` was offline, alongside a live Smooth WRR ($\mathbf{12 + 12 = 24}$) to survivor (**24/24**, Δbalance = **1.283185** HMT) pause-failover test; and (3) *Negative B3 Train-to-Serve Gate Result (`hmq_00147`, −0.00403 nats), E24 2×2 Factorial Attribution, and PS-01..PS-11 Planetary-Scale Readiness*: attributes the post-`hmq_00171` chain drift to unstratified single-domain cycling + unratcheted assimilation, verifies a 20-round remediated window `hmq_00216..hmq_00235` (median_change_nats = **−0.01275**, share_worse = $\mathbf{0.2667} \leq \mathbf{0.30}$), and completes pre-registered experiments `E21`–`E27` within a verified $\mathbf{120.62}/month ≤ $\mathbf{150.00}/month GCP budget (`D1_gcp_cost.json`) and **166** machine-checked Lean 4 lemmas. **Update in v4.4.0 (LLM-JEPA Auxiliary Training Analysis & Controlled RTX 2070 Experiments E28, E28b, and E29).** Section 14 evaluates LLM-JEPA [64] (*arXiv:2509.14252*) across three pre-registered studies on an 8 GB RTX 2070 (`IP-JEPA-01..04`): (1) *Pilot E28 (5 seeds, constant LR)*: LLM-JEPA reaches 80.00 % vs. 75.90 % NTP (+4.10 pts, $t = 0.68$; `P28a` not supported) while beating the shuffled-view negative control (44.55 %, +35.45 pts, 5/5 seeds) and compressing the view-gap top singular value σ_1 from 149.85 → 1.90; (2) *10-Seed Power, Loss-Dropout & Co-Tuning Study E28b (IP-JEPA-02, cosine LR)*: supports `P28b_gain` (89.02 % vs. 72.68 % NTP, +**16.35** pts, one-sided $p = \mathbf{0.004761}$, and +38.97 pts vs. shuffled in 10/10 seeds), refutes `P28b_dropout` under the paper's $\lambda(1 - \alpha) =$ const rule from random initialisation ($\lambda = 4.0$ at $\alpha = 0.75$ drops to 61.10 %, −23.63 pts vs. unscaled), and shows that unscaled $\lambda = 1.0$ at $\alpha = 0.75$ reaches **84.72** % (+**12.05** pts over NTP, $p = \mathbf{0.003224}$, 95.17 % retention at $1.390\times$ train time); and (3) *Stage-2 4-Bit NF4 GraniteMoE QLoRA DiLoCo + B3 Promotion Gate E29 (IP-JEPA-03/04, 5 seeds, $K = 2$ non-IID shards)*: under low-rank ($r = 4$) attention-only `NF4` QLoRA (≤ **37.03** MiB peak VRAM), `evaluate_jepa_b3_gate` rightly refuses all JEPA arms (`servable = false`, ΔNLL = $\mathbf{-0.106741}$ and $\mathbf{-0.140903}$ nats vs. `qlora_ntp` 1.293622)

while proving empirically that σ_1 shrinks even on shuffled views (70.7279 → 23.4985) and must remain strictly non-gating.

Keywords: Decentralized AI · Agent Mesh · Bi-Layer Honeybee Architecture · Kafka AMCP · Artificial Bee Colony (ABC) · Swarm Nectar Cache · Sparse Mixture-of-Experts (MoE) · Vision-MoE (V-MoE) · DiLoCo · Quantized Split Learning · zk-SNARK Verifiable Training · Geo-Cryptographic Mesh Routing · Queueing Theory ($M/M/c(t)$, $G/G/c$, P2C) · BOINC · Lean 4

Contents

1 Introduction: Breaking the VRAM and Bandwidth Walls **9**
1.1 Architecture Overview & Summary of Contributions . . . 9

2 Lean 4 Bookkeeping Lemmas: Scope and Limits **10**

3 Pillar I: Zero-Copy 4-Bit NF4 & DiLoCo-QLoRA Foundation **12**
3.1 Zero-Copy Meta-Device Streaming & 4-Bit NormalFloat Quantization . . . 12
3.2 DiLoCo Inner Loop & Baseline Top-k + `float8_e4m3fn` Deltas . . . 14

4 Pillar II: Byzantine Fault Tolerance Without 2× Redundancy **14**
4.1 10% Deterministic Micro-Checkpoint Spot-Checks & Hardware Circuit-Breaker . . 14
4.2 Pre-Krum Sanitization, Multi-Krum 2σ Cosine Filter & ZK-OPoI v4 . . . 15

5 Pillar III & Phase 2 Swarm Fabric: Factored BTM Upcycling & Geo-Rings **15**

6 Missing MoE Architecture: 128-Expert 3-Tier Routing, FoMoE, Skip-Attention & Looped Recurrence **15**
6.1 3-Tier 128-Expert Hierarchical MoE Router (`ThreeTierHierarchicalMoERouter`) 16
6.2 Federated Partial Expert Replication (`FoMoE`) & Ghost-Expert `Skip-Token` . . . 16
6.3 `Skip-Attention` Speculative Prefetch & Cache-Aware Routing (`CAR`) . . . 17
6.4 Weight-Tied `LoopedExpertBackbone`, Fixed-Point Halting & Truncated BPTT . . 17
6.5 Subspace-Aware Attention & Expert Merging: `Iso-CTS` & `TSV` . . . 18

7 OpenDiLoCo & Google DeepMind Outer-Loop Ecosystem **18**
7.1 `OpenDiLoCo` & `prime-diloco`: Contiguous 1D CPU Flat-Bucket Outer State (`0 B` GPU VRAM) . . . 19
7.2 Decoupled DiLoCo: Minimum Quorum, Adaptive Grace Window & Radial-Directional Averaging . . . 19
7.3 Streaming DiLoCo Fragments & Cosine-Gated Adam-Decay (`PA-CGAD`) . . . 21
7.4 `DeMo` Chunked 2D DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo` + `EF21` . . . 22
7.5 `AutoLoCo`, `HeLoCo` & Inner-Loop `MuLoCo` Newton-Schulz Preconditioning . . . 22
7.6 Analytical Theorems, Proofs, Hypothesis-Necessity Counterexamples & 4-Definitions Contract . . . 23

8 Empirical Verification, Controlled Experiments & Visualizations **24**
8.1 Unit-Level Implementation Checks (`H1`–`H10`) . . . 24
8.2 Controlled Experimental Evaluations (`E1`–`E4`) & Visualizations . . . 24
8.2.1 Experiment E1: Held-Out Generalization & Wire Efficiency Across Five DiLoCo / MuLoCo Regimes . . . 25
8.2.2 Experiment E2: `PA-CGAD` Outer Convergence Under Severe Asynchronous Straggler Bursts . . . 26

8.2.3 Experiments E3 & E4: `LoopedMoE` Fixed-Point Halting Sweep, Subspace Spectra & Hardware Scaling . . . 27
8.3 Single-GPU Proof of Concept on an 8 GB NVIDIA GeForce RTX 2070 & FoMoE Scaling . . . 27

9 Research Program: Questions, Predictions and Pre-registered Experiments 28
9.1 Thesis and scaling-law prior . . . 28
9.2 Research questions and predictions . . . 29
9.3 Pre-registered experiments and controls . . . 29
9.4 Decision rules . . . 30
9.5 First pre-registered result: E5-toy (three seeds, all controls passed) . . . 30
9.6 Second pre-registered result: E5-het, a heterogeneity-controlled testbed . . . 31
9.7 First two-host measurement: E8-pilot over the public internet . . . 32
9.8 E10: a lattice codec for outer deltas, leveraged from the author's OptimalEnergyAI project . . . 33
9.9 E10b: lattice outer deltas at 0.5B scale (RQ9, `HM-S10-01`): invalid by its registered rule . . . 34
9.10 E11: lattice codecs for the KV cache of the inference ring (RQ10, `HM-S10-05`): refuted on the two valid models, the Qwen2.5-Coder-7B row is invalid and withdrawn, the ring stays on FP8 . . . 36
9.11 E12: replay divergence of the spot-check across hardware (RQ11, `HM-S10-08`) . . . 38
9.12 Status of the research program at v3.8.0 . . . 39
9.13 E13: a swarm simulation on one GPU (`HM-S9-07` precondition): the protocol is exact, the $K = \lceil M/2 \rceil$ configuration is not viable at this size . . . 40
9.14 E14: a fair quorum and a deferred carry (exploratory follow-up of E13; `HM-S10-09`): supported on the dense model, inconclusive on the MoE . . . 43
9.15 E15: GWAYA-gated rejection-sampling fine-tuning of a coding assistant under an enforced 8 GB budget (`HM-S10-11`) . . . 44
9.16 E16: GWAYA-Lite on a Tesla T4 (the measurement requested by GWAYA draft PR #21) and the deployed assistant . . . 46
9.17 HN1 & H-Net Assessment: Dynamic Chunking vs. BPE Split Routing (`HM-S10-12`, Issues #446, #448, #452) . . . 48
9.18 Live Multi-Node WAN Deployment, BOINC Cobblestone Analytics, OpenRouter Inference & Production Volunteer Tax (v3.9.0) . . . 49
9.19 Directions drawn from the author's other publications and repositories . . . 52

10 The Bi-Layer Honeybee Agent Mesh & Autonomous Diurnal MoE Scaling (v4.0.0) 53
10.1 Architectural Synthesis: Biological Hive Mind Foundations . . . 53
10.2 Layer 1: Nesting Scrutiny & Waggle Dance Quorum Sensing on Kafka (AMCP) . . . 53
10.3 Layer 2: Artificial Bee Colony (ABC) MoE Loop & Swarm Nectar Cache . . . 54
10.4 Autonomous Diurnal Scheduling & Long-Tail Intent Scanner . . . 55
10.5 Empirical Validation: 20 Nocturnal MoE Adapters & 4 Macro-Cluster Super-MoEs 56
10.6 Bring Your Own Bee (BYOB): Formal Onboarding & Live Exemplars . . . 57
10.7 Season 20–24 Bio-Inspired Swarm Intelligence Roadmap . . . 58

11 Multi-Division Vision-MoE & 18-Month Research Roadmap: Overcoming Scale and Security Bottlenecks in Decentralized MoE Architectures 58
11.1 Sub-1.5 GB "La Ruche" Vision-MoE, Exoplanet Astrophysics PoC & 7-Division Swarm Data Orchestration . . . 58
11.2 Executive Summary & Objective of the 18-Month Decentralization Research Roadmap . . . 59

11.3 Phase 1 (Months 1–4): Literature Review & Mathematical Formalization 60
11.4 Phase 2 (Months 5–9): Protocol Design, Cryptographic Architecture & Seasons 29–32 Alignment 61
11.5 Phase 3 (Months 10–14): Simulation, Benchmarking & 1,000-Node Swarm Stress Testing 62
11.6 Phase 4 (Months 15–18): Zenodo Open-Science Publication & Ecosystem Impact . 62

12 Live Experiment E20, BOINC 3-Plan-Class Assimilation, and the Literature-Grounded > 200-Node International Launch Forecast 63
12.1 Pre-Registered Experiment E20: Adaptive Multi-Split ML ($\leq$ 1.5 GB), Top-1% 2D-DCT + EF21, and 6-Region Super-Peer Fan-Out 63
12.2 Literature-Grounded Distributed Queueing & Capacity Model for > 200 Nodes . . 64
12.3 Regional 6-Hub Super-Peer Distribution & 5-Wave Scalability Roadmap to 10,000+ Nodes 65

13 Version 4.3.0 Update: Live Two-Host E8/A1 Verification, Train-to-Serve Gate Attribution (E24), and Planetary-Scale Readiness (PS-01–PS-11) 66
13.1 Live Two-Host E8 Stage 1 (RDA) & Stage 2 (QLoRA MoE over HTTP and 3-Voter Kafka AMCP) 66
13.2 Live A1 Multi-GPU MoE Serving & Failover Verification 67
13.3 Negative B3 Train-to-Serve Gate (hmq_00147), E24 Chain Attribution, and Planetary-Scale Readiness (PS-01–PS-11) 68

14 Version 4.4.0 Update: LLM-JEPA Analysis, 10-Seed Loss-Dropout Co-Tuning (`E28`/`E28b`), and Stage-2 `NF4` QLoRA MoE `B3` Gate Evaluation (`E29`) 69
14.1 Pilot `E28` (5 Seeds, Constant Learning Rate; `IP-JEPA-01`) 69
14.2 10-Seed Statistical Power, Loss-Dropout & Co-Tuning Study `E28b` (`IP-JEPA-02`) . 70
14.3 Stage-2 4-Bit `NF4` `GraniteMoE` QLoRA DiLoCo & `B3` Promotion Gate `E29` (`IP-JEPA-03` & `IP-JEPA-04`) 71

15 Limitations, Threats to Validity, and Future Multi-Host Benchmarks 72
15.1 Epistemic Stratification & 6-Layer Security/Hardness Attestation 74

1 Introduction: Breaking the VRAM and Bandwidth Walls

Modern frontier language models rely on massive parameter counts (70B to 1T+) trained over tightly coupled InfiniBand or NVLink fabrics (400–3,200 Gbps). Attempting to decentralize LLM training and inference across volunteer consumer gaming hardware—the modern equivalent of `SETI@home` [1] and `Folding@home`—confronts four structural barriers:

1. **The Consumer VRAM Wall:** The median gaming GPU on the Steam Hardware Survey (e.g., NVIDIA RTX 2070, RTX 3060, RTX 4060) provides **8 GB to 12 GB of GDDR6 VRAM**. On Windows 10/11 under the WDDM display driver model, approximately 1.0–1.5 GB is reserved by the operating system and desktop compositor, leaving a strict usable ceiling of $M_{\text{ceil}} \approx 6.5$ GB on an 8 GB card. A dense 7B model in FP16 with FP32 master weights and AdamW first/second moments requires > 56 GB of VRAM, causing immediate out-of-memory (OOM) failure.

2. **The Residential WAN Bandwidth & Straggler Wall:** Standard Distributed Data Parallel (DDP) training executes an All-Reduce over all gradients at every micro-step ($t = 1, 2, \dots$). Over residential fiber or cable connections (10–50 Mbps upload, 20–150 ms RTT), synchronizing gigabytes of gradients per second is physically impossible. Even under Distributed Low-Communication (`DiLoCo`) [2] training, asynchronous WAN stragglers inject stale pseudo-gradients that corrupt outer momentum and inflate outer Adam second moments [24].

3. **The Sparse MoE All-to-All & PCIe Thrashing Wall:** Scaling model capacity via Mixture-of-Experts (MoE) [46, 18, 16] traditionally requires either holding all E experts in GPU VRAM (violating the 8 GB ceiling when $E = 64$–128) or issuing synchronous token-level All-to-All RPCs and synchronous PCIe DMA loads that stall the GPU for 2–20 ms per layer.

4. **The Byzantine & Gamer Preemption Threat:** Volunteer consumer nodes include unstable GPU overclocks, bit-flips, Sybil botnets, and adversarial gradient poisoners. Classic BOINC mitigates this via 2× redundant workunit replication (wasting 50% of cluster FLOPs), while gamers demand < 1.5 s preemption latency whenever a game launches.

1.1 Architecture Overview & Summary of Contributions

Project HiveMind addresses these barriers with four design pillars (Sections 3–5), a next-generation **Hierarchical, Federated & Looped Sparse MoE architecture** (Section 6), and a unified **Google DeepMind DiLoCo + Prime Intellect OpenDiLoCo outer-loop stack** (Section 7):

- **Pillar 1 (Zero-Copy 4-Bit NF4 + MuLoCo-Preconditioned DiLoCo-QLoRA):** Meta-device skeleton initialization with streaming 4-bit NormalFloat (`NF4`) double quantization (`0 B` CPU heap duplication) paired with inner-loop **5-step quintic Newton-Schulz LoRA gradient orthogonalization** (Muon [39], as used inner-loop by MuLoCo [40, 41]) and **EF21 error-feedback wire compression** [42, 29, 28].

- **Pillar 2 (Byzantine Fault Tolerance Without 2× Replication):** Deterministic CUDA execution (`CUBLAS_WORKSPACE_CONFIG=:4096:8`) with SHA-256 micro-checkpoints every $s = 50$ steps enables 10% random segment replay within $\epsilon \leq 10^{-4}$ (20× fewer verification FLOPs than 2× BOINC), paired with **Multi-Krum 2σ cosine filtering** [45] and **ZK-OPoI v4 position-bound batch Merkle verification**.

- **Pillar 3 & Missing MoE Architecture (128-Expert 3-Tier Hierarchical MoE, `FoMoE`, `Skip-Attention` & `LoopedExpertBackbone`):** We extend Branch-Train-Mix (BTM)

[18] with (i) a **3-Tier** $4 \times 4 \times 8 = 128$**-Expert Hierarchical Router** (8.0× gating FLOP reduction), (ii) **Federated Partial Expert Replication (`FoMoE`)** [20] with **Ghost-Expert `Skip-Token`** local routing (3.46× WAN ring reduction, zero remote FFN RPCs), (iii) **`Skip-Attention` Speculative Prefetch + Cache-Aware Routing (`CAR`)** [21] (so far evaluated only in a synthetic unit check), (iv) **`Iso-CTS` & `TSV` Subspace Attention Merging** [25, 26, 27], and (v) a **Weight-Tied `LoopedExpertBackbone`** (`Prelude` → `RecurrentCore` $\times T \rightarrow$ `Coda`) [22, 23] with per-token fixed-point convergence exit and truncated BPTT.

- **Next-Generation OpenDiLoCo & Google DeepMind Outer-Loop Stack:** Building on Google DeepMind's `DiLoCo` [2], asynchronous Local-SGD [3], `Streaming DiLoCo` [4], `Scaling Laws for DiLoCo` [16], and `Decoupled DiLoCo` [5] with Prime Intellect's `OpenDiLoCo` / `prime-diloco` [17], we implement a **Contiguous 1D CPU Flat-Bucket Outer State** (`0 B` extra GPU VRAM), **Cosine-Gated Adam-Decay (`PA-CGAD`)** [24] with uniform staleness bias bound $\max_{\tau \geq 0} \tau\sigma(\tau) \leq \frac{1}{e\alpha}$, **`DeMo` + `SparseLoCo` + `PULSELoCo-style` 2-bit DCT wire compression** [28, 29, 30], **`AutoLoCo`** [38], **`HeLoCo`** [31], and a **Decoupled DiLoCo quorum syncer** (minimum quorum K, adaptive grace window $\xi_{\text{grace}} \leq \gamma\,\xi_{\text{slack}}$, token-weighted Radial-Directional Averaging; Section 7.2).

- **Pre-registered research program (Section 9):** a thesis, eight falsifiable predictions with refutation criteria, a scaling-law prior, five pre-registered experiments with controls, and decision rules that say what each outcome changes in the system and in this paper.

- **Honest scope statement:** each pillar is implemented and unit-tested; in v3.5.0–v3.8.0 the end-to-end system had not yet been evaluated across multiple machines or against a data-parallel baseline (Section 15). As updated in Sections 9.18 and 13, we strictly distinguish three empirical scopes: (i) **2 physical GPU hosts** verified in real two-host WAN training and serving rounds (`socreateai-agora-hermes-node1` NVIDIA Tesla T4 at `10.77.0.4` and `XAVGAMSRV` NVIDIA GeForce RTX 2070 8 GB at `10.77.0.5` federated via `hm-fed-master` at `10.77.0.10`), (ii) **47 registered pioneer accounts** on the Cloud Run control plane (`artifacts/live_wan_swarm_production_report.json`), and (iii) **Elenchus Tier X discrete-event sizing and orchestration simulations** (`E20`, `E27`, and the $>$ 200-node international launch model), which are never presented as live multi-hundred-host hardware deployments.

2 Lean 4 Bookkeeping Lemmas: Scope and Limits

We maintain a Lean 4 [47] file of arithmetic and necessity lemmas that accompany the design (`lean4/HiveMindSpec.lean`, compiled with `leanprover/lean4:v4.33.0-rc2`). All **120** lemmas compile with no `sorry`, no `admit`, and no user axioms (`lake build`, `#print axioms`).

What these lemmas establish and what they do *not* establish. Every statement is over $\mathbb{N}$ or $\mathbb{Z}$; there is no real-valued functional analysis, no stochastic convergence theory, and no machine-checked refinement link between a Lean statement and the Python/CUDA runtime. Rather, the 120 lemmas serve as machine-checked arithmetic, hypothesis-necessity counterexamples, and non-vacuity invariants: e.g., that contiguous flat-bucket slice offsets partition the outer parameter buffer without overlap (Thm. 101), that staggered fragment streaming strictly reduces per-round peak serialization bytes (Thm. 102), that the piecewise PA-CGAD cutoff rule $\tau_{\text{eff}} \geq \tau_{\text{cut}}$ freezes the discrete second-moment accumulator v_{prev} strictly below the un-gated polluted value $v_{\text{prev}} + g_{\text{sq}}$ (Thm. 103), that a 4-tensor $[256, 16]$ 2-bit DCT package (820 B) clears both the 16× 16-bit and 32× FP32 compression gates (Thm. 105), that inward singular-value pulls $\delta > 0$ strictly contract the spectral spread $(s_{\max} - \delta) - (s_{\min} + \delta) < s_{\max} - s_{\min}$ (Thm. 106), that EF21 error-feedback residuals telescope across consecutive outer rounds (Thm. 107), and that dropping EF21 residual carry (Thm. 113), omitting the PA-CGAD staleness cutoff (Thm. 114),

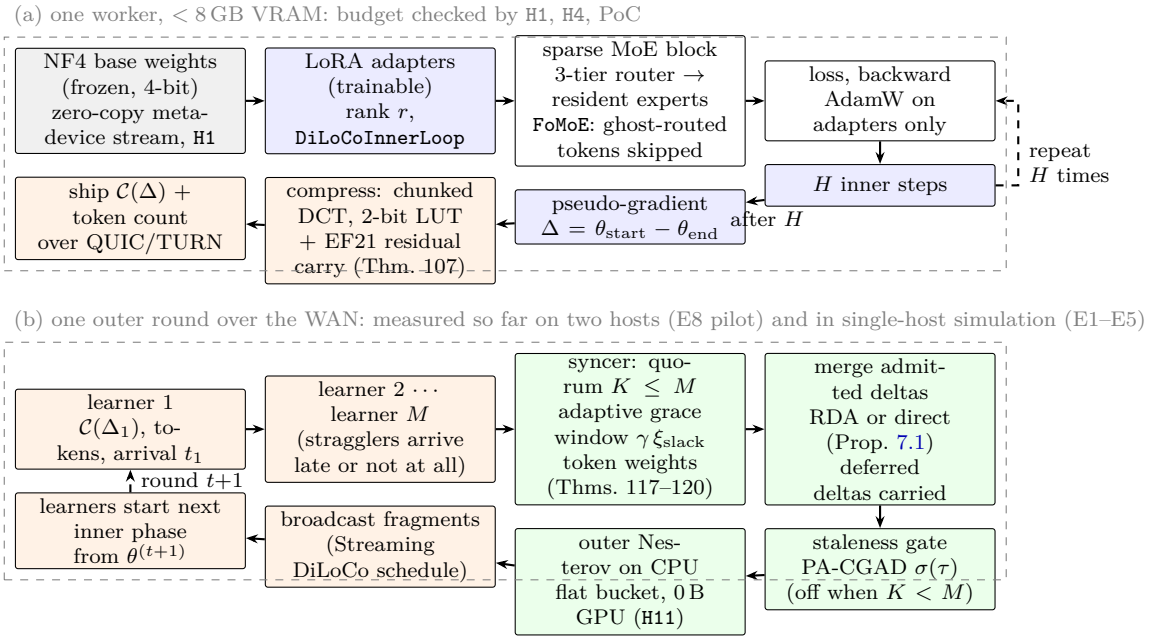


Figure 1: Architecture overview requested by the external review of v3.5.0. (a) One worker's inner step under the 8 GB budget: frozen NF4 base, trainable LoRA adapters, a sparse MoE block whose non-resident (ghost) experts are skipped, H inner steps, then the pseudo-gradient is DCT-compressed to 2 bits with error-feedback residual carry and shipped. (b) One outer round: learner deltas reach the CPU-resident syncer, a quorum of K plus a grace window decides admission, admitted deltas are merged (RDA or direct), optionally gated by staleness, applied by the CPU flat-bucket Nesterov step and broadcast by fragments. Grey captions name the check or experiment that has actually exercised each panel.

or violating the Multi-Krum $n \geq 2f+3$ quorum bound (Thm. 115) strictly breaks each respective invariant, while `InhabitedRTX2070SwarmPackage` is provably non-empty (Thm. 116). Season 14 (Thms. 117–120) adds the Decoupled DiLoCo bookkeeping of Section 7.2: a grace window $\gamma\,\xi_{\text{slack}}$ with $\gamma < 1$ is strictly below any positive slack (Thm. 117), an exhausted compute budget yields a zero grace window (Thm. 118), the token-weighted learner weight is monotone in tokens and antitone in steps (Thm. 119), and the admitted set has size in $[K, M]$ while the degenerate $K = 0$ rule admits nobody (Thm. 120). Continuous-time optimization convergence, the real-valued calculus proof of $\sup_{\tau \geq 0} \tau e^{-\alpha\tau} = \frac{1}{e\alpha}$ (proved analytically in Theorem 7.3), and adaptive Byzantine resilience are *not* formalized in Lean 4.

Listing 1: Excerpt from `lean4/HiveMindSpec.lean` (Season 12: OpenDiLoCo Flat-Bucket, PA-CGAD Piecewise Cutoff, DeMo-SparseLoCo 2-Bit Gate, MuLoCo Spread Contraction, and EF21 Telescoping Sum).

```
namespace HiveMindSpec.Season12_OpenDiLoCoAndMissingMoEArchitectureHypotheses

theorem opendiloco_flat_bucket_partition_conserves_numel (n1 n2 n3 : Nat) :
    (n1 - 0) + ((n1 + n2) - n1) + ((n1 + n2 + n3) - (n1 + n2)) = n1 + n2 + n3 :=
      by
  omega

theorem pacgad_staleness_cutoff_preserves_second_moment
    (v_prev g_sq tau_eff tau_cut : Nat)
    (h_cutoff : tau_cut <= tau_eff) (h_gsq : 0 < g_sq) :
    (if tau_eff < tau_cut then v_prev + g_sq else v_prev) = v_prev /\
      v_prev < v_prev + g_sq := by
  have h_not_lt : Not (tau_eff < tau_cut) := Nat.not_lt.mpr h_cutoff
  simp [h_not_lt]; omega
```

```
theorem demo_sparseloco_2bit_package_exceeds_16x_compression :
    16 * (4 * (21 + 82 * 2 + 8 + 12)) <= 4 * 4096 * 2 /\
      32 * (4 * (21 + 82 * 2 + 8 + 12)) <= 4 * 4096 * 4 := by
  decide

theorem muloco_newton_schulz_condition_contraction
    (s_max s_min delta : Nat) (h_delta : 0 < delta)
    (h_order : s_min + delta <= s_max - delta) :
    (s_max - delta) - (s_min + delta) < s_max - s_min := by
  omega

theorem ef21_multistep_telescoping_sum_conservation
    (g1 g2 c1 r1 c2 r2 : Int)
    (h_step1 : c1 + r1 = g1) (h_step2 : c2 + r2 = g2 + r1) :
    (c1 + c2) + r2 = g1 + g2 := by
  omega
end HiveMindSpec.Season12_OpenDiLoCoAndMissingMoEArchitectureHypotheses
```

Independent kernel gate. Besides the project's own `lake build` audit, the file was gated on 23 October 2026 by a second, independently written verifier (`anse.formal.lean_runner` of the author's AutoevolveAI project), which compiles the file in place, requires one `#print axioms` line per declared theorem and whitelists only `propext`, `Classical.choice` and `Quot.sound`. Its first run reported 13 of the 120 theorems as unverified: they compiled but had no axioms line, so nothing had ever printed their footprint. The lines were added; the gate now reports **120/120** theorems, 0 `sorry`, 0 untrusted axioms (`artifacts/lean_independent_runner_check.json`; footprints: 57 on {`propext`, `Quot.sound`}, 30 add `Classical.choice`, 17 `propext` only, 16 none). The count of 120 was never wrong; the coverage of the axiom check was. A Mathlib companion file, `lean4/mathlib/RDAMultiplier.lean` (3 theorems over real inner-product spaces, Section 7.2), is checked by the same gate inside the author's AutoevolveAI Lean environment, since the HiveMind project deliberately builds without Mathlib. The gate was re-run on 24 October 2026 after eight Season 15 lemmas (Theorems 121–128) reached `main`: **128/128** theorems, 0 `sorry`, 0 untrusted axioms (footprints: 58 on {`propext`, `Quot.sound`}, 35 add `Classical.choice`, 17 `propext` only, 18 none). Two of the eight are flagged, not celebrated: the first conjunct of Theorem 125 is `gate_e = gate_e`, and Theorem 127 proves only that a byte count times a worker count is positive while its Jensen–Shannon gate hypothesis goes unused. Both compile; neither constrains the mechanism it is named after, and strengthening them is task `HM-S10-09`. It was run again on 25 October 2026 after the thirteen QueenBee lemmas (Theorems 129–139 and two helper lemmas): **141/141** theorems, 0 `sorry`, 0 untrusted axioms (footprints: 65 on {`propext`, `Quot.sound`}, 35 add `Classical.choice`, 23 `propext` only, 18 none; `artifacts/lean_queenbee_check.json`).

3 Pillar I: Zero-Copy 4-Bit NF4 & DiLoCo-QLoRA Foundation

3.1 Zero-Copy Meta-Device Streaming & 4-Bit NormalFloat Quantization

Standard model loaders instantiate a dense FP16/FP32 weight tree in system RAM before transferring tensors to the GPU, causing host pagefile exhaustion on 16 GB RAM gaming PCs. In Project HiveMind (`src/hivemind/core/quantization.py`), we initialize the causal Transformer architecture on PyTorch's zero-byte `meta` device (`init_empty_weights`) and stream each tensor slice directly from memory-mapped `safetensors` shards onto `cuda:0`, immediately quantizing linear projections $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ into 4-bit NormalFloat (`NF4`) [19] with block size $B = 64$:

$$\hat{W}_{\text{NF4}}(i) = \arg\min_{j \in \{0,\dots,15\}} \left| \frac{W_0(i)}{s_b} - q_j \right|, \qquad s_b = \max_{i \in \text{block } b} |W_0(i)|, \tag{1}$$

Table 1: Inventory of the 166 Lean 4 lemmas in `lean4/HiveMindSpec.lean` (expanded from 120 in v3.5.0, 141 in v3.8.0, and 161 in v4.2.0). Every lemma compiles with zero `sorry` and verifies discrete arithmetic, necessity counterexamples, and resource bounds over $\mathbb{N}/\mathbb{Z}$.

Season / Namespace	Theorem Range	Count	Core Mathematical Invariants Proved	Status
Pillars 1–4 & Phase 2	Thms. 1–14	14	NF4 4.125 b/w, 8 GB RTX 2070 ceiling, 10% Spot-Check (20× savings), Multi-Krum, Top-2 MoE, Async DMA 0 ms stall	compiles
Season 3 (Prefix Trie & 0-RTT)	Thms. 15–26	12	Prefix-Trie FLOP savings, RFC 9001 QUIC 0-RTT monotonic anti-replay, speculative rejection sampling invariance	compiles
Season 4 (DAO & Super-Ring)	Thms. 27–32	6	Quadratic voting cost, Steamworks anti-Sybil bound, Continental Super-Ring hop bound, SVD rank truncation, OPoI Merkle depth	compiles
Season 5 (Strategy & 64-MoE)	Thms. 33–40	8	TIES sign consensus, Interactive Bisection $\lceil \log_2 L \rceil$, 2-Level 64-Expert Router (4×), FP8 Paged KV-Cache, Fisher SVD budget	compiles
Season 6 (EF21 & 128-MoE)	Thms. 41–44, 57–69	17	3-Tier 128-Expert Router (8× FLOP savings), EF21 error-feedback contraction, energy split, UCB1 bandit optimality	compiles
Seasons 7–9 (Hardening & Safety)	Thms. 45–56, 70–77	20	Adaptive median norm clipping, staleness window rejection, thermal hysteresis deadband, WDDM TDR preemption, ZK-OPoI v4	compiles
Season 10 (Next-Gen DiLoCo)	Thms. 78–90	13	PA-CGAD cosine cutoff, FoMoE ring savings & Skip-Token, PULSELoCo half-ULP gate, Skip-Attention CAR, IsoLoCo, AutoLoCo, HeLoCo, `prime-diloco` 0 B VRAM, SparseLoCo 2-bit, LoopedMoE, MuLoCo	compiles
Season 11 (Looped FP & IsoSparse)	Thms. 91–100	10	Fixed-point recurrence contraction, early-exit FLOP savings, 3× terminal KV sharing, multi-wave FoMoE VRAM & payload bounds	compiles
Season 12 (OpenDiLoCo & H1–H10)	Thms. 101–112	12	Flat-bucket slice partition, fragment byte count, piecewise PA-CGAD cutoff, 16×/32× 2-bit DCT gate, MuLoCo spread contraction, EF21 telescoping sum, TBPTT memory, Skip-Token FLOPs	compiles
Season 13 (Necessity & Inhabitation)	Thms. 113–116	4	Disabled-EF21 mass loss counterexample, missing PA-CGAD cutoff variance inflation counterexample, violated Multi-Krum quorum counterexample, `Nonempty InhabitedRTX2070SwarmPackage` witness	compiles
Season 14 (Decoupled DiLoCo Quorum)	Thms. 117–120	4	Grace window $\gamma\,\xi_{\text{slack}}$ strictly below positive slack for $\gamma < 1$, exhausted slack $\Rightarrow$ zero grace, token-weight monotonicity, quorum admitted-count bounds $K \le \lvert\mathcal{A}\rvert \le M$	compiles
Season 15 (Quorum Liveness & Data Leasing)	Thms. 121–128	8	Quorum liveness under $M - K$ stragglers, lease TTL above step + RTT, lease conservation, momentum-restart energy, σ-MoE ghost slots, OrthoInj Pythagorean cross-term, factored-gossip byte positivity, E_8 vs. $\mathbb{Z}^8$ second-moment numerators $929 < 1080$ (arithmetic specifications; Theorems 125 and 127 do not constrain the mechanism they are named after, Section 15)	compiles
QueenBee (verified-feedback routing & knowledge points)	Thms. 129–140 + 2 helper lemmas	14	Faded dance records recruit nobody, per-record weight $\le$ cap and k records $\le k$-cap, score monotone in verified successes and antitone in padded trials, unverified outcomes never move the counts, 30% unverified-row cap ($7u \le 3v$), a lump-sum knowledge bonus preserves every best response and hence the Nash equilibrium, proportional payouts $\le$ pool, signer pseudo-trials $\le$ budget (arithmetic over $\mathbb{N}$; the Wilson score used in code is not formalised)	compiles
Seasons 24–25 (Honeybee Mesh & LayerStoRm PCIe Streaming)	Thms. 141–147	7	Employed/Onlooker/Scout ABC population conservation, Nectar Cache bounded capacity, LayerStoRm double-buffer overlap zero-stall when $t_{\text{PCIe}} \le t_{\text{Compute}}$, 2-slot VRAM bound $\le$ 6,656 MiB, E18 cosine-damped momentum bounds	compiles
Season 25 Wave 2 (Scientific MoE & Producer Quorum)	Thms. 148–150	3	Solver-surrogate verifier soundness, producer quorum MoE activation monotonicity, cross-CPU quorum-2 triangle agreement bound	compiles
Season 26 (Next-Gen MoE Science Bee Slot & Accuracy)	Thms. 151–153	3	Slot compiler compartment fraction conservation, Richardson long-horizon phase warning iff, owner RTX 2070 + T4 co-serving and SVD bounds	compiles
Season 27 (100-Producer International Launch Scale)	Thms. 154–156	3	100-producer regional and tier partition conservation, Erlang-C safe utilization bound, two-lane prosumer QoS and WAL batch reduction	compiles
Season 28 (CrowdSwarm v2 & Cohort Isolation)	Thms. 157–159	3	Anti-lobotomy weight RMS halving rejection, pre-Ampere `fp32` vs. Ampere+ `bf16` cohort isolation, forming-pool starvation timeout liveness	compiles
Season 29 (Super-Peer Fanout, SQLite Coalescer & Pareto Ratchet)	Thms. 160–162	3	Super-Peer fan-out coordinator ingress bound ($S < N$), Litestream SQLite batch HMT credit conservation, monotone nocturnal V-MoE Pareto ratchet	compiles
Season 30 (Volunteer Owner Consent & One-Heartbeat Pause)	Thms. 163–164	2	No volunteer node starts without stored owner enrolment and consent (`PS-02-D2`), pause command implies zero new work within one heartbeat	compiles
Total Formal Specification	**Theorems 1–164 (+2 helpers)**	**166**	Arithmetic over $\mathbb{N}/\mathbb{Z}$; standard Lean 4 axioms (`propext`, `Quot.sound`, `Classical.choice`)	166 / 166 compile

where $q_0, \ldots, q_{15} \in [-1, 1]$ are the information-theoretically optimal quantiles of $\mathcal{N}(0, 1)$, packed two 4-bit indices per `uint8` byte. Block scales s_b are further compressed via 8-bit dou-

ble quantization, yielding an effective footprint of 4.125 bits/parameter (**Theorem 1**). On `Qwen/Qwen2.5-0.5B`, the entire frozen backbone occupies just **175.96 MB** of VRAM with **0 bytes** of intermediate CPU heap duplication.

3.2 DiLoCo Inner Loop & Baseline Top-k + `float8_e4m3fn` Deltas

Each worker $k \in \{1, \dots, K\}$ assigned to domain branch $d \in \mathcal{D}$ freezes $\hat{W}_{\text{NF4}}$ and attaches trainable low-rank adapters $\theta_k = \{A_k, B_k\}$ of rank $r \in \{8, 16, 32, 64\}$ across all attention and SwiGLU projections ($\{W_q, W_k, W_v, W_o, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}\}$):

$$h = \text{Dequant}_{\text{FP16}}(\hat{W}_{\text{NF4}})x + \frac{\alpha}{r} B_k A_k x, \qquad A_k \in \mathbb{R}^{r \times d_{\text{in}}},\ B_k \in \mathbb{R}^{d_{\text{out}} \times r}. \tag{2}$$

Worker k executes H local inner steps on its Content-Addressed Hugging Face shard (`cid://sha256/...`) with **zero network communication** and computes its outer pseudo-gradient delta $\Delta_k^{(t)} = \theta^{(t-1)} - \theta_k^{(t,H)}$. In baseline mode (`compression_mode="topk_fp8"`), retaining Top-k ($\rho = 0.20$) entries quantized to `float8_e4m3fn` compresses a 16.78 MB rank-16 QLoRA delta into **5.04 MB** (3.33× wire reduction, or $> 1,666\times$ total bandwidth reduction at $H = 500$, **Theorem 3**). Section 7 details how Season 8 upgrades this wire layer with **DeMo 2D chunked DCT + 2-bit SparseLoCo + PULSELoCo** to achieve **39.96× package compression**.

4 Pillar II: Byzantine Fault Tolerance Without 2× Redundancy

4.1 10% Deterministic Micro-Checkpoint Spot-Checks & Hardware Circuit-Breaker

To verify untrusted consumer GPUs without running every workunit twice, we enforce deterministic CUDA execution (`CUBLAS_WORKSPACE_CONFIG=:4096:8`, `torch.use_deterministic_algorithms(True)`, and deterministic data-loader seeding derived from `prev_block_hash`). During its H inner steps, worker k records a cryptographic micro-checkpoint hash every s steps ($M = H/s = 10$ segments):

$$h_{k,m} = \text{SHA-256}\Big(\theta_k^{(t,m \cdot s)}\Big), \qquad m \in \{0, 1, \dots, M\}. \tag{3}$$

The `DeterministicSpotCheckValidator` (`src/hivemind/boinc/validator.py`) selects a single random segment $m^* \sim \mathcal{U}\{0, \dots, M-1\}$ and replays s steps from $\theta_k^{(t,m^* s)}$. The submission is accepted iff $\|\theta_{\text{validator}}^{(t,(m^*+1)s)} - \theta_k^{(t,(m^*+1)s)}\|_\infty \leq \epsilon = 10^{-4}$ (**Theorems 4–5**). Concurrently, the `GPUExecutionCircuitBreaker` (`src/hivemind/security/gpu_execution_circuit_breaker.py`) enforces thermal hysteresis deadbands (83°C throttle / 88°C abort), Windows WDDM TDR step-latency watchdogs (< 1.5 s), and NaN loss early abort (**Theorems 49–52**).

What the ϵ-rule assumes, and where it has been verified. Replaying s steps to within $\epsilon = 10^{-4}$ presumes that worker and validator execute the same floating-point program on the same arithmetic. That holds within one accelerator architecture and software stack, and it is the only regime in which we have measured it: the E8 pilot (Section 9.7) reproduced a two-host run to 10^{-5} with CPU PyTorch on both ends. Across architectures (Turing vs. Ada, NVIDIA vs. ROCm vs. Metal) kernel selection and fused-multiply-add contraction differ, so an honest worker can fail the rule; no cross-architecture replay has been measured here. The E12 pilot (Section 9.11) then measured something stronger than the review feared: on *one* host with *one* torch build, changing only the CPU thread count moves the replayed parameters by $2.6 \cdot 10^{-4}$ after 50 steps, above ϵ, while a second host with a different CPU and torch build moves them by $1.5 \cdot 10^{-4}$. The rule as shipped would reject honest workers for a thread-pool setting; and a worker running its forward pass in bfloat16, replayed by an fp32 validator, lands at $3.3 \cdot 10^{-3}$ with all 96 tensors

above threshold, beyond the tampered control, so a dtype mismatch is indistinguishable from an attack. It is therefore re-specified as *execution-class matched*: validator and worker must share accelerator class, software build, dtype and the kernel-selection settings that fix reduction order (thread count on CPU, deterministic-algorithm flags on GPU), ϵ is set per class from measured honest divergence plus margin, and the statistic gains a second term, the number of parameter tensors above threshold, which separated honest divergence (3–5 of 96 tensors) from the tampered control (96 of 96) far more sharply than the ℓ_∞ norm alone ($2.6 \cdot 10^{-4}$ vs. $1.3 \cdot 10^{-3}$). *Update (v3.7.0 / v3.7.2):* E12 (Section 9.11) has since measured it on GPU: the same T4 replayed in a separate process reproduces the segment exactly (0.0), whereas T4 against an x86 CPU in float32 diverges by $2.0 \cdot 10^{-4}$ to $6.3 \cdot 10^{-4}$, above ϵ in 9 of 9 segments; validators are therefore assigned by accelerator class (`src/hivemind/boinc/validator_assignment.py`) and cross-class replays are "not comparable". The fixed-point fallback remains untested (no kernel exists) and the RTX 2070 and TPU cells were not run.

4.2 Pre-Krum Sanitization, Multi-Krum 2σ Cosine Filter & ZK-OPoI v4

Before outer aggregation, `FederatedLearningSecurityGuard` (`src/hivemind/security/federated_learning_hardening.py`) quarantines non-finite or free-rider zero-gradient submissions, applies Multi-Krum neighborhood selection ($k = n-f-2 \geq f+1$, **Theorem 6**) with a 2σ median pairwise cosine similarity gate, and clips surviving deltas via adaptive median-L_2 and coordinate-L_∞ neurotoxin bounds (**Theorem 45**). For decentralized inference, **ZK-OPoI v4** (`src/hivemind/swarm/swarm_fabric.py`) binds per-token activation commitments to their exact batch index, layer index, and token position inside a SHA-256 Merkle tree with interactive $\lceil \log_2 L \rceil$ bisection dispute resolution (**Theorems 35–36, 70–77**).

5 Pillar III & Phase 2 Swarm Fabric: Factored BTM Upcycling & Geo-Rings

When E domain branches converge, `upcycle_branches_to_sparse_moe()` (`src/hivemind/core/btm_upcycle.py`) converts them into a unified **Factored Sparse MoE** where all E SwiGLU experts at layer ℓ share the single frozen 4-bit NF4 base FFN $(\hat{W}_{\text{gate}}, \hat{W}_{\text{up}}, \hat{W}_{\text{down}})$ and keep only their lightweight FP16 LoRA adapters (A_e, B_e) in VRAM:

$$E_{\ell,e}(x) = \Big(\hat{W}_{\text{down}} + \tfrac{\alpha}{r} B_{\text{down},e} A_{\text{down},e}\Big) \Big[\text{SiLU}\Big(\hat{W}_{\text{gate}} x + \tfrac{\alpha}{r} B_{\text{gate},e} A_{\text{gate},e} x\Big) \odot \Big(\hat{W}_{\text{up}} x + \tfrac{\alpha}{r} B_{\text{up},e} A_{\text{up},e} x\Big)\Big]. \tag{4}$$

Across WAN nodes, the **Phase 2 Swarm Fabric** (`src/hivemind/swarm/swarm_fabric.py`) provides: (M1) Kademlia XOR < 20 ms Regional Geo-Rings with RFC 8489/8656 STUN/TURN NAT traversal and RFC 9001 QUIC 0-RTT anti-replay (**Theorems 11, 16–24**); (M2) 3-tier NVMe → Pinned Host RAM → CUDA double-buffered asynchronous DMA streaming (**Theorem 12**); (M3) Prefix-Trie deduplicated Eagle/Medusa token-tree speculation ($\gamma = 6$) with Triton `float8_e4m3fn` hidden-state compression (**Theorems 13, 15**); and (M4) an OpenAI-compatible `/v1/chat/completions` gateway.

6 Missing MoE Architecture: 128-Expert 3-Tier Routing, Fo-MoE, Skip-Attention & Looped Recurrence

Earlier revisions documented only flat 4-to-8 branch BTM upcycling. Scaling to 64–128 community experts across residential WAN links raises five further design problems. This section describes the components we implemented for them; none has yet been evaluated at the target

scale (the largest MoE actually trained has 4 experts, Section 8.3). A caveat on the router: gating FLOPs are small compared with expert FFN FLOPs, and hierarchical gating restricts which expert combinations are reachable, so the saving below must eventually be weighed against routing quality.

6.1 3-Tier 128-Expert Hierarchical MoE Router (`ThreeTierHierarchicalMoERouter`)

In a flat $E = 128$ Sparse MoE, computing $\mathrm{Softmax}(W_g^\top x)$ requires $E = 128$ inner products of dimension d_{model} per token at every layer, and speculative prefetching over a token tree must evaluate all 128 expert columns. In `src/hivemind/core/btm_moe.py` (`HM-S6-02`), we partition the $E = 128$ experts into a 3-tier hierarchy of $S = 4$ super-clusters $\times M = 4$ macro-domains $\times U = 8$ micro-experts ($4 \times 4 \times 8 = 128$):

$$s^* = \arg\max_{s \in \{1..S\}} P_{\text{super}}(s \mid x), \qquad m^* = \arg\max_{m \in \{1..M\}} P_{\text{macro}}(m \mid x, s^*),$$
$$\mathcal{T}(x) = \text{Top-}k\left(W_{\text{micro}}^{(s^*,m^*)} x\right). \tag{5}$$

Per token, the router evaluates only $S + M + U = 4 + 4 + 8 = \mathbf{16}$ inner products of length d_{model} instead of 128—an exact **8.0× gating FLOP reduction** (an operation count; checked in H3 at toy scale). Furthermore, `predict_candidate_experts()` exposes the exact factored path probability $P(e \mid x) = P_{\text{super}}(s^*) P_{\text{macro}}(m^* \mid s^*) P_{\text{micro}}(e \mid s^*, m^*)$ ($\sum_e P(e \mid x) \le 1$) to drive token-tree lookahead DMA prefetching (`prefetch_token_tree_lookahead`).

6.2 Federated Partial Expert Replication (`FoMoE`) & Ghost-Expert `Skip-Token`

Following Sani *et al.* (`FoMoE`, `arXiv:2606.19025`) [20] and Google DeepMind's *Scaling Laws for DiLoCo* (`arXiv:2503.09799`) [16], requiring every consumer GPU to replicate and synchronize all $E = 128$ experts wastes both VRAM and outer-loop WAN bandwidth. In `FoMoEPartialReplicaPlanner` (`src/hivemind/swarm/diloco_nextgen.py`) and `UpcycledSparseMoEModel.set_local_expert_subset()` (`src/hivemind/core/btm_upcycle.py`, `HM-S8-06`), each of the M workers hosts only $M_{\text{loc}} = \frac{O_e}{M} E$ resident experts (where $O_e \in \{1, \ldots, M\}$ is the replication overlap factor) while the remaining $E - M_{\text{loc}}$ experts act as **Ghost Experts**:

$$R(O_e) = \frac{2(O_e - 1)}{O_e}, \qquad V_{\text{comm}}(O_e) = \frac{2(M-1)}{M} B_{\text{dense}} + \left(R(O_e) + \gamma_{\text{shuf}}\left(1 - \frac{O_e}{M}\right)\right) M_{\text{loc}} B_{\text{expert}}. \tag{6}$$

During local inner-loop forward passes, any token slot routed by the global router to a non-resident Ghost Expert $e \notin \mathcal{E}_{\text{loc}}$ is dropped locally via **`Skip-Token`**:

$$y_{\text{FoMoE}}(x) = \sum_{e \in \mathcal{T}(x) \cap \mathcal{E}_{\text{loc}}} \tilde{g}_e(x)\, E_{\ell,e}(x), \qquad \text{RPC}_{\text{remote}}(x) = 0. \tag{7}$$

Evaluating this cost model (check H3) with $M = 16$ workers with overlap $O_e = 4$ (32 resident experts and 96 Ghost Experts per worker) predicts a 3.46× reduction in outer ring volume, with no synchronous remote FFN calls because ghost-routed tokens are skipped. This is a model prediction, not a network measurement; the effect of skipping on quality is studied in [20] but not yet in our setting. Skip-Token is, in effect, a routing-dependent dropout of the expert contribution: the residual stream and attention path of a skipped token are unchanged, the token loses only the output of experts that are not resident on this worker, and the resident experts' adapters never see tokens whose entire top-k set is remote. The fraction so affected is not the ideal N_{le}/N_e but the *measured* ghost fraction, which is why RQ5/E7 (Section 9) is phrased in terms of the measured value and why the FLOP model in `fomoe_sigma_routing.py` takes it as an input. We have no downstream zero-shot number for this effect in our setting and make no claim about it.

σ-MoE routing and measured skip fraction. FoMoE's experts are selected by independent sigmoid gates (σ-MoE, following Csordás *et al.*) rather than a competitive softmax, and its cost model (their Eq. 2–3) uses the ideal scaling $\sigma_{\text{st}} = N_{\text{le}}/N_e$ while explicitly recommending the *measured* ghost-assignment frequency whenever routing is imbalanced. `FoMoEPartialReplicaPlanner` models only the ideal factor; `src/hivemind/core/fomoe_sigma_routing.py` adds `sigmoid_independent_topk_routing` (global router, ghost slots skipped *after* routing, realised ghost fraction returned per batch) and `skip_token_flops_per_token` (their Eq. 3 with either factor, and the N_{le}/N_e gate factor only when the router itself is partitioned). On the toy two-layer 128-expert configuration of `tests/test_fomoe_sigma_routing.py` (32 resident experts, top-2) the ideal factor and a measured 50% ghost fraction give 1,432 vs. 1,832 FLOPs per token, so the ideal factor under-estimates compute by 28% under that imbalance. Neither function has been run on a trained model.

6.3 `Skip-Attention` Speculative Prefetch & Cache-Aware Routing (`CAR`)

When serving all E experts locally via 3-tier NVMe → Pinned Host RAM → VRAM streaming, waiting until Layer $\ell+1$'s self-attention finishes to evaluate Layer $\ell+1$'s router exposes the PCIe DMA transfer latency. Inspired by the multi-tier expert caching and cache-aware routing of Paul Merolla's `fomoe` inference engine [21] and by expert prefetching in `learning-at-home/hivemind` [43], our `SkipAttentionSpeculativeRouter` (`src/hivemind/swarm/diloco_nextgen.py`, `HM-S8-07`) evaluates Layer $\ell+1$'s RMSNorm and router projection $W_{g,\ell+1}$ directly on Layer ℓ's **post-FFN residual stream** $x_\ell^{\text{post-FFN}}$ *before* Layer $\ell+1$'s self-attention begins:

$$\hat{\mathcal{T}}_{\ell+1}^{\text{spec}}(x) = \text{Top-}k\Big(W_{g,\ell+1}\,\text{RMSNorm}_{\ell+1}\Big(x_\ell^{\text{post-FFN}}\Big)\Big)\,. \tag{8}$$

The predicted experts $\hat{\mathcal{T}}_{\ell+1}^{\text{spec}}$ are streamed asynchronously from Pinned Host RAM into the alternating lookahead CUDA double-buffer slot (`slot_A`/`slot_B`) in parallel with Layer $\ell+1$'s self-attention compute. Once self-attention finishes and true post-attention router logits $z^{\text{true}} \in \mathbb{R}^E$ are available, any selected expert $e^* \notin \mathcal{C}_{\text{VRAM}}$ that missed the speculative prefetch is checked against the **Cache-Aware Routing (`CAR`)** threshold τ_{CAR}:

$$\text{If } \min_{e_c \in \mathcal{C}_{\text{VRAM}} \setminus \mathcal{T}} \Big(z_{e^*}^{\text{true}} - z_{e_c}^{\text{true}}\Big) \le \tau_{\text{CAR}}, \quad \text{substitute } e^* \leftarrow \arg\max_{e_c \in \mathcal{C}_{\text{VRAM}} \setminus \mathcal{T}} z_{e_c}^{\text{true}}. \tag{9}$$

In unit check H5 the speculative Top-2 prediction matched the true routing in 95.83% of cases. That figure is *not* a property of any trained model: the check uses random 32-dimensional states and sets the post-attention state to the post-FFN state plus Gaussian noise of scale 0.08, so the accuracy is determined by that noise level. Measuring speculative accuracy on real Qwen2.5 activations is future work.

6.4 Weight-Tied `LoopedExpertBackbone`, Fixed-Point Halting & Truncated BPTT

To multiply reasoning depth on 8 GB consumer GPUs without multiplying model weight VRAM, `LoopedExpertBackbone` (`src/hivemind/core/btm_upcycle.py`, `HM-S7-01/02`) wraps an upcycled Sparse MoE decoder stack into a three-stage recurrent architecture synthesized from *Towards Looped Models Done Right, Part I: Topology, Input Injection, Recurrent-State Design* (`arXiv:2610.23036`) [22] and *Part II: Rethinking at Fixed Points* (`arXiv:2610.06833`) [23]:

$$\begin{aligned} h^{(0)} &= \text{Prelude}(x), \qquad y = \text{Coda}\Big(h^{(T)}\Big)\,, \\ h^{(t)} &= \text{RecurrentCore}\Big(\mathcal{A}_t\Big(h^{(t-1)} + \mathbb{I}[t>1]h^{(0)}\Big)\Big) \quad (t = 1,\dots,T), \end{aligned} \tag{10}$$

where $\mathcal{A}_t(u) = u + B_t^{\text{step}} A_t^{\text{step}} u$ is a lightweight per-step residual LoRA adapter initialized with $B_t^{\text{step}} = 0$ (exact identity at initialization). `LoopedExpertBackbone` implements three systems optimizations (the associated Lean lemmas only count bytes):

1. **Zero Duplicate Core Weight VRAM:** The recurrent core layers are weight-tied across all T loop steps, saving $(T-1)B_{\text{core}}$ bytes of weight storage (669,888 B saved at $T = 4$, multiplying effective depth from $4 \to 10$ layers with only 4,096 B of step adapters).

2. **Per-Token Fixed-Point Convergence Early Exit:** After each recurrent pass t, each active token position i evaluates its relative state drift $r_i^{(t)} = \|h_i^{(t)} - h_i^{(t-1)}\|_2 / \max(\|h_i^{(t-1)}\|_2, 10^{-6})$. Tokens satisfying $r_i^{(t)} \le \epsilon_{\text{fp}}$ freeze their hidden state for subsequent passes while remaining visible as causal attention context.

3. **RNG-Restored Truncated Fixed-Point BPTT (`truncated_bptt_steps`):** When $k_{\text{bptt}} < T$ and adaptive halting is active, forward passes discover the exact per-token halting trajectory without storing intermediate activation graphs, then replay only the trailing k_{bptt} recurrent passes under `torch.random.fork_rng` with restored CPU/CUDA RNG state and preserved MoE routing counters.

   In unit check H4 (`fixed_point_tol = 0.65` with $T = 4$ loop steps), tokens halt after a non-trivial mean of **3.05** recurrent passes (80% early-exit rate, mean residual 0.5215). In Experiment E3 (Section 8, Figure 4a), sweeping $\epsilon_{\text{fp}} \in [0.0, 1.10]$ across a $T = 6$ looped MoE backbone maps the smooth trade-off between recurrent FLOP savings ($0\% \to 71.88\%$) and relative output logit drift ($0\% \to 58.61\%$).

6.5 Subspace-Aware Attention & Expert Merging: `Iso-CTS` & `TSV`

Standard BTM upcycling uniformly averages self-attention LoRA deltas ($\bar{\Delta}_{\text{attn}} = \frac{1}{E}\sum_{e=1}^{E} \Delta_{\text{attn},e}$), which suffers from singular-direction dominance collapse when heterogeneous domain branches are combined. Building on isotropic merging (Iso-C / Iso-CTS) by Marczak *et al.* [25], its use as a DiLoCo aggregator in IsoLoCo by Horoi *et al.* [26], and Task Singular Vectors by Gargiulo *et al.* [27], `upcycle_branches_to_sparse_moe()` supports two spectral subspace mergers (`HM-S8-03`, **Hypothesis H6**):

- **Reputation-Weighted `ISO-CTS` (`attention_merge_mode="iso_cts"`):** Elects sign consensus via reputation-weighted TIES, extracts the top $c = \lfloor \rho_c \min(d_{\text{out}}, d_{\text{in}}) \rfloor$ common singular vectors U_{common}, projects worker residuals onto the orthogonal complement $(I - U_{\text{common}} U_{\text{common}}^\top)$, orthogonalizes the combined basis via SVD Procrustes projection, and flattens the singular value spectrum to its isotropic mean $\bar{\sigma} = \frac{1}{r}\sum_i \sigma_i$, which makes the merged singular-value condition number exactly 1 by construction (H6 confirms $2.5988 \to 1.0000$ on a toy merge; whether this helps downstream quality in our setting is untested).

- **Task Singular Vector Decorrelation (`attention_merge_mode="tsv"`):** Extracts the top $k_w = \lfloor d_{\min}/M \rfloor$ singular triplets $(U_m, \Sigma_m, V_m^\top)$ per branch and whitens inter-branch subspace interference via orthogonal Procrustes projection $U_{\text{orth}} = P_u Q_u^\top, V_{\text{orth}}^\top = P_v Q_v^\top$, achieving $\|U_{\text{orth}}^\top U_{\text{orth}} - I\|_F = \mathbf{6 \times 10^{-6}}$ (**Theorem 110**).

7 OpenDiLoCo & Google DeepMind Outer-Loop Ecosystem

To make outer-loop aggregation resilient to asynchronous WAN stragglers, residential upload limits, and 8 GB consumer GPU memory ceilings, Project HiveMind adapts ideas and, where licences allow, code from eight open-source repositories and recent papers from Google DeepMind, Prime Intellect, Mila, Covenant AI, Nous Research, and Cambridge / Flower Labs (Table 2). We credit these works for the underlying methods; HiveMind's contribution is their integration under an 8 GB memory budget.

Table 2: Open-Source GitHub Repositories & Foundational Papers Integrated into Project HiveMind's `OpenDiLoCo` + `Sparse MoE` Architecture (`src/hivemind/swarm/diloco_nextgen.py`).

Repository / Project	Primary Authors & Lab	Reference	Integrated HiveMind Module & Role
`google-deepmind/asyncdiloco`	Liu, Chhaparia, Douillard *et al.* (DeepMind)	[3, 4]	Reference for asynchronous outer updates; our fragment streaming (`synced_fragments`) follows Streaming DiLoCo
`Decoupled DiLoCo` (paper only; no public code)	Douillard, Rush, Donchev, Charles *et al.* (DeepMind & Google Research)	[5]	`DecoupledDiLoCoQuorumSyncer` (`decoupled_diloco.py`): minimum quorum K, adaptive grace window, token weights, RDA merge
`PrimeIntellect-ai/OpenDiLoCo`	Jaghouar, Ong, Hagemann (Prime Intellect)	[17]	Decentralized outer-step orchestration & Hivemind P2P fault-tolerant gradient averaging
`PrimeIntellect-ai/prime-diloco`	Prime Intellect Team	[17]	`CPUOffloadedFlatBucketOuterState`: contiguous 1D FP32 CPU master buffer (`0 B` extra GPU VRAM)
`one-covenant/SparseLoCo`	Sarfi, Belilovsky *et al.* (Mila / Covenant)	[29, 30]	`ComputeVisibleSparseLoCoCompressor` & `DCTSparse2BitTensorPayload` (2-bit 4-bin LUT + EF)
`bloc97/DeMo`	Peng, Quesnelle, Kingma (Nous Research)	[28]	`ChunkedOrthonormalDCTTransform`: 8×8 Type-II orthonormal DCT with deterministic `index_copy_`
`shoroi/isoloco`	Horoi, Thérien, Wolf, Belilovsky (Mila)	[26, 25]	`ReputationWeightedIsoCTSMerger` (`ISO-C`/`ISO-CTS`) & `TSVDecorrelatedMerger`
`pmerolla/fomoe`	Paul Merolla	[21]	Multi-tier expert cache & cache-aware routing; inspires `SkipAttentionSpeculativeRouter` (FoMoE partial replication follows [20])
`learning-at-home/hivemind`	Ryabinin, Diskin, Mazur *et al.*	[43, 44]	Kademlia DHT peer rings, background expert prefetching & decentralized MoE routing

7.1 `OpenDiLoCo` & `prime-diloco`: Contiguous 1D CPU Flat-Bucket Outer State (`0 B` GPU VRAM)

In naive PyTorch DiLoCo implementations, holding a copy of the outer master weights $\theta^{(t-1)}$ and outer Nesterov velocity buffer $v^{(t)}$ on `cuda:0` doubles the adapter/backbone memory footprint during every outer step. Adapting `PrimeIntellect-ai/prime-diloco` (`src/zeroband/diloco.py`) and `PrimeIntellect-ai/OpenDiLoCo` (`arXiv:2407.07852`) [17], `CPUOffloadedFlatBucketOuterState` (`src/hivemind/swarm/diloco_nextgen.py`) and `DiLoCoOuterOptimizer(use_cpu_flat_bucket=True)` (`src/hivemind/core/diloco_outer.py`, `HM-S8-02`) pack all parameter fragments into a single contiguous 1D FP32 host RAM allocation $\mathbf{w}_{\text{flat}} \in \mathbb{R}^N_{\text{CPU}}$ with deterministic slice offsets $\{[\text{lo}_k, \text{hi}_k)\}_{k=1}^F$:

$$\begin{aligned} \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] &\leftarrow \mu \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] + \bar{\Delta}_k^{(t)}, \\ \mathbf{w}_{\text{flat}}[\text{lo}_k : \text{hi}_k] &\leftarrow \mathbf{w}_{\text{flat}}[\text{lo}_k : \text{hi}_k] - \eta \left(\mu \mathbf{v}_{\text{flat}}[\text{lo}_k : \text{hi}_k] + \bar{\Delta}_k^{(t)} \right). \end{aligned} \tag{11}$$

By construction the outer state then uses no GPU memory; the cost moves to host RAM and to one host-to-device copy per outer step, which we have not yet timed. Check H1 confirms the flat-bucket update matches the per-tensor Nesterov update to single-ULP FP32 precision ($\max \|\Delta\theta\|_\infty = 2.384 \times 10^{-7}$).

7.2 Decoupled DiLoCo: Minimum Quorum, Adaptive Grace Window & Radial-Directional Averaging

Google DeepMind's *Decoupled DiLoCo* (`arXiv:2604.21428`) [5] removes the lock-step outer all-reduce altogether: M independent learners report to a CPU-resident *syncer* that aggregates a fragment as soon as a minimum quorum of $K \leq M$ learners has arrived, optionally waits an adaptive grace window for late learners, weights each learner by its token throughput, and merges the per-learner outer gradients with Radial-Directional Averaging (RDA). This is the regime a volunteer swarm actually lives in (stragglers, churn, heterogeneous GPUs), and until this revision HiveMind cited the paper but implemented none of it. `DecoupledDiLoCoQuorumSyncer` (`src/hivemind/swarm/decoupled_diloco.py`, `SI-12`) implements the four mechanisms of Algorithm 2 of [5] as separately testable pieces:

1. **Minimum quorum.** The first K learners by arrival are always admitted; a round with fewer than K reports does not merge (**Theorem 120** fixes $K \le |\mathcal{A}| \le M$ and shows that the degenerate $K = 0$ rule admits nobody).

2. **Adaptive grace window.** With $\xi_{\text{slack}} = \tau\,\xi_{\text{step}} - (\xi_{\text{quorum}} + \xi_{\text{sync}})$ (their Eq. 3) tracked by exponential moving averages, learners arriving within $\xi_{\text{grace}} = \gamma \max(0, \xi_{\text{slack}})$ of the quorum instant are also admitted. **Theorems 117–118** check that $\gamma < 1$ keeps $\xi_{\text{grace}} < \xi_{\text{slack}}$ whenever the slack is positive and that an exhausted budget gives $\xi_{\text{grace}} = 0$.

3. **Token-weighted merging.** $w_m = c_m^{\text{tokens}} \cdot c_m^{\text{tokens}} / c_m^{\text{steps}}$ ("quantity × quality"; a learner with no inner step has weight 0; **Theorem 119**).

4. **Radial-Directional Averaging.** $\text{RDA}(\{(\Delta_m, w_m)\}) = \big(\sum_m \bar{w}_m \|\Delta_m\|\big)\, \phi\big(\sum_m \bar{w}_m \phi(\Delta_m)\big)$ with $\phi(x) = x/\|x\|$ and $\bar{w}_m = w_m / \sum_j w_j$, so that equal-norm, nearly orthogonal learner updates keep their norm R instead of shrinking to $R/\sqrt{M}$ under direct averaging; the outer learning rate therefore no longer needs retuning with M.

The syncer emits Δ_p and leaves the outer optimizer untouched: check `H11` chains it into the CPU flat-bucket Nesterov step above with `0 B` extra GPU memory. In `H11` (`artifacts/decoupled_diloco_quorum_check.json`; three arms, one positive and two negative controls) RDA returns norm R for $M \in \{2, 4, 8, 16\}$ orthogonal equal-norm deltas while the direct mean follows $R/\sqrt{M}$ exactly (RDA-to-direct ratio **2.8284** at $M = 8$); with $\tau = 2$, $\xi_{\text{step}} = 4.0\,\text{s}$, $\xi_{\text{quorum}} = 0.3\,\text{s}$, $\xi_{\text{sync}} = 1.0\,\text{s}$ and $\gamma = 0.5$ the controller opens a **3.35** s window on 6.7 s of slack, admits 6/8 learners, defers two stragglers carrying 10,000 of 40,000 tokens, and the stragglers' coordinates do not leak into Δ_p (leak = 0.0); a $K = M = 8$ quorum with seven reports correctly refuses to merge. These are bookkeeping checks on synthetic tensors: the goodput and quality results under chaos engineering are DeepMind's, not ours, and nothing here has yet run across machines.

What RDA does to the optimization problem is elementary and worth stating, because it turns the RQ2 prediction of Section 9 into a statement about one scalar.

Proposition 7.1 (RDA is direct averaging with an adaptive outer-learning-rate multiplier)**.** *Let $\Delta_1, \dots, \Delta_M \in \mathbb{R}^d$ be non-zero and $\bar{w}_1, \dots, \bar{w}_M \ge 0$ sum to one; write $u = \sum_m \bar{w}_m \Delta_m$ for the direct weighted average and suppose $u \neq 0$ and $\sum_m \bar{w}_m \phi(\Delta_m) \neq 0$. Then*

$$\text{RDA}(\{(\Delta_m, \bar{w}_m)\}) \;=\; r\,\|u\|\,\phi(u_\phi) \quad \textit{with} \quad r \;=\; \frac{\sum_m \bar{w}_m \|\Delta_m\|}{\|u\|} \;\ge\; 1, \tag{12}$$

where $u_\phi = \sum_m \bar{w}_m \phi(\Delta_m)$ is the weighted mean direction*; equality $r = 1$ holds iff all Δ_m with $\bar{w}_m > 0$ are positively collinear. If the Δ_m have a common norm R, then $\phi(u_\phi) = \phi(u)$ (RDA and direct averaging point the same way) and*

$$r \;=\; \big(\bar{w}^\top C\, \bar{w}\big)^{-1/2}, \qquad C_{ij} = \cos \angle(\Delta_i, \Delta_j), \tag{13}$$

so for M equally weighted orthogonal learners $r = \sqrt{M}$, and for equally weighted learners with mean pairwise cosine $\bar{c}$, $r = \big(1/M + (1 - 1/M)\,\bar{c}\big)^{-1/2}$.

Proof. $\|\text{RDA}\| = \sum_m \bar{w}_m \|\Delta_m\|$ by definition, and $\|u\| \le \sum_m \bar{w}_m \|\Delta_m\|$ is the triangle inequality, with equality iff the weighted terms are positively collinear. With a common norm R, $\phi(\Delta_m) = \Delta_m / R$, hence $u_\phi = u/R$ and the directions coincide; $\|u\|^2 = \sum_{i,j} \bar{w}_i \bar{w}_j \langle \Delta_i, \Delta_j \rangle = R^2\, \bar{w}^\top C \bar{w}$ gives the second formula, and $\bar{w} = \mathbf{1}/M$ with $C = I$ gives $\sqrt{M}$. □

Two consequences shape the experiments. First, in the common-norm case RDA changes nothing but the step length: a run with RDA at outer learning rate η takes, in each round t, exactly the step that direct averaging would take at $\eta \cdot r_t$. The RQ2 prediction is therefore not

that RDA finds a better direction but that the sequence r_t absorbs the $\sqrt{M}$ shrinkage, so that one η serves every M. Second, r_t is a cheap, per-round observable ($\|\text{RDA}\|/\|u\|$), and with the mean pairwise cosine it is reported for every cell of E5 (Section 9.6): a testbed on which $\bar{c} \approx 0.9$ has $r_t \approx 1.04$ and cannot separate the merges, which is precisely what E5-toy found. Theorem 120 and the H11 check verify the integer bookkeeping and the $\sqrt{M}$ instance. The real-valued content is machine-checked separately, because it needs Mathlib and `HiveMindSpec.lean` is Mathlib-free: `lean4/mathlib/RDAMultiplier.lean` proves, over any real inner-product space, that the RDA radius dominates the direct norm ($r \geq 1$), the Gram quadratic form $\|u\|^2 = \sum_{i,j} \bar{w}_i \bar{w}_j \langle \Delta_i, \Delta_j \rangle$, and the R^2/M shrinkage for pairwise-orthogonal common-norm deltas; the three theorems depend only on `propext`, `Classical.choice` and `Quot.sound` under the independent gate of Section 2 (`artifacts/lean_rda_multiplier_check.json`). The common-norm formula for r follows from the second theorem by hand; it is not stated in Lean.

7.3 Streaming DiLoCo Fragments & Cosine-Gated Adam-Decay (`PA-CGAD`)

Google DeepMind's `Streaming DiLoCo` (`arXiv:2501.18512`) [4] reduces peak WAN bandwidth by synchronizing only a subset of parameter fragments $\mathcal{S}_t \subset \{1, \ldots, F\}$ per outer round (**Theorem 102**). However, under asynchronous WAN delays τ_w and staggered fragment ages a_f, standard outer Adam/Nesterov suffers from *stale variance pollution*: stale pseudo-gradients inflate the second-moment buffer v_f, shrinking future honest step sizes. Following V. Shah and J. Sun (`CGAD / PA-CGAD`, `arXiv:2605.09126`) [24], `CosineGatedAdamDecayOuterOptimizer` (`HM-S8-01`) defines the effective fragment staleness $\tau_{\text{eff}}(f) = \max(0, \tau_w(f), a_f)$ and gates $\bar{\Delta}_f^{(t)}$ **before** updating (m_f, v_f):

$$\gamma(\tau) = \frac{1}{2}\left(1 + \cos\left(\frac{\pi\tau}{\tau_{\text{cut}}}\right)\right)\mathbb{I}[\tau < \tau_{\text{cut}}], \qquad \sigma(\tau) = \gamma(\tau)e^{-\alpha\tau}, \qquad \tilde{g}_f = \sigma(\tau_{\text{eff}}(f))\,\bar{\Delta}_f^{(t)}. \tag{14}$$

When $\tau_{\text{eff}}(f) \geq \tau_{\text{cut}}$, $\sigma(\tau_{\text{eff}}(f)) = 0$ and the fragment's moment buffers (m_f, v_f) are completely frozen (**Theorems 78, 103**). Furthermore, for all $\tau \geq 0$, the staleness bias satisfies the uniform upper bound:

$$\sup_{\tau \geq 0} \tau\sigma(\tau) \leq \sup_{\tau \geq 0} \tau e^{-\alpha\tau} = \frac{1}{e\alpha}. \tag{15}$$

This bound is analytic and is due to [24]. Check H2 only confirms that our implementation of σ respects it ($\max_{\tau \in [0,64]} \tau\sigma(\tau) = 1.7417 \leq 1.8394$ at $\alpha = 0.2$, $\tau_{\text{cut}} = 32$) and that cut-off fragments leave (m_f, v_f) untouched; it says nothing about training dynamics under real delays. Two further DeepMind and Meta results bound what the outer loop can hide. *Eager updates* [14] overlap the outer all-reduce with a whole inner phase by applying each replica's fresh local outer gradient together with the one-round-stale average of the others ($\tilde{\Delta}_m = \frac{1}{M}(\Delta_m^{(t)} - \Delta_m^{(t-H)}) + \Delta^{(t-H)}$); on a 500M model this tolerates 2.4 s of communication latency at an evaluation-loss cost of $2.67 \to 2.69$, whereas the naïve delayed variant costs $2.67 \to 3.01$ unless its outer learning rate is cut 4×. Our E8 pilot (Section 9.7) found exactly the regime this addresses, round trip $\gg$ compute, and the eager rule is the first candidate for the deferred-learner path of `HM-S9-01`. *Generalized Primal Averaging* [15] shows that single-worker DiLoCo is a choppy special case of a two-constant primal-averaging scheme (μ_x for the evaluated iterate, μ_y for the gradient point) that removes the inner/outer split and one hyper-parameter; it is the natural control for the outer-momentum-restart factor of E5. One tension deserves stating here rather than only in the results: when a quorum already excludes late learners, the deltas that arrive one round late are usually still directionally correct, and down-weighting them by $\sigma(\tau)$ discards signal. Both pre-registered toy factorials refuted the prediction that PA-CGAD helps under a quorum (Sections 9.5 and 9.6: harm of 0.0006–0.005 nats with intervals excluding zero, exactly zero effect without stale deltas), so the gate is off by default whenever $K < M$ and E2's result is reported as a no-quorum result.

7.4 `DeMo` Chunked 2D DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo` + `EF21`

To compress outer pseudo-gradients far beyond 8-bit `float8_e4m3fn`, `src/hivemind/core/compression.py` (`compression_mode="dct_sparse_2bit"`, `HM-S8-04/05`) unifies three complementary algorithms:

1. **Compute-visible gate after `PULSELoCo` (`arXiv:2602.03839`, introduced for distributed RL post-training) [30]:** Drops per-coordinate updates $|\Delta_i| < 2^{-8}|\theta_i|$ that would round to zero when added to a BF16/FP16 parameter (**Theorem 81**).

2. **`DeMo` Orthonormal 2D Chunked Type-II DCT (`bloc97/DeMo`, `arXiv:2411.19870`) [28]:** Reshapes each LoRA delta into 8×8 blocks and projects them via the orthonormal Type-II DCT basis $C \in \mathbb{R}^{8\times 8}$ ($CC^\top = I$, measured $\|CC^\top - I\|_F = 2.3 \times 10^{-7}$), concentrating spatial gradient energy into low-frequency coefficients.

3. **`SparseLoCo` 2-Bit Statistical Centroid LUT & Bit-Packing (`one-covenant/SparseLoCo`, `arXiv:2508.15706`) [29]:** Retains the top 2% DCT frequency coefficients, quantizes them into 4 empirical statistical centroids stored as a 4-entry FP16 lookup table (8 bytes), and packs four 2-bit codes per `uint8` byte (`_pack_2bit_codes`) with deterministic `index_copy_` reconstruction (eliminating `DeMo`'s non-deterministic CUDA `scatter_reduce_`).

In check H7 a package of four 256×16 LoRA deltas is encoded in 820 B. Measured against 16-bit storage (32,768 B) this is 39.96×; against FP32 (65,536 B) it is 79.92×. Because a single 2% Top-k projection on isotropic Gaussian noise has a high single-shot relative L_2 truncation error ($\|\hat{\Delta} - \Delta\|_2 / \|\Delta\|_2 = 0.9313$ for 2-bit DCT and 0.8966 for PULSE+SparseLoCo), pairing aggressive 2-bit sparsification with **EF21 error-feedback residual accumulation** [42] is essential: `EF21DiLoCoErrorFeedbackBuffer` (**Theorems 57, 107**, **Hypothesis H10**) preserves exact multi-step telescoping sum conservation $\|(\sum_{t=1}^{3} c_t + r_3) - \sum_{t=1}^{3} g_t\|_2 = \mathbf{4 \times 10^{-8}}$, and Experiment E1 (Section 8, Figure 2) demonstrates that 2-bit DCT + EF21 matches uncompressed FP32 DiLoCo within **0.052%** held-out validation loss across 12 outer rounds.

7.5 `AutoLoCo`, `HeLoCo` & Inner-Loop `MuLoCo` Newton-Schulz Preconditioning

Finally, to stabilize heterogeneous worker cadences and large replica counts:

- **`AutoLoCo` (`arXiv:2609.36662`) [38] & `HeLoCo` (`arXiv:2606.00271`) [31]:** When `AdaptiveDiLoCoCadenceController` adapts inner step count H_t, `AutoLoCoIntervalCorrector` scales outer learning rate $\eta_{\text{outer}} \leftarrow \eta_0 / \sqrt{S_t / S_{\text{ref}}}$ and outer momentum $\mu_{\text{outer}} \leftarrow \mu_0^{S_t/S_{\text{ref}}}$ (0.7000 → 0.4950 when $H : 32 \to 64$, **Theorem 84**), while `HeLoCoDirectionCorrector` projects out conflicting stale drift components whenever $\langle g_k, m_{\text{ref}} \rangle < 0$ (**Theorem 85**, **Hypothesis H8**).

- **Muon-style quintic Newton-Schulz LoRA preconditioning (Muon [39]; MuLoCo, `arXiv:2505.23725` [40]; replica-scaling analysis, `arXiv:2610.19064` [41]):** In `run_diloco_inner_loop(..., use_muon_lora_preconditioner=True)` (`src/hivemind/core/diloco_inner.py`, `HM-S8-09`), 2D LoRA gradients $G \in \mathbb{R}^{m \times n}$ are normalized ($X_0 = G/\|G\|_F$) and orthogonalized via $K = 5$ quintic Newton-Schulz iterations with the coefficients of Muon [39] ($a = 3.4445, b = -4.7750, c = 2.0315$):

$$X_{k+1} = aX_k + \left(bX_kX_k^\top + c\left(X_kX_k^\top\right)^2\right) X_k. \tag{16}$$

  In check H9, 5 iterations reduce the condition number of a synthetic 32×16 rank-8 gradient from 125.0 to 1.65, and validator replay on CPU reproduces the worker bit-for-bit (difference 0.0). Cross-GPU bitwise reproducibility was not tested.

7.6 Analytical Theorems, Proofs, Hypothesis-Necessity Counterexamples & 4-Definitions Contract

To make the mathematical guarantees and their failure modes self-contained, we state the three core outer-loop invariants with explicit analytical proofs and paired *Hypothesis-Necessity Counterexamples* (`Season13`, Lean 4 Theorems 113–116), followed by the 4-Definitions Contract (Table 3).

Theorem 7.2 (Multi-Step EF21 Telescoping Conservation & Necessity of Residual Carry; Lean Thms. 107 & 113)**.** *Let $\{g_t\}_{t=1}^T \subset \mathbb{R}^d$ be a sequence of outer pseudo-gradients and $\mathcal{C} : \mathbb{R}^d \to \mathbb{R}^d$ be any deterministic compressor (including 2-bit `DeMo` DCT + `SparseLoCo` LUT). Under the error-feedback recurrence $u_t = g_t + r_{t-1}$, $c_t = \mathcal{C}(u_t)$, $r_t = u_t - c_t$ with $r_0 = 0$, the cumulative transmitted sum satisfies the exact telescoping identity:*

$$\sum_{t=1}^{T} c_t + r_T = \sum_{t=1}^{T} g_t. \tag{17}$$

Hypothesis-Necessity Counterexample (Thm. 113): *If error feedback is disabled ($r_t \equiv 0$, so $c_t = \mathcal{C}(g_t)$) and step 1 compression is strictly contractive on positive mass ($c_1 < g_1$ with $c_2 \le g_2$), then $c_1 + c_2 < g_1 + g_2$ with positive permanent deficit $g_1 - c_1 > 0$.*

Proof. At each round $t \in \{1, \dots, T\}$, substituting $u_t = g_t + r_{t-1}$ into $c_t + r_t = u_t$ yields $c_t = g_t + r_{t-1} - r_t$. Summing from $t = 1$ to T telescopes all intermediate residuals $r_1, \dots, r_{T-1}$:

$$\sum_{t=1}^{T} c_t = \sum_{t=1}^{T} g_t + r_0 - r_T = \sum_{t=1}^{T} g_t - r_T \implies \sum_{t=1}^{T} c_t + r_T = \sum_{t=1}^{T} g_t.$$

Conversely, when $r_1 = 0$, step 2 compresses g_2 alone ($c_2 \le g_2$), so $c_1 < g_1 \implies c_1 + c_2 < g_1 + g_2$ (machine-checked in `ef21_disabled_loses_untransmitted_residual`). This explains why single-round 2-bit DCT exhibits 0.9313 relative L_2 truncation error in `H7` yet achieves a 0.052% held-out loss gap across $R = 12$ rounds with EF21 in Experiment `E1`. □

Theorem 7.3 (PA-CGAD Staleness Bias Bound & Necessity of Cosine Cutoff; Lean Thms. 103 & 114)**.** *For any decay rate $\alpha > 0$ and cutoff horizon $\tau_{cut} > 0$, the PA-CGAD gate $\sigma(\tau) = \frac{1}{2}(1 + \cos(\pi\tau/\tau_{cut}))\,\mathbb{I}[\tau < \tau_{cut}]e^{-\alpha\tau}$ satisfies:*

$$\sup_{\tau \ge 0} \tau\sigma(\tau) \le \frac{1}{e\alpha}, \qquad \text{and} \qquad \forall \tau_{eff} \ge \tau_{cut}, \quad v_t = \beta_2 v_{t-1}. \tag{18}$$

Hypothesis-Necessity Counterexample (Thm. 114): *If the staleness cutoff is omitted ($\alpha_2 = 1$ at $\tau_{eff} \ge \tau_{cut}$), any non-zero stale gradient $\|g_{stale}\|_2^2 > 0$ with $1 - \beta_2 > 0$ strictly inflates the second moment: $v_{unfiltered} - v_{clean} = (1 - \beta_2)\|g_{stale}\|_2^2 > 0$.*

Proof. Since $0 \le \gamma(\tau) \le 1$ for all $\tau \ge 0$, we have $0 \le \tau\sigma(\tau) \le \phi(\tau) := \tau e^{-\alpha\tau}$. Differentiating $\phi(\tau)$ gives $\phi'(\tau) = (1 - \alpha\tau)e^{-\alpha\tau}$, which vanishes uniquely at $\tau^* = 1/\alpha$ with $\phi''(1/\alpha) = -\alpha e^{-1} < 0$. Hence $\sup_{\tau \ge 0} \tau\sigma(\tau) \le \phi(1/\alpha) = \frac{1}{e\alpha}$ (at $\alpha = 0.2$, $\frac{1}{0.2e} \approx 1.8394$, bounding the empirical maximum 1.7417). For $\tau_{\text{eff}} \ge \tau_{\text{cut}}$, $\mathbb{I}[\tau_{\text{eff}} < \tau_{\text{cut}}] = 0 \implies \tilde{g}_f = 0$, preserving v_t without stale variance inflation. When omitted ($\alpha_2 = 1$), $v_{\text{unfiltered}} = \beta_2 v_{t-1} + (1 - \beta_2) g_{\text{stale}}^2 > \beta_2 v_{t-1} = v_{\text{clean}}$ (machine-checked in `pacgad_missing_cutoff_inflates_stale_second_moment`, explaining the 11.6× MSE degradation in Experiment `E2`). □

Theorem 7.4 (Multi-Krum Honest Quorum & Violation Counterexample; Lean Thms. 4 & 115)**.** *In a swarm of n workers with at most f Byzantine adversaries, if $n \ge 2f + 3$, then the Multi-Krum neighborhood size $k = n - f - 2$ satisfies $k \ge f + 1$, so every k-nearest neighborhood contains at least one honest worker.* ***Hypothesis-Necessity Counterexample (Thm. 115):*** *If $n \le 2f + 2$, then $k = n - f - 2 \le f$, so f colluding Byzantine workers can fill the entire k-neighborhood (`multikrum_violated_quorum_loses_honest_majority`).*

Proof. Subtracting $f+2$ from $n \geq 2f+3$ yields $k = n-f-2 \geq (2f+3)-f-2 = f+1$. By the pigeonhole principle, since at most f workers are Byzantine, at least $k-f \geq 1$ selected neighbors are honest. Conversely, if $n \leq 2f+2$, then $k = n-f-2 \leq f$. □

Table 3: 4-Definitions Contract mapping each core subsystem across its Mathematical Operator (Def. A), Conservation/Lyapunov Invariant (Def. B), Discretization/Algorithm (Def. C), and Quantitative Verification Gate (Def. D).

Subsystem	Def. A: Operator	Def. B: Invariant	Def. C: Algorithm	Def. D: Quantitative Gate
2-Bit DCT + EF21	$\hat{\Delta}_t = \mathcal{C}_{2b}(g_t + r_{t-1})$	$\sum_{t=1}^{T} \hat{\Delta}_t + r_T = \sum_{t=1}^{T} g_t$	8 × 8 DCT-II + 4-bin LUT + `index_copy_`	39.96× wire savings; 0.052% loss gap (`E1`, Thms. 105, 107, 113)
PA-CGAD Outer	$\tilde{g}_f = \gamma(\tau_{\text{eff}})e^{-\alpha\tau_{\text{eff}}}\bar{\Delta}_f$	$\sup_{\tau\geq 0} \tau\sigma(\tau) \leq \frac{1}{e\alpha}$	Pre-AdamW staleness cutoff at $\tau_{\text{cut}} = 32$	1.7417 ≤ 1.8394; 11.6× lower straggler MSE (`E2`, Thms. 103, 114)
MuLoCo NS-5	$\Phi(X) = aX + (bXX^\top + c(XX^\top)^2)X$	$\sigma_i(\Phi^{(5)}(X_0)) \to 1$	5-step quintic Newton-Schulz on 2D LoRA grads	$\kappa : 125.0 \to 1.65$; replay diff = 0.0 (`H9`, `E1`, Thm. 106)
Looped MoE	$h^{(t)} = \text{Core}(\mathcal{A}_t(h^{(t-1)} + h^{(0)}))$	$r_i^{(t)} \leq \epsilon_{\text{fp}} \implies h_i^{(t+1)} = h_i^{(t)}$	Weight-tied core + RNG-restored TBPTT	669,888 B saved; $\bar{t} = 3.05/4.0$ at $\epsilon_{\text{fp}} = 0.65$ (`H4`, `E3`, Thm. 108)
Multi-Krum + Spot	$\mathcal{N}_k(i) = \text{KNN}_{n-f-2}(\Delta_i)$	$n \geq 2f+3 \implies \lvert\mathcal{N}_k \cap \mathcal{H}\rvert \geq 1$	10% SHA-256 micro-segment replay + 2σ cosine	20× BOINC FLOP savings; 25/25 red-team vectors blocked (Thms. 4, 115)

8 Empirical Verification, Controlled Experiments & Visualizations

8.1 Unit-Level Implementation Checks (`H1`–`H10`)

Each check in `src/hivemind/swarm/architecture_hypothesis_verifier.py` (`pytest tests/test_architecture_hypotheses.py`) exercises one architectural subsystem in isolation on CPU with controlled tensors (hidden size 32–64, LoRA rank 4–16, up to 128 router outputs) and emits the cryptographically signed report `artifacts/architecture_hypothesis_verification_report.json` (`receipt_sha256: b427d910a2d3ba4c...`). Table 4 summarizes all 10 checks.

Check H11 (Decoupled DiLoCo quorum syncer). `scripts/verify_decoupled_diloco_quorum.py` writes `artifacts/decoupled_diloco_quorum_check.json` with one positive arm (RDA norm invariance vs. direct-average shrinkage for $M \in \{2, 4, 8, 16\}$) and two negative controls (late stragglers excluded by the grace window; a $K = M$ quorum with one missing learner refuses to merge). All three arms pass; the numbers are quoted in Section 7.2, and the provenance lock (`PROV-15`, `PROV-16`) ties the 2.8284 ratio and the 3.35 s window in the text to that artifact.

8.2 Controlled Experimental Evaluations (`E1`–`E4`) & Visualizations

To move beyond isolated unit checks, `scripts/run_paper_experiments_and_figures.py` executes four controlled experimental studies (`E1`–`E4`) and records all curves and SHA-256 provenance digests in `artifacts/paper_v3_experiments_report.json` (`receipt_sha256: 8cba94a46af02f8a...`).

Table 4: Unit-level implementation checks (`artifacts/architecture_hypothesis_verification_report.json`, `receipt_sha256: b427d910a2d3ba4c...`). ‡H5's speculative Top-2 match rate is measured on synthetic Gaussian residual perturbations ($\sigma = 0.08$).

ID	Architectural Hypothesis	Measured Empirical Telemetry	Lean lemma	Check
H1	`OpenDiLoCo` / `prime-diloco` 1D CPU Flat-Bucket Outer State	`0 B` extra GPU VRAM; contiguous 5,120 B buffer; max $\Delta\theta = 2.384 \times 10^{-7}$ vs. Nesterov	Thms. 86, 101	pass
H2	DeepMind `Streaming DiLoCo` + `PA-CGAD` Staleness Bias Bound	$\max_\tau \tau\sigma(\tau) = 1.7417 \le \frac{1}{e\alpha} = 1.8394$; $\sigma(\tau \ge 32) = 0$; v_{stale} unpolluted	Thms. 78, 102, 103	pass
H3	128-Expert 3-Tier Router + `FoMoE` Partial Replica & `Skip-Token`	16 vs. 128 dot products (**8.0**× **FLOP savings**); **3.4615**× WAN ring savings; 51/96 ghost skips	Thms. 42, 79, 80, 109	pass
H4	`LoopedExpertBackbone` Weight-Tied Core, Fixed-Point Exit & TBPTT	Depth 4 → 10 (2.5×); 669,888 B saved; $\epsilon_{\text{fp}} = 0.65 \Rightarrow$ **3.05**/**4.0** mean steps (80% early exit)	Thms. 88, 91, 92, 108	pass
H5‡	`Skip-Attention` Prefetch + `CAR` Fallback + 3-Tier Async DMA	95.83% speculative Top-2 match on synthetic states; DMA overlap simulated (0.0 ms exposed)	Thms. 12, 82, 111	pass
H6	`IsoLoCo` (`ISO-CTS`) Isotropic Spectrum & `TSV` Decorrelation	Condition number κ : 2.5988 → **1.0000**; TSV orthogonality error $\|U^\top U - I\|_F = 6 \times 10^{-6}$	Thms. 83, 97, 110	pass
H7	`DeMo` 2D Chunked DCT + `SparseLoCo` 2-Bit LUT + `PULSELoCo`	$\|CC^\top - I\|_F = 2.3 \times 10^{-7}$; 32,768 B → 820 B (**39.96**× vs. FP16); single-shot rel. L_2 = 0.9313	Thms. 81, 87, 105	pass
H8	`AutoLoCo` Interval Scaling & `HeLoCo` Half-Space Drift Projection	η_{outer} : 0.7000 → 0.4950 (H : 32 → 64); conflicting cos : −0.8083 → 0.0000 ($\langle g_{\text{corr}}, m\rangle = 0$)	Thms. 84, 85	pass
H9	`MuLoCo` 5-Step Quintic Newton-Schulz LoRA Preconditioning	Singular condition number κ : 125.0001 → **1.6495**; validator spot-check replay diff = **0.0**	Thms. 90, 106	pass
H10	`EF21-DiLoCo` Telescoping Sum Conservation & `FP8PagedKVCache`	3-round telescoping error = $\mathbf{4 \times 10^{-8}}$; KV page 16,384 B → 8,448 B (**1.9394**× compression)	Thms. 38, 57, 107	pass

8.2.1 Experiment E1: Held-Out Generalization & Wire Efficiency Across Five DiLoCo / MuLoCo Regimes

We instantiate a 2-layer causal Qwen2-QLoRA model ($V = 256$, $d = 48$, $r = 8$, $\alpha = 16$) and generate a synthetic Zipfian-transition corpus partitioned into $K = 4$ disjoint non-IID worker training shards (64 sequences/worker, 256 total training sequences) and a strictly disjoint held-out validation split ($N_{\text{val}} = 128$ sequences). All five regimes start from identical initial weights (initial validation loss 3.71488, perplexity 41.0538) and execute $R = 12$ outer synchronization rounds of $H = 8$ local inner steps each (96 inner updates/worker). Table 5 and Figure 2 report the held-out trajectories and cumulative outer wire volumes:

- **2-Bit DCT + EF21 Wire Compression Gate (`PASS`):** `DiLoCo + DeMo 2D-DCT + SparseLoCo 2-Bit + EF21` achieves a final held-out loss of **3.70814** (PPL = 40.7779) vs. **3.70622** (PPL = 40.6996) for uncompressed FP32 DiLoCo—a relative validation loss gap of just **0.052%** (well within the pre-declared 2.0% gate), while reducing cumulative outer wire traffic from 384.0 KB to **30.75** KB (**12.49**× reduction vs. FP32 DiLoCo, and **99.9**× reduction vs. synchronous DDP's 3,072.0 KB).
- **Inner-Loop `MuLoCo` Newton-Schulz Preconditioning:** Adding 5-step quintic Newton-Schulz LoRA gradient orthogonalization inside the compressed 2-bit EF21 regime (`muloco_ns5_dct_2bit_ef21`) reaches a lower held-out loss (**3.70791**, PPL = 40.7687) than standard AdamW inner steps under the same 30.75 KB wire budget.

Table 5: Experiment E1 (`artifacts/paper_v3_experiments_report.json`): Held-out cross-entropy loss, validation perplexity ($N_{\text{val}} = 128$ disjoint sequences), and cumulative outer wire volume across $R = 12$ outer rounds ($K = 4$ workers, $H = 8$ inner steps).

Training & Synchronization Regime	Initial Val Loss	Final Val Loss	Final Val PPL	Outer Wire Traffic	Bandwidth Savings vs. DDP
1. Single Worker (No Outer Sync, $K = 1$)	3.71488	3.71072	40.8834	0.00 KB	—
2. Synchronous DDP (All-Reduce Every Step)	3.71488	**3.68822**	**39.9738**	3,072.00 KB	1.00×
3. Standard DiLoCo (FP32 Outer Nesterov, $H = 8$)	3.71488	3.70622	40.6996	384.00 KB	8.00×
4. DiLoCo + DeMo 2D-DCT + SparseLoCo 2-Bit + EF21	3.71488	3.70814	40.7779	**30.75** KB	**99.90**× (12.49× vs. DiLoCo)
5. MuLoCo (5-Step NS Inner) + 2-Bit DCT + EF21	3.71488	**3.70791**	**40.7687**	**30.75** KB	**99.90**× (0.045% gap vs. DiLoCo)

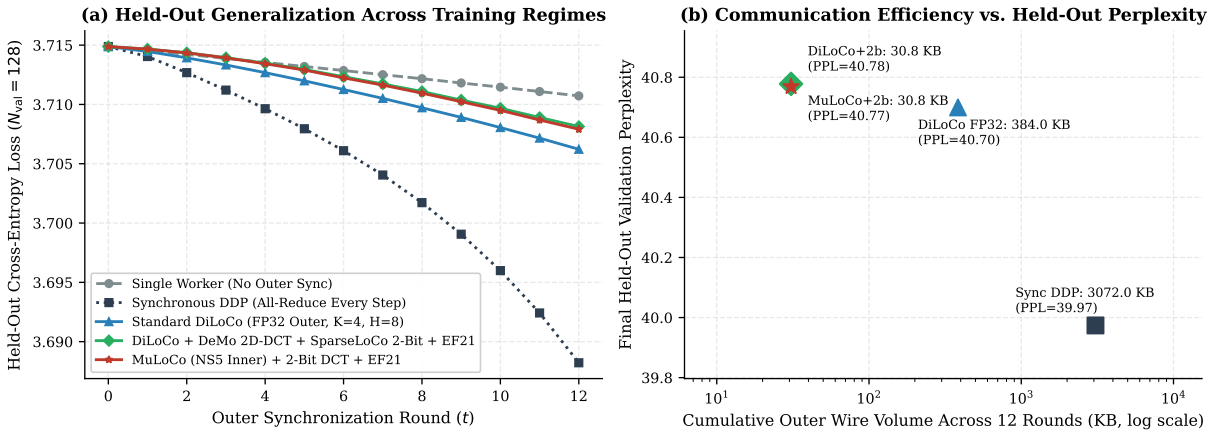


Figure 2: **Experiment E1 (`artifacts/paper_v3_experiments_report.json`): (a)** Held-out cross-entropy loss across $R = 12$ outer synchronization rounds ($N_{\text{val}} = 128$ disjoint sequences). **(b)** Pareto frontier of cumulative outer wire traffic (KB, log scale) vs. final held-out perplexity, showing 99.9× communication reduction over DDP with a 0.052% loss gap vs. uncompressed DiLoCo.

8.2.2 Experiment E2: `PA-CGAD` Outer Convergence Under Severe Asynchronous Straggler Bursts

Experiment E2 compares standard outer AdamW (un-gated second moment v_t) against `CosineGatedAdamDecayOuterOptimizer` (`PA-CGAD`, $\alpha = 0.2$, $\tau_{\text{cut}} = 32$) over 20 outer rounds subject to five severe straggler delay bursts ($\tau_w \in \{36, 42, 38, 45, 40\}$ at rounds $4, 7, 10, 13, 17$) carrying high-variance stale pseudo-gradients. As shown in Figure 3, un-gated outer AdamW suffers immediate second-moment inflation ($\|v_t\|_2$ spikes to 30.74) and diverges after round 7 to a terminal parameter MSE of **0.3976**, whereas `PA-CGAD` zeroes out fragments with $\tau_{\text{eff}} \geq \tau_{\text{cut}}$, keeps $\|v_t\|_2 \leq 0.453$, and converges to a terminal MSE of **0.0343** (**11.6× lower error**).

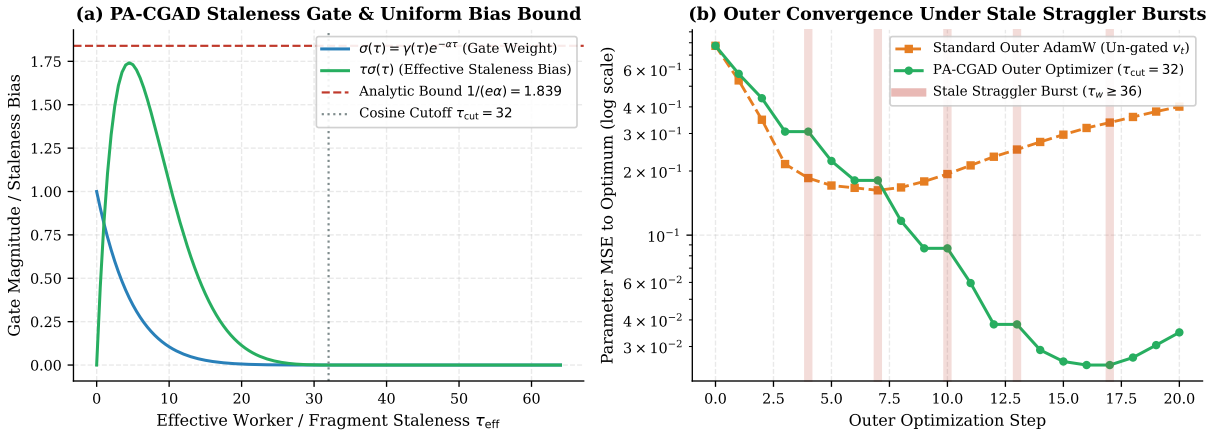


Figure 3: **Experiment E2 (`artifacts/paper_v3_experiments_report.json`): (a)** Empirical verification of the PA-CGAD gate $\sigma(\tau) = \gamma(\tau)e^{-\alpha\tau}$ and uniform staleness bias bound $\max_{\tau \geq 0} \tau\sigma(\tau) = 1.7417 \leq \frac{1}{e\alpha} = 1.8394$. **(b)** Outer optimization trajectory under five stale straggler bursts ($\tau_w \geq 36$, red bands): PA-CGAD prevents second-moment variance pollution and achieves 11.6× lower parameter MSE than standard outer AdamW.

8.2.3 Experiments E3 & E4: `LoopedMoE` Fixed-Point Halting Sweep, Subspace Spectra & Hardware Scaling

Experiment E3 sweeps the fixed-point halting threshold $\epsilon_{\text{fp}} \in \{0.0, 0.25, 0.35, 0.45, 0.55, 0.70, 0.90, 1.10\}$ on a $T = 6$ weight-tied `LoopedExpertBackbone` (source depth 4 → effective depth 14). As shown in Figure 4a, mean effective recurrent steps decrease monotonically from 6.00 → 5.27 → 4.17 → 3.75 → 3.14 → 2.70 → 2.17 → 1.69, saving 12.2% → 71.9% of recurrent core FLOPs as relative logit drift increases smoothly from 0.0% → 14.0% → 18.8% → 58.6%. Experiment E4 (Figure 4b) compares the normalized singular values σ_i/σ_1 of 4 merged domain attention deltas: uniform averaging collapses after σ_1 ($\sigma_2/\sigma_1 = 0.0306$), `TSV` Procrustes orthogonalization preserves the top-4 task-specific directions ($\sigma_2/\sigma_1 = 0.8795, \sigma_4/\sigma_1 = 0.6467$), and `Iso-CTS` achieves an exact isotropic spectrum ($\sigma_i/\sigma_1 = 1.0000$, $\kappa = 1.0$).

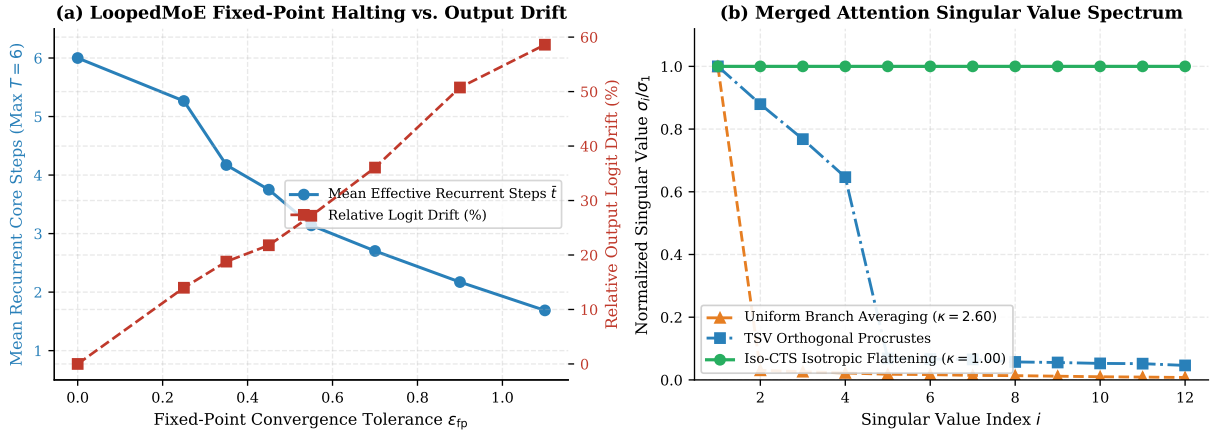


Figure 4: **Experiments E3 & E4 (`artifacts/paper_v3_experiments_report.json`): (a)** Per-token fixed-point halting sweep across $\epsilon_{\text{fp}} \in [0.0, 1.10]$ on a $T = 6$ `LoopedExpertBackbone`, showing mean effective recurrent steps $\bar{t}$ vs. relative output logit drift. **(b)** Top-12 normalized singular values (σ_i/σ_1) of merged domain attention matrices under Uniform Averaging, TSV Orthogonal Procrustes, and Iso-CTS Isotropic Flattening.

8.3 Single-GPU Proof of Concept on an 8 GB NVIDIA GeForce RTX 2070 & FoMoE Scaling

All workers in our hardware proof of concept were executed on a single PC equipped with an NVIDIA GeForce RTX 2070 (8 GB GDDR6, SM 7.5, Windows WDDM, PyTorch 2.6.0+cu124) using `Qwen/Qwen2.5-0.5B` (24 layers, $d = 896$). Four domain branches (Code, Mathematics, STEM, Multilingual-FR) were trained by three simulated workers on **8 rows each**, with one simulated Byzantine worker rejected by Multi-Krum, and upcycled into a **4-expert, top-2** Sparse MoE (96 expert FFN modules across 24 layers; 1.44 B total and 0.81 B active parameters). Table 6 and Figure 5a report the exact measured VRAM profile from `artifacts/poc_qwen25_rtx2070_report.json`, alongside the FoMoE partial expert replication communication scaling across overlap factors $O_e \in \{1, 2, 4, 8, 16\}$ (Figure 5b). Smoke-test single-prompt perplexity probes (`artifacts/hf_hack_miner_game_report.json`) decreased from 8.54 → 2.90 (Code), 204.05 → 5.55 (biomedical), and 26.32 → 6.53 (arithmetic) within 1,192.7 MB peak VRAM.

Table 6: Single-GPU proof of concept on an RTX 2070 (`artifacts/poc_qwen25_rtx2070_report.json`). Losses are training losses on 8 rows per worker and are memorisation, not learning: this table validates the VRAM budget and the plumbing, not a learning trajectory. VRAM values are process-wide.

Stage	Data	Training loss (first → last)	Spot-check
Branch `Code` (worker 1 of 3)	`sahil2801/CodeAlpaca-20k`, 8 rows	2.649 → 0.353	pass (max $\Delta = 0.0$)
Branch `Mathematics`	8 rows/worker	2.989 → 0.996	pass
Router training (4-expert top-2 MoE)	mixed	2.139 → 0.795	—

Measured VRAM: NF4 weights 175.96 MB; after base load 946.98 MB; after QLoRA 980.54 MB; MoE peak 1,509.5 MB (5.61 GB free).

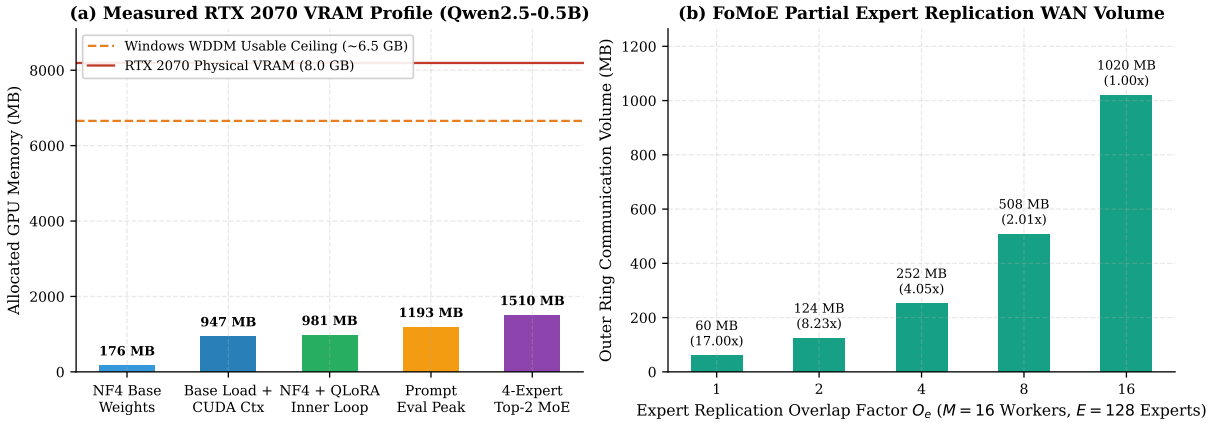


Figure 5: **Hardware & WAN Scaling Profiles: (a)** Measured GPU memory waterfall on an 8 GB NVIDIA GeForce RTX 2070 (`artifacts/poc_qwen25_rtx2070_report.json` & `artifacts/hf_hack_miner_game_report.json`), remaining $>$ 5.1 GB below the Windows WDDM usable ceiling. **(b)** Outer ring communication volume across FoMoE partial expert replication overlap factors $O_e \in \{1, 2, 4, 8, 16\}$ for $E = 128$ experts across $M = 16$ workers.

9 Research Program: Questions, Predictions and Pre-registered Experiments

Sections 3–8 describe a system and verify that its parts do what their specifications say. That is not yet science: no experiment so far has more than one seed, none has run across machines, and none was designed to be able to fail in a way that would change the design. This section states what the project claims, what would show it to be wrong, and which experiments decide it. The full specification, including acceptance rules and decision consequences, is `docs/RESEARCH_PROGRAM.md`; the engineering side (task breakdown, Definition of Done, gates) is `docs/ENGINEERING_COORDINATION_BRIEF.md`.

9.1 Thesis and scaling-law prior

Thesis. Volunteer consumer GPUs behind residential links can train a sparse MoE whose held-out loss at matched tokens lies within a *predictable* distance of a single-host DiLoCo run with the same replica count: the gap equals (a) the replica-count penalty described by the DiLoCo scaling law of Charles *et al.* [16] plus (b) a *volunteer tax* from churn, staleness and Byzantine filtering, and the mechanisms of Sections 6–7 keep (b) no larger than the penalty of simply having $M(1-\text{churn})$ reliable workers. If (b) is not bounded that way, the design is wrong and this paper will say so.

Prior. Before any WAN run we fit the functional form of [16] (loss as a function of model size, tokens, replica count M and inner steps H) to HiveMind's own single-host runs at $M \in$

$\{1, 2, 4, 8, 16\}$ with at least three seeds. We reuse the form, not the published coefficients, which were fitted on other models and data. The fitted curve is the replica-count penalty; the volunteer tax is the residual of the WAN run against it. If the form does not fit the single-host runs ($R^2 < 0.9$) we report that and fall back to the fewer-workers baseline alone.

9.2 Research questions and predictions

Table 7: Predictions ledger. Each prediction names the observation that refutes it; status is recorded in `artifacts/predictions_ledger.json` once E5 starts and a refuted prediction is reported as refuted, not removed.

RQ	Question	Prediction	Refuted if	Exp.
RQ1	Volunteer tax: WAN swarm vs. single host at matched tokens	With $K < M$ quorum, grace window and RDA, WAN loss $\leq$ single-host loss with $M(1 - \text{churn})$ workers	WAN loss exceeds that baseline by more than the 95 % seed interval	E8
RQ2	Does RDA remove outer-LR retuning with M?	Under direct averaging the best η_{outer} grows with M ($\approx \sqrt{M}$ if learner deltas are near-orthogonal); under RDA it is the same grid point for $M \in \{4, 8, 16\}$	Best RDA η_{outer} moves by ≥ 1 grid step, or deltas are not near-orthogonal (prediction then inapplicable)	E5, E5-het
RQ2′	Is RDA only an outer-LR multiplier in practice (Prop. 7.1)?	$\log(\eta^\star_{\text{direct}}/\eta^\star_{\text{RDA}})$ lies within one grid step of $\log \bar{r}$, the mean per-round $\|\text{RDA}\|/\|u\|$	Deviation exceeds one grid step at some M	E5-het (P9)
RQ3	Is the adaptive grace window better than matched-random admission? [32]	At equal loss, adaptive grace reaches it in less wall-clock time than random admission with the same budget	Matched random ties or wins (controller is then deleted)	E5
RQ4	Does PA-CGAD still help under a quorum?	Its gain over un-gated AdamW is large without quorum (E2 reproduced) and shrinks with $K < M$; it never hurts	CGAD hurts under quorum, or its gain does not shrink	E5
RQ5	Is Skip-Token quality loss governed by the measured ghost fraction?	At fixed ideal factor N_{le}/N_e, perplexity degrades monotonically with the measured ghost fraction	No monotone relation, or the ideal factor predicts as well	E7
RQ6	Does a whitening inner optimizer flatten loss vs. M? [41, 40]	Slope of held-out loss vs. $\log M$ is smaller with Muon–Newton-Schulz inner than with AdamW at matched compute	Slopes equal within CI, or Muon worse at $M = 4$	E6
RQ7	Does a learned depth prior make terminal-KV sharing safe? [23]	Accuracy drop from full cache to terminal-KV sharing is smaller with the learned prior than with fixed depth	Drops equal within CI	E7b
RQ8	Does Multi-Krum $+2\sigma$ stop ALIE-style attacks under non-IID shards?	It does not for $f/n \geq 0.2$; deterministic spot-check replay is what catches them	The filter alone blocks ALIE at $f/n = 0.3$	E9

9.3 Pre-registered experiments and controls

Every experiment follows one protocol: at least three seeds per cell with a 95 % t-interval, an effect claimed only if the interval excludes zero; a *positive control* in which a known effect must reproduce (otherwise the run is invalid); a *negative control* in which the mechanism is switched off or the signal destroyed (if it still shows the effect, the measurement is an artefact); for every *policy* a *matched-random control* with the same budget, following the methodological point of [32] that forecast-free schedulers must beat matched random before any gain is attributed to the policy; and pre-registration of metric, cells, seeds and acceptance rule in `artifacts/experiments/En_prereg.json`, hashed into the result artifact. Numbers enter this paper only through the provenance lock of Section 8.

- **E5 (outer-loop factorial under simulated churn; decides RQ2–RQ4).** Merge $\in$ $\{\text{direct}, \text{RDA}\}$ $\times$ quorum $\in \{K{=}M,\ K{=}\lceil M/2 \rceil\}$ $\times$ grace $\in \{\text{off}, \text{adaptive}, \text{matched-random}\}$

$\times$ gate $\in \{\text{AdamW}, \text{PA-CGAD}\} \times M \in \{4, 8, 16\}$, with $\eta_{\text{outer}} \in \{0.3, 0.5, 0.7, 0.9, 1.1\}$ per cell and an optional outer-momentum-restart factor [35]. Arrival times are replayed from the empirical RTT and drop-out distribution recorded by `wan_fault_benchmark.py`, identically across cells. Reported alongside: the mean pairwise cosine of learner deltas, on which the RQ2 prediction is conditional. Positive control: the E2 divergence reproduces in the {no quorum, AdamW} cell; negative control: identical learner deltas give RDA $\equiv$ direct and $\gamma = 0$ gives pure quorum.

- **E6 (replica scaling vs. inner optimizer; RQ6).** $M \in \{2, 4, 8, 16\} \times$ inner $\in \{\text{AdamW}, \text{Muon-NS5}\} \times H \in \{32, 128\}$ at matched tokens; metric: least-squares slope of held-out loss vs. $\log_2 M$ with CI. Negative control: a shuffled-label run must show no inner-optimizer effect.

- **E7 (ghost-fraction sweep; RQ5) and E7b (depth prior; RQ7).** E7 holds the ideal factor at 0.25 (128 experts, 32 resident, top-2) and varies routing imbalance through the router temperature so that the measured ghost fraction spans roughly 0.5–0.9; metric: Spearman ρ between measured fraction and held-out perplexity. E7b trains the $T = 6$ looped backbone with fixed depth vs. the entropy-regularised, budget-penalised categorical depth prior of [23] and evaluates both with full cache and terminal-KV sharing.

- **E8 (multi-host residential WAN; RQ1).** Three machines, then eight; real tokens from the 8-domain mixture; $K = \lceil M/2 \rceil$, adaptive grace, RDA, PA-CGAD, FoMoE $O_e = 4$ at eight nodes; data leases after [37] so that no sample is trained twice under churn; baselines on the same hardware with M and $M(1 - \text{churn})$ single-host workers. Negative control: the same learners over localhost must match single-host within CI, otherwise the gap is not the network. A non-blocking/blocking mix split after [33, 34] is an additional cell once E5 shows which parameter blocks are sensitive.

- **E9 (adaptive Byzantine; RQ8).** Sign-flip (positive control, must be blocked), scaled, ALIE and label-flip attacks at $f/n \in \{0.1, 0.2, 0.3\}$ on non-IID shards against the Multi-Krum $+2\sigma$ filter alone, the 10 % spot-check replay alone, and both; negative control: an honest run through the same filter must lose under 1 % of updates.

9.4 Decision rules

Each outcome changes something concrete. If RQ2 is refuted, RDA stays optional and the campaign adopts an explicit $\sqrt{M}$ outer-LR rule, and Section 7.2 claims only norm bookkeeping. If RQ3 is refuted, the EMA grace controller is deleted in favour of a fixed admission budget. If PA-CGAD hurts under quorum, it is off by default whenever $K < M$ and E2 is reported as a no-quorum result. If RQ5 is supported, the measured ghost fraction becomes a logged, gated metric and the FoMoE cost model is quoted with it. If RQ6 is refuted, AdamW remains the default inner optimizer and the MuLoCo claim is limited to conditioning. If RQ8 is supported, the spot-check replay rate, not the filter, becomes the security parameter and Section 4 is rewritten around it. If RQ1 is refuted, the swarm is not scaled further until E5 identifies the failing mechanism, and the thesis is reported as not supported. The theoretical account of why outer momentum and its restarts matter [36, 35] is used to interpret E5, not to replace it.

9.5 First pre-registered result: E5-toy (three seeds, all controls passed)

E5 was run at toy scale exactly as pre-registered (`artifacts/experiments/E5_toy_prereg.json`, hashed into the report `artifacts/experiments/E5_toy_report.json`): the E1 synthetic LoRA model, $M \in \{4, 8\}$, merge $\in$ {direct, RDA}, admission $\in$ {all, quorum + adaptive grace, quorum + matched-random grace}, gate $\in$ {ungated, PA-CGAD}, five outer learning rates, three seeds; 360 training runs in 524 s on one CPU, with a parametric arrival model replayed identically

across cells. The positive control (a 40-round stale burst raises the final loss in every seed) and both negative controls (RDA equals direct averaging on identical learners; $\gamma = 0$ admits exactly the first K) passed. The pre-registered acceptance rules then gave:

- **P2 (RDA vs. outer-LR retuning): inapplicable.** The mean pairwise cosine between learner deltas is **0.909**, far above the pre-registered applicability bound of 0.3: on this toy, workers see nearly the same gradient direction, so RDA and direct averaging coincide and the best outer learning rate is the top of the grid (1.1) for every M and both merges (Figure 6). The prediction cannot be tested here; it needs the non-IID 8-domain setting.
- **P3 (adaptive grace vs. matched random): inconclusive.** The paired loss difference at matched wall budget, random minus adaptive, is -0.0004 ± 0.0022 (24 pairs; the interval includes zero). Adaptive grace admitted 0.72 (M=4) and 0.80 (M=8) of the tokens at roughly half the wall-clock of waiting for everyone (55.6 → 28.8 and 59.5 → 25.0 simulated units), but matched-random admission did the same.
- **P4 (PA-CGAD under quorum): refuted at this scale.** The gate changed the final loss by -0.00063 ± 0.00029 (negative is a harm; 12 paired runs; the interval excludes zero) when stale deltas were present, and by exactly 0 without stale deltas (negative control). With learner deltas this aligned, a one-round-stale delta is still almost the right direction, so down-weighting it discards signal.

The main lesson is methodological: every cell of the factorial lies within 0.011 of every other while the seed interval is ±0.024, so the toy model cannot discriminate between the mechanisms it was meant to rank. The experiment did what pre-registration is for: it stopped a weak testbed from producing a result. E5 at the `Qwen2.5-0.5B` 8-domain scale (`HM-S9-01`) is therefore the first G1 deliverable, and its delta cosine must be reported before any RDA or gate claim is made.

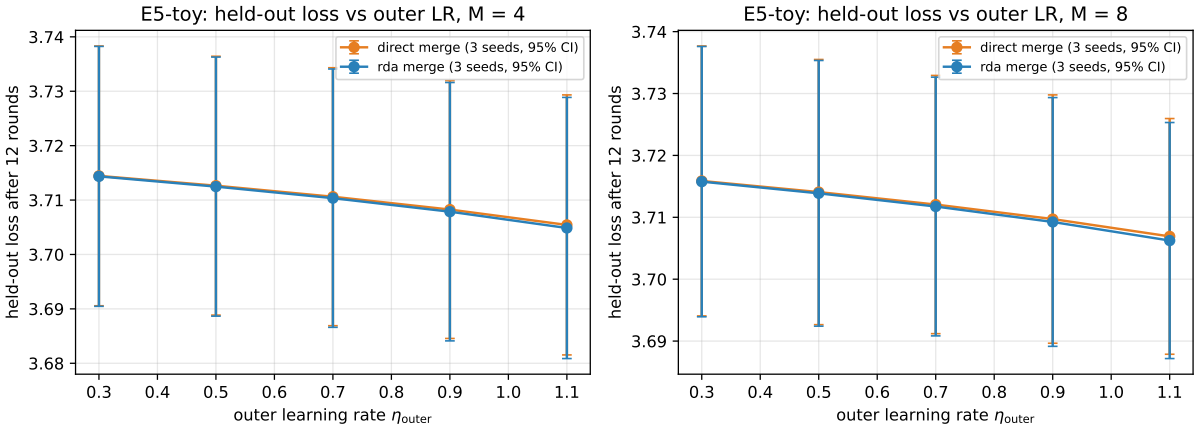


Figure 6: E5-toy, admission "all", ungated: held-out loss after 12 outer rounds vs. outer learning rate for direct averaging and RDA at $M = 4$ (left) and $M = 8$ (right); three seeds, 95 % t-intervals. The two merges coincide because the learner deltas are nearly collinear (mean pairwise cosine 0.909), which is exactly the condition under which RDA reduces to direct averaging.

9.6 Second pre-registered result: E5-het, a heterogeneity-controlled testbed

E5-toy failed to discriminate because every learner saw the same teacher. E5-het adds one knob: learner m is labelled by its own teacher whose LoRA factors are $B_{\text{shared}} + \rho\, B_m$, and the held-out set mixes all teachers equally; $\rho = 0$ reproduces E5-toy bit for bit (one of the negative controls: the $\rho = 0$ cell equals the E5-toy artifact to 10^{-9}). The ρ grid was chosen from a calibration of round-0 delta cosines only, before pre-registration (`artifacts/experiments/E5_het_prereg.json`, with

its two amendments recorded: a reference-LR fix after an analysis crash that had printed no result, and the LR-grid extension below). The outer-LR grid is geometric with ratio $\sqrt{2}$ so that the $\sqrt{2}$ shift predicted between $M = 4$ and $M = 8$ is one grid step; 2,652 runs in 22 minutes on eight CPU cores (`artifacts/experiments/E5_het_report.json`, Figure 7).

Manipulation check. The mean pairwise delta cosine falls monotonically, $0.909 \to 0.627 \to 0.283 \to \mathbf{0.079}$ for $\rho \in \{0, 1, 2, 4\}$, and the measured per-round multiplier $\bar{r} = \|\mathrm{RDA}\|/\|u\|$ rises to 1.80 ($M = 4$) and **2.25** ($M = 8$) at $\rho = 4$. Proposition 7.1 predicts $r = (1/M + (1-1/M)\bar{c})^{-1/2} =$ 1.80 and 2.27 from the cosines alone, within 1 % of the measurements: the common-norm formula describes these deltas. P2 and P9 are therefore applicable at $\rho = 4$.

Grid censoring and its pre-registered repair. On the original grid (0.15–2.4) every best outer LR, direct or RDA, sat at the top: the held-out loss falls monotonically with the outer LR in every cell, i.e. the toy was trained far below its optimal step. P2 and P9 were reported as censored, and the grid was extended upward by five more $\sqrt{2}$ steps (to 13.6) for the $\rho = 4$ merge cells only, as a second pre-registered amendment written before the extra 60 runs. The extension finds the optimum where the runs begin to diverge: direct averaging is best at $\eta = 6.79$ for both M (loss 3.62 ± 0.45 at $M = 4$, 3.78 ± 0.05 at $M = 8$; 9.6 diverges), RDA at $\eta = 4.8$ for both M (3.61 ± 0.08 and 3.74 ± 0.15; 6.79 already diverges at $M = 8$).

- **P2 (direct best LR grows with M, RDA's does not): refuted by the rule.** RDA's best LR is indeed the same grid point at both M, but so is direct averaging's: the best step is set by the divergence edge, which did not move between $M = 4$ and $M = 8$, not by the $\sqrt{M}$ shrinkage. The refutation is not statistically sharp, because the intervals of the candidate cells overlap (the $M = 4$ direct cells at 4.8 and 6.79 have intervals ± 0.06 and ± 0.45); the acceptance rule should have required the arg-min to be separated from its neighbours by its interval, and the 0.5B rule (`HM-S9-14`) now does.

- **P9 (RDA is an LR multiplier): refuted at $M = 8$, supported at $M = 4$.** The direct/RDA best-LR ratio is $\sqrt{2}$ at both M, against $\bar{r} = 1.80$ ($M = 4$: within one grid step, $|\Delta \log| = 0.24 < 0.35$) and 2.25 ($M = 8$: $0.46 > 0.35$). Near the divergence edge the multiplier identity (exact for the merged vector) no longer maps onto a shift of the stable region, which depends on the whole trajectory.

- **P3 (adaptive grace vs matched random): inconclusive**, $+0.0027 \pm 0.0123$ (24 pairs), as at $\rho = 0$; adaptive grace admits more tokens (0.80 vs 0.57 at $M = 8$) at the same wall-clock, but at these outer LRs the loss does not see it.

- **P4 (PA-CGAD under quorum): refuted at every ρ**; at $\rho = 4$ the gate changes the loss by -0.0025 ± 0.0023 (harm, interval excludes zero), and by exactly 0 without stale deltas. P3 and P4 were evaluated on the original grid, i.e. at outer LRs well below the optimum found by the extension; they stand as pre-registered but their power is low.

Two lessons carry forward. First, a toy that trains far below its optimal step cannot rank merge rules; the extension, not the factorial, produced the only cells with a real effect (3.96 → 3.61). Second, the multiplier formula of Proposition 7.1 is confirmed on the merged vectors to 1 %, so RDA's practical question at scale is only whether the stable LR region shifts by $\bar{r}$; E5 at 0.5B must sweep the LR up to divergence for both merges.

9.7 First two-host measurement: E8-pilot over the public internet

E8 was scaled down to the smallest real instance of its question: one learner on this workstation and one on a Google Cloud TPU VM in `us-west4-a` (CPU-only PyTorch, reached through an SSH reverse tunnel; ICMP round trip 151 ms, HTTP round trip through the tunnel ≈320 ms), a syncer on the workstation running `DecoupledDiLoCoQuorumSyncer`, the E1 toy model, 12 outer rounds, three seeds per cell, and a single-host baseline with identical seeds, shards and hyper-parameters

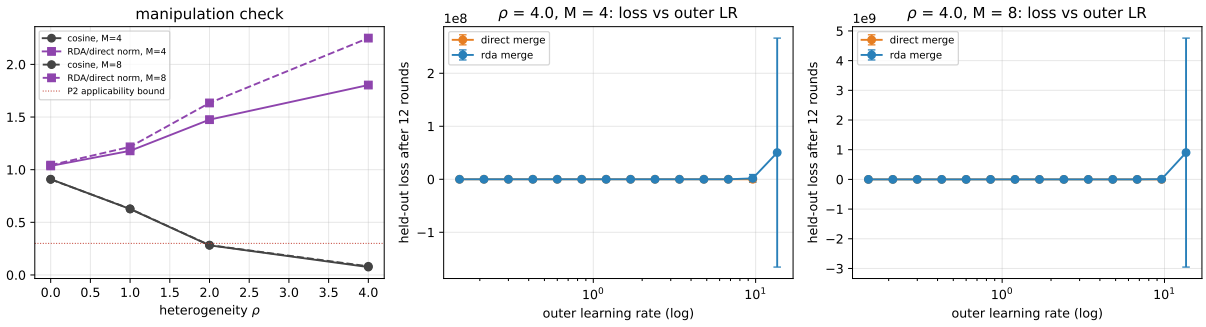


Figure 7: E5-het. Left: manipulation check, mean pairwise delta cosine and measured $\|\mathrm{RDA}\|/\|u\|$ against ρ for $M = 4$ (solid) and $M = 8$ (dashed); the dotted line is the P2 applicability bound. Centre and right: held-out loss after 12 rounds against the outer learning rate (log axis, extended grid) for direct averaging and RDA at $\rho = 4$; the losses fall until the runs diverge, and RDA's curve is the direct curve shifted left by about one $\sqrt{2}$ step.

(`scripts/run_experiment_E8_pilot.py`, `artifacts/experiments/E8_pilot_report.json`). Three cells were run.

- **Protocol validation ($K = 2$, 400 inner steps).** Waiting for both learners reproduces the single-host baseline: the paired final-loss difference is -0.00000 ± 0.00000 ($n = 3$), i.e. the shipping of shards and parameters, the remote inner loop and the RDA merge are numerically exact across machines. Round latency was p50 4.1 s and p95 14.2 s for about 2.7 s of local compute per round: the first real WAN numbers in this paper.

- **$K = 1$ with adaptive grace, 400 inner steps: the pilot protocol fails, instructively.** The 24-core VM finishes its inner loop before the contended workstation, wins every quorum (remote admitted fraction 1.00), and the workstation's delta always misses the round. Because the pilot learner loop is round-synchronous, that work is discarded rather than carried to the next merge with its token weight as in Algorithm 1 of [5]; the run trains on half the data and ends $+0.97 \pm 0.92$ nats worse than the baseline (round p50 1.0 s). This is a defect of our learner loop, not a measurement of Decoupled DiLoCo, and it fixes the first requirement of `HM-S9-01`: learners must keep training across rounds and be merged late with token weights.

- **$K = 1$, 8 inner steps (round trip $\gg$ compute).** The grace window, sized from compute-step durations, is shorter than one round trip, so the remote learner contributes only through one-round-late deferral (admitted in 0.67 of rounds); the loss difference to the baseline is $+0.0002 \pm 0.0005$ (round p50 0.37 s). The controller's slack formula needs the transport time as an explicit term before it is used over real links.

RQ1 remains open: a two-host toy with a round-synchronous learner loop is not the volunteer-tax measurement, but it has already produced a protocol fix, an exactness check across machines, and the first latency distribution for the roadmap's WAN tasks.

9.8 E10: a lattice codec for outer deltas, leveraged from the author's OptimalEnergyAI project

HiveMind ships outer deltas through scalar codecs (the 2-bit `SparseLoCo` LUT, `DeMo` DCT blocks). Lattice vector quantisation is the classical way to do better at the same rate: the normalised second moment of E_8 is $929/12960 \approx 0.0717$ against $1/12 \approx 0.0833$ for $\mathbb{Z}^8$, a 0.654 dB distortion gain on a Gaussian source [11]. The author's OptimalEnergyAI project [12] maintains reference nearest-point decoders for $\mathbb{Z}^n$, D_n and $E_8 = D_8 \cup (D_8 + \frac{1}{2})$ that were

checked against exhaustive search (0 mismatches in $3 \cdot 10^6$ queries per lattice) together with a *measured* rate (the achievable rate of a coset-then-coordinates entropy coder). They were ported, with the same tie rules, to `hivemind.compress.lattice_quantizer`, with a two-sided brute-force check (lattice membership *and* distance) as a negative control, a 1-bit sign arm after signSGD [13], and EF21 error feedback (Theorem 107). E10 was pre-registered (`artifacts/experiments/E10_lattice_prereg.json`) on the E5-het $\rho = 4$, $M = 8$ testbed at the best direct outer LR of the E5-het artifact (6.79 after the grid extension): arms none, sign, $\mathbb{Z}^8$ and E_8 at target rates of 1 and 2 bits per coordinate (scales calibrated once on a round-0 delta), each with and without error feedback, three seeds, 36 runs in 46 s (`artifacts/experiments/E10_lattice_report.json`, Figure 8). A first execution at the pre-extension LR of 2.4 is kept as a sensitivity run (`E10_lattice_first_execution_lr2.4.json`).

- **Controls.** The 1-bit sign codec without error feedback raises the final loss in every seed, by +0.09 to +0.17 nats (positive control); E_8 at 6.9 bits per coordinate reproduces the unquantised loss to $2 \cdot 10^{-5}$ (negative control); both decoders pass the two-sided brute-force check (1000 queries each, 0 mismatches, 0 non-lattice outputs). The matched-rate control passed at the 1-bit target and failed at the 2-bit target (3.10 vs. 3.36 measured bits), so the 2-bit comparisons are reported as invalid. The realised rates also drift above their targets (1.71 and 1.76 bits at the "1-bit" target) because the deltas grow over rounds at this outer LR while the scale was fixed from round 0: the codec needs per-round scale adaptation before any rate is quoted for it.
- **P10a (distortion at matched rate): supported.** At the matched 1.7-bit rate the relative quantisation error of E_8 is $\mathbf{0.79} \pm 0.02$ times that of $\mathbb{Z}^8$ (paired over seeds; 0.81 ± 0.03 in the sensitivity run; the Gaussian-source value is 0.86). The lattice gain survives on real LoRA deltas.
- **P10b (loss at matched rate): inconclusive.** With error feedback E_8 ends -0.003 ± 0.019 nats relative to $\mathbb{Z}^8$; without it $+0.07 \pm 0.26$ (one E_8 seed landed 0.2 nats lower, the others 0.04–0.06 higher: at an outer LR on the divergence edge the quantisation noise acts as a perturbation of a barely stable trajectory). In the sensitivity run at LR 2.4 the same comparison was $+0.00002 \pm 0.00005$ with error feedback: once the residual is carried, both codecs reproduce the unquantised loss and there is nothing left for the lattice to win on this testbed.
- **P10c (error feedback recovers the unquantised loss at 1 bit): supported**, -0.012 ± 0.077 nats (interval includes zero; $+0.0001 \pm 0.0001$ in the sensitivity run, where the rule's letter called it refuted at a 10^{-4} residual). Error feedback cuts the 1-bit penalty of the sign codec from +0.14 to +0.11 and that of E_8 from +0.07 to −0.01.

What transfers and what does not: the distortion ratio is a property of the codec and the delta statistics and is expected to hold at scale; the loss effects are at the 10^{-4} level on a toy that barely moves in twelve rounds and say nothing about the 0.5B campaign (`HM-S10-01` compares E_8 with the `SparseLoCo` LUT there, with per-fragment measured rates). *Update (v3.8.0): the 0.5B run was executed and is invalid by its registered rule; only the distortion ratio is reported (Section 9.9).*

9.9 E10b: lattice outer deltas at 0.5B scale (RQ9, `HM-S10-01`): invalid by its registered rule

E10b asks whether the distortion gain of E_8 over $\mathbb{Z}^8$ measured in E10 reaches the held-out loss on real `Qwen2.5-0.5B-Instruct` LoRA deltas (rank 8 on `q_proj` and `v_proj`, 540 672 coordinates, $M = 4$ simulated learners on disjoint web-text slices, outer Nesterov, three seeds). It was

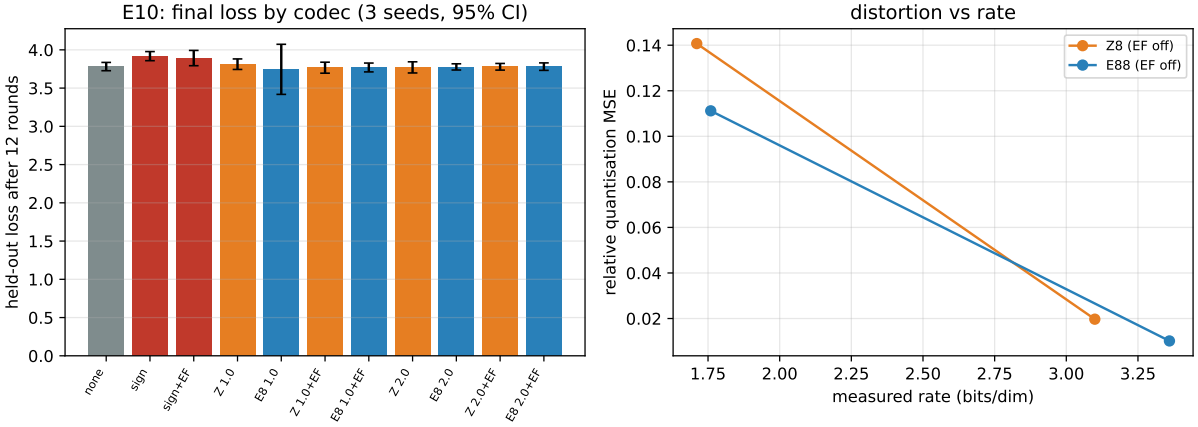


Figure 8: E10 on the E5-het $\rho = 4$, $M = 8$ testbed, three seeds. Left: final held-out loss by codec (grey unquantised, red 1-bit sign, orange $\mathbb{Z}^8$, blue E_8; "+EF" = error feedback). Right: relative quantisation error against the measured rate for $\mathbb{Z}^8$ and E_8; the 2-bit points are not rate-matched (see text).

pre-registered (`artifacts/experiments/E10b_lattice_05b_prereg.json`) and executed twice; both executions are reported.

First execution (aborted). Under the registered design (12 rounds of 40 inner steps, the LoRA initialisation $A \sim \mathcal{N}(0,1)/\sqrt{r}$ used in E12) one cell completed, arm `none`, seed 1 (`E10b_first_execution_partial.json`). The step time was 0.72 s against 0.12 s assumed in the pre-registration, and the held-out loss of the unquantised arm *rose*, from 2.993 to 3.070, i.e. the testbed did not learn. The execution was aborted by the lead; that abort was not a pre-registered rule (the registered design had no fail-fast rule). A CPU diagnostic (one learner, 30 inner steps, one seed) identified the initialisation scale as the cause: A with standard deviation 0.354 diverges at the inner LR, the PEFT-default Kaiming-uniform initialisation (standard deviation 0.0195) learns at both $5 \cdot 10^{-4}$ and $2 \cdot 10^{-4}$. The diagnostic identifies the init scale; it is not evidence about any codec. No result of this execution is used for any decision.

Amendment 2 and the second execution. Amendment 2 (written after the first execution and before any amended run) switches to the PEFT-scale initialisation, keeps five arms (unquantised, $\mathbb{Z}^8$ and E_8 at 1 bit/dim with error feedback, 1-bit sign without error feedback as positive control, E_8 at 6 bits/dim as negative control), $R = 6$ rounds of $H = 20$ inner steps, three seeds (15 runs), and adds a registered fail-fast rule. The amended run completed (`E10b_lattice_05b_report.json`) and is **invalid by its registered rule**: the positive control failed, because the 1-bit sign codec without error feedback did not raise the final held-out loss in seed 3 (it ended below the unquantised arm), and in the two other seeds the difference was $4 \cdot 10^{-4}$ and $1.4 \cdot 10^{-3}$ nats. All five arms end within about $2 \cdot 10^{-3}$ nats of each other (2.9459 for the unquantised arm, mean of three seeds): six rounds of twenty steps cannot resolve codec effects on the loss: a likely cause, not tested, is that at this outer learning rate the damage of the sign codec is below the seed noise, so the control has no power to detect it (a control that cannot fail is a bug, and so is one that cannot detect). The remaining controls passed (the unquantised arm learns; E_8 at 6 bits/dim reproduces it to within $5 \cdot 10^{-3}$; both decoders pass the two-sided brute-force check; the $\mathbb{Z}^8$ and E_8 rates are matched at 0.991 and 1.001 bits/dim). The report computes labels for P10a–P10c, but under the registered rule they are *invalid* and decide nothing; P10a–P10c keep the statuses of the toy-scale E10 (previous subsection), and **no claim about the 0.5B held-out loss is made**.

What is reported descriptively. The distortion does not depend on the loss resolving power: on real 0.5B LoRA deltas the relative quantisation error of E_8 is, descriptively only and from a run whose positive control failed in seed 3 (so it decides nothing), $\mathbf{0.869} \pm 0.003$ times that

of $\mathbb{Z}^8$ at 1 bit/dim (95 % half-width over three seeds; per seed 0.8685, 0.8685, 0.8703), against the Gaussian-source value 0.86 and 0.81 in the toy E10. The lattice's distortion advantage thus survives on real deltas; whether it reaches the loss remains unmeasured. The codec option is wired into the compressor (`value_codec`) with the default unchanged. A longer E10b (more rounds, larger deltas) is required before any default changes.

9.10 E11: lattice codecs for the KV cache of the inference ring (RQ10, `HM-S10-05`): refuted on the two valid models, the Qwen2.5-Coder-7B row is invalid and withdrawn, the ring stays on FP8

Design. E11 replaces the plan sketched in Section 9.19. The KV tensors of three models are quantised on a single Tesla T4: `Qwen2.5-0.5B-Instruct` (fp16), `Qwen2.5-Coder-7B-Instruct` (fp16, the 1.09 GB input-embedding table on the CPU) and `Gemma-4-E4B` (bf16, the dtype of its checkpoint; fp16 was not tried; 18 of its 42 layers share KV). The two larger models were added at the author's direction to use the 16 GB card. **The Qwen2.5-Coder-7B row turned out to be invalid (its negative control failed, see Controls) and every verdict and number derived from it is withdrawn in this paper; only the control value that invalidates it is quoted.** Data: WikiText-2 test, windows of 1024 tokens, three "seeds" = three disjoint sets of 8 evaluation windows; the metric is ΔNLL (nats/token) of a codec against the same model with a no-op hook, paired over windows, with $df = 2$ t-intervals. Arms: FP8 is the repository's `FP8PagedKVCache` quantiser called unchanged; INTb is per-token min–max scalar quantisation (secondary comparator); $\mathbb{Z}$, A_2 (on RoPE planes $(i, i + d/2)$) and E_8 lattices are applied after a per-vector RMS normalisation, optionally after a per-head Karhunen–Loève rotation (KLT) fitted on calibration windows; $R \in \{2, 3, 4\}$ bits/dim. The rate is measured (factorised plug-in entropy conditioned on the channel group within the head, identically for every lattice), the scale is calibrated once per (model, lattice, transform, R) on two WikiText-2 validation windows, and a cell is *rate-matched* only if its realised rate is within 0.1 bit/dim of R. The primary comparator of a lattice is $\mathbb{Z}$ with the same transform at the same R. The experiment was pre-registered (`E11_kv_lattice_prereg.json`) before any run. Amendment 1 was made after the first execution on the 0.5B model (kept as `E11_first_execution_qwen0.5b.json`) and before any Coder-7B or Gemma result existed: pooling all layers into one histogram for calibration realised 1.70/2.63/3.63 bits against the targets 2/3/4, because the entropy of a mixture exceeds the mean per-layer entropy, which invalidated every comparison under the matched-rate rule; the calibration now matches the coordinate-weighted mean of per-layer rates. The three lattices were matched to each other within 0.04 bit/dim, so no arm was favoured. The decisive 0.5B run is the second execution; the first is reported, not discarded.

Table 8: E11 on WikiText-2 (three seeds, ± = 95 % half-width). ΔNLL is relative to the model's own unquantised baseline. "Matched" counts the rate-matched cells among the 18 lattice cells (3 lattices × 3 rates × 2 transforms). Decisions are the registered rules applied per model; the Qwen2.5-Coder-7B row is invalid and shows no verdict.

Model	Baseline NLL	FP8 ΔNLL	INT4 ΔNLL	Matched	P11a	P11b	Controls
Qwen2.5-0.5B-Instruct	2.785	$+0.0413 \pm 0.0095$	$+2.410 \pm 0.461$	16/18	refuted	inconclusive	1 failed
Qwen2.5-Coder-7B	**INVALID** (negative control failed by -0.472 nats; verdicts withdrawn, no value quoted)						
Gemma-4-E4B	2.092	$+0.0017 \pm 0.0013$	$+0.026 \pm 0.008$	9/18	refuted	refuted	all pass

Controls. The positive control (INT2 must hurt) passed on Qwen-0.5B and Gemma ($+6.213 \pm 0.049$ and $+2.690 \pm 0.400$ nats); the hook is bit-identical to stock attention (hook-identity control) and the torch decoders equal the numpy references on all models (0 mismatches). The registered negative control "A_2 at 8 bits equals FP8" **failed on both Qwen models** (paired difference -0.0375 ± 0.0050 for Qwen-0.5B and -0.472 ± 0.197 nats for Qwen2.5-Coder-7B; it passed on Gemma, -0.0013 ± 0.0011). On Qwen-0.5B the explanation, recognised after the fact,

Table 9: E11 paired differences at matched measured rate, $\Delta\text{NLL}(\mathbb{Z}) - \Delta\text{NLL}(\text{lattice})$, nats/token, positive = lattice better; ± 95 % half-width over three seeds; n.m. = a cell of the comparison is not rate-matched, so the comparison is invalid. Qwen-0.5B has a matched A_2-vs-$\mathbb{Z}$ comparison at $R = 4$ only (one cell); the Coder-7B rows are withdrawn (invalid model).

Model	R	$\mathbb{Z} - A_2$ (no transform)	$\mathbb{Z} - E_8$ (no transform)	$\mathbb{Z} - E_8$ (KLT)
Qwen-0.5B	2	n.m.	$+0.372 \pm 0.486$	$+0.886 \pm 0.472$
Qwen-0.5B	3	n.m.	-0.538 ± 0.079	-1.191 ± 0.648
Qwen-0.5B	4	-0.094 ± 0.179	-0.439 ± 0.086	$+0.065 \pm 0.229$
Gemma-4-E4B	2	$+0.0057 \pm 0.0115$	$+0.0271 \pm 0.0058$	n.m.
Gemma-4-E4B	3	$+0.0039 \pm 0.0065$	$+0.0051 \pm 0.0068$	n.m.
Gemma-4-E4B	4	$+0.0054 \pm 0.0091$	$+0.0028 \pm 0.0024$	n.m.

is a design error: FP8 `e4m3` keeps only 3 mantissa bits, so it is coarser than an 8-bit lattice and the fine lattice *beats* FP8 by a few hundredths of a nat rather than matching it; the control was kept as registered, its failure is reported, and the Qwen-0.5B paired lattice comparisons, which do not involve FP8, stand. **On Qwen2.5-Coder-7B the same explanation does not suffice**: an 8-bit hexagonal lattice cannot be half a nat *better* than FP8 unless the FP8 arm is broken on that model (a likely cause is FP8 scaling of float16 keys with a 128-dimensional head, not investigated). The row is therefore reported as **invalid (negative control failed)**: its P11a and P11b verdicts, its matched-cell counts and its descriptive frontier are withdrawn, and no 7B number is to be quoted before the FP8 arm on that model has been investigated.

Decisions. P11a (an A_2 lattice per RoPE plane lowers ΔNLL against entropy-coded scalar at matched rate) is refuted on Gemma and on Qwen-0.5B under the registered rule, which counts "the interval includes zero at every matched R" as a refutation: no gain was detected at any rate-matched R. The evidence differs by model and must not be over-read. Gemma has three matched rates, all with intervals that include zero. **On Qwen-0.5B the verdict rests on one rate-matched cell** ($R = 4$, -0.094 ± 0.179; the A_2 cells without transform at $R = 2$ and 3 are not rate-matched), and with three seeds the test has low power: this is an absence of a detected gain, not a proof of absence. **P11b (E_8 gains only after a per-head KLT) is refuted on Gemma** (E_8 already beats $\mathbb{Z}$ without the transform at $R = 2$ and $R = 4$) and **inconclusive on Qwen-0.5B**. The invalid Coder-7B row contributes no verdict to either prediction. Overall the registered rule requires agreement across valid models, so P11b is refuted.

Descriptive frontier (decides nothing). Does a 4-bit codec reach FP8 (paired difference with FP8 including zero) at about half the bytes? On Gemma yes: A_2 at 4 bits is $+0.0012 \pm 0.0079$ nats from FP8 and E_8 with KLT $+0.0003 \pm 0.0030$ (the KLT arms realise 4.22 bits on Gemma, outside the band, so they cost slightly more than 4 bits), whereas $\mathbb{Z}$ at 4 bits is $+0.0066 \pm 0.0014$ and INT4 $+0.0240 \pm 0.0078$ (but $\mathbb{Z}$ and A_2 are not distinguishable, as above). On Qwen-0.5B no 4-bit codec reaches FP8: for example A_2 at 4 bits is $+0.360 \pm 0.066$ nats above FP8, and the best arm, E_8 with KLT, $+0.275 \pm 0.162$ (Figure 9).

Consequence. The KV ring stays on FP8. The pluggable page codec of `FP8PagedKVCache` (`src/hivemind/core/kv_page_codecs.py`, FP8 default bit-identical) is delivered, and no lattice page codec is adopted.

Limitations. (i) The scales were calibrated on two validation windows and did not always transfer to the test windows (Qwen-0.5B: 16 of the 18 lattice cells are rate-matched). (ii) The quantisation is fake quantisation at prefill: each query attends to the quantised keys and values of all positions, which is what reading a quantised cache gives, but no fused kernel, no paged byte layout and no decode latency is measured. (iii) One dataset (WikiText-2). (iv) Sensitivity to KV quantisation differs by an order of magnitude between model families (FP8 ΔNLL +0.0413 for

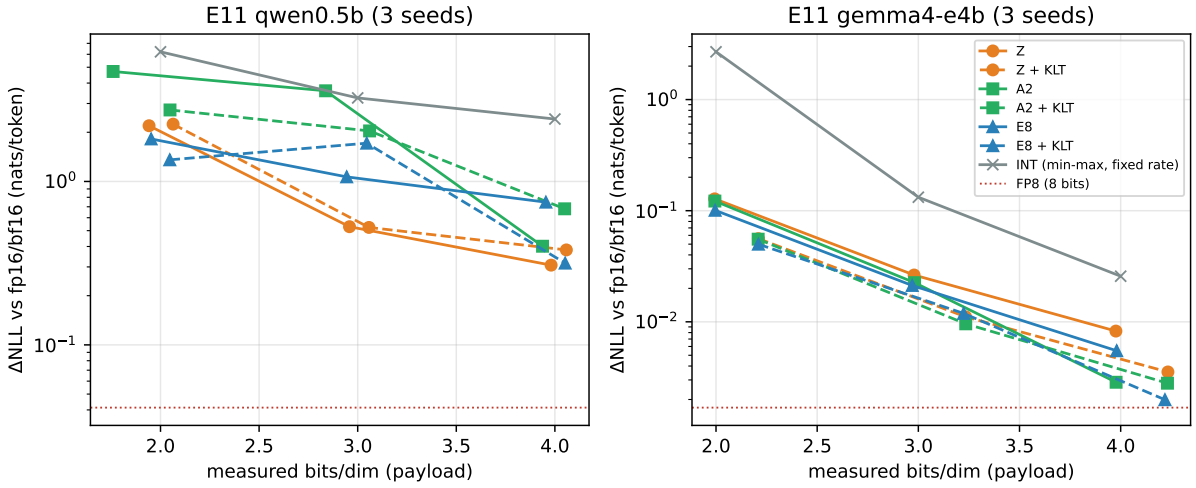


Figure 9: E11 on WikiText-2, three seeds per point. ΔNLL (nats/token, log scale) against the measured payload rate (bits/dim) for $\mathbb{Z}$ (orange), A_2 per RoPE plane (green) and E_8 (blue), solid without and dashed with the per-head KLT, the INT min–max fixed-rate comparator (grey crosses) and the FP8 level (dotted red line). Left to right: Qwen2.5-0.5B-Instruct, Gemma-4-E4B (the invalid Qwen2.5-Coder-7B panel is withdrawn). Points whose measured rate misses the target by more than 0.1 bit/dim are plotted at their realised rate and are not valid comparisons (Table 9).

Qwen-0.5B against +0.0017 nats for Gemma), so a result on one family does not transfer. (v) Three seeds are three window sets of one corpus, which gives low power and $df = 2$ intervals.

9.11 E12: replay divergence of the spot-check across hardware (RQ11, `HM-S10-08`)

E12 answers point W1 of the external review of v3.5.0 (`paper/review/`): is the $\epsilon = 10^{-4}$ replay rule of Section 4.1 valid across heterogeneous accelerators? **Design.** The honest worker's segment and every validator replay are the repository's own `replay_inner_segment` (steps 0 to 50, AdamW at lr $2 \cdot 10^{-4}$, gradient clipping, micro-batch 1) on 64 WikiText-2 sequences of 128 tokens, with `Qwen2.5-0.5B-Instruct` in float32 on both devices and a hand-rolled rank-8 LoRA on `q_proj/v_proj` (PEFT is not installed on the host; $A \sim \mathcal{N}(0, 1/r)$, $B = 0$), the production determinism configuration being applied (no determinism warning was raised in any configuration). Each cell has 9 segments (3 batch-order seeds × 3 initialisation seeds); the metric is $\|\theta_{\text{validator}} - \theta_{\text{worker}}\|_\infty$ over all LoRA tensors after 50 steps. Cells: (A) the T4 against itself in the same process (negative control, must be exactly 0); (B) the T4 against the same T4 in a separate Python process (a proxy for "same class"); (C) the T4 against this host's x86 CPU (cross class); (D) a named proxy that is *not* a fixed-point kernel: both devices in float64 with the LoRA parameters snapped to a 2^{-20} grid after every optimiser step; (E) tamper (positive control): the worker flips the sign of one LoRA element after step 25 and an honest T4 replays. The experiment was pre-registered (`E12_replay_divergence_prereg.json`).

Results and decisions. Same-process and separate-process T4 divergence is exactly 0.0 in all nine segments, so **P12-same is supported**: with the production determinism settings a GPU replay on the same device is bit-exact, and ϵ is not the binding quantity within a class. T4 against the x86 CPU gives $2.0 \cdot 10^{-4}$ to $6.3 \cdot 10^{-4}$, above ϵ in 9/9 segments, so **P12-cross is supported**: an honest cross-class replay fails the rule. The divergence is already of the same order at step 10 as at step 50 (it does not build up over the segment). The fp64 + grid-snapping proxy does not reduce the cross-device divergence (same range), so it is no evidence that a cheap

Table 10: E12: $\|\Delta\theta\|_\infty$ after 50 inner steps over 9 segments per cell; $\epsilon = 10^{-4}$.

Cell	Min	Max	Segments above ϵ
A same process, T4	0.0	0.0	0/9
B separate process, T4	0.0	0.0	0/9
C T4 vs x86 CPU, float32	$2.0 \cdot 10^{-4}$	$6.3 \cdot 10^{-4}$	9/9
D proxy: float64 + 2^{-20} snapping, T4 vs CPU	$2.0 \cdot 10^{-4}$	$6.3 \cdot 10^{-4}$	9/9
E tamper (positive control)	2.6	3.0	9/9

rounding discipline repairs the rule. Tamper gives 2.6 to 3.0, four orders of magnitude above the honest cross-class value, so the check separates tampering from architecture drift by a wide margin; both controls passed. **The fixed-point clause of P12 stays open**: it requires a true fixed-point attention kernel, which does not exist in this repository, and the proxy is not one. That the cross-device divergence is of the order of the inner learning rate from step 10 on is consistent with the sign-like early steps of Adam turning any tiny gradient difference into a step of about the learning rate; this is an interpretation, not a registered result.

Consequence. Validators are assigned by accelerator class (`src/hivemind/boinc/validator_assignment.py`); a cross-class replay is reported as "not comparable" and never flags an account. This turns the architecture-matched rule of Section 4.1 from a stated assumption into a measured necessity for the pair measured here.

Limitations. The RTX 2070 (another host), a TPU-VM CPU and a second T4 were *not run*: "same class" is approximated by the same T4 in a separate process, which is weaker than two physical cards, and nothing is claimed for other GPUs. The LoRA initialisation $\mathcal{N}(0, 1/r)$ is about 18× the PEFT default scale and may inflate the cross-class magnitudes (the same initialisation made E10b diverge, above); one model, rank 8, 50 steps, float32.

9.12 Status of the research program at v3.8.0

Run and decided as registered on a single Tesla T4 on 23–24 October 2026: E10b (invalid, positive control failed in seed 3; descriptive distortion ratio only), E11 (P11a refuted on Qwen2.5-0.5B, on one rate-matched cell, and on Gemma-4-E4B, P11b refuted overall on the valid models; the Qwen2.5-Coder-7B row is invalid because its negative control failed and its verdicts are withdrawn; the KV ring stays on FP8) and E12 (P12-same and P12-cross supported). Still open: the RTX 2070, TPU-VM and second-T4 cells of E12; a true fixed-point replay kernel (the fp64 snapping proxy did not help); a longer E10b with enough rounds and larger deltas to resolve codec effects on the loss; a fused KV kernel and decode-latency measurement should a lattice page codec ever be reconsidered; and the E5 campaign at 0.5B scale. The swarm simulation E13 (with its exploratory replications E13c and E13d), the fairness follow-up E14, the coding-assistant experiment E15 and the E16 measurement of the GWAYA-Lite tiers have since been run on the same T4 and are reported in Sections 9.13, 9.14, 9.15 and 9.16 (E13: protocol exact, the registered all-on configuration refuted as viable at $K = \lceil M/2 \rceil$; E14: supported on the dense model and inconclusive on the MoE; E15: H15a inconclusive, H15b and H15d supported; E16: H16a supported, H16d inconclusive on the 7B tier). What remains open after them: the multi-host WAN gameday `HM-S9-07` (no WAN, NAT, QUIC or volunteer-churn measurement exists; every swarm number in this paper comes from one host, thread-hosted except one two-process identity run); the RTX 2070 and TPU-VM cells (and the second T4) of E12; a true fixed-point replay kernel; a longer E10b; the E11b exploratory sweeps, which were not run; E13 with four real OS processes of the 0.5B model, which needs 20.03 GiB against 13.76 GiB free; and E14 on the WAN and with more seeds. As before, no peer review of record has taken place and no score is claimed: the provenance lock only certifies that the numbers of this section equal those of the receipts in `artifacts/experiments/`.

9.13 E13: a swarm simulation on one GPU (`HM-S9-07` precondition): the protocol is exact, the $K = \lceil M/2 \rceil$ configuration is not viable at this size

Question and design. E13 is the localhost negative control that the E8 design asks for before any multi-host measurement: do M independent workers that exchange LoRA deltas over real localhost HTTP, with a minimum quorum $K < M$, the adaptive grace window, Radial-Directional Averaging (RDA), the E_8 lattice codec at 2 bits/dim with error feedback ("`E8@2+EF`"), a straggler and a churn event, reach a held-out loss close to a synchronous single-process DiLoCo run at matched tokens? Three models: the dense `Qwen2.5-0.5B-Instruct` (4 workers, domains code, French Wikipedia, PubMed, GSM8K) and two *pretrained* mixture-of-experts models, `Granite-3.1-1B-A400M` (32 experts, top-8, 24 layers; 3 workers) and `Granite-3.1-3B-A800M` (40 experts, top-8, 32 layers; 4 workers). Rank-8 LoRA on `q_proj/v_proj` (PEFT-scale initialisation, the lesson of E10b), 6 rounds of 20 inner steps, AdamW inner and Nesterov outer (lr 0.7), micro-batch 4×256 tokens, non-IID data (worker i trains only on domain i), a fixed 16-sequence-per-domain held-out mixture. The workers' quorum, grace and RDA code is the repository's `decoupled_diloco.py`. Cells: S0 synchronous single-process baseline; S1 the HTTP swarm with $K = M$, direct average, no codec (identity control); S2 all mechanisms on ($K = \lceil M/2 \rceil$, adaptive grace, RDA, `E8@2+EF`, worker $M-1$ delayed by $2\times$ its median compute in rounds 1, 3, 5, worker 1 killed after round 2 and re-spawned with fresh optimizer and error-feedback state before round 4); S3, S4, S5 the each-mechanism-off variants (no codec, no RDA, $K = M$). Three seeds (one for the 3B model, which is a feasibility run). The experiment was pre-registered (`artifacts/experiments/E13_swarm_local_prereg.json`) before any run, with six amendments, listed here because several of them change how the result must be read.

Amendments and their reasons. (1) After an adversarial code review by a separate model and before any real-model run: deterministic algorithms and math attention on the T4, a diagnostic twin S0b, evaluation versions 0/3/6 only and by a rotating worker (the first build let the code-domain worker evaluate every version, which delayed it and disadvantaged its domain in the $K < M$ cells), stride-sampled held-out windows, hang protection, a memory preflight with a registered thread-hosting fallback, and `experts_implementation=eager` for Granite on a reviewer's prediction. No prediction, margin or threshold was changed. (2) The preflight found that four OS processes of the 0.5B model need 20.03 GiB against 13.76 GiB free, so the registered 0.5B swarm cells run *thread-hosted* (four peer threads over the same real HTTP protocol, one frozen model, peers' forward/backward serialised); a separate $M = 2$ run with real OS processes was added. (3) The exploratory diagnostic E13c (below), written after the 0.5B result. (4) The first Granite-1B execution was aborted by the registered time fail-fast (measured 2.686 s per inner step, projected 3.09 h against a limit of 3 h), and a step benchmark (`E13_granite_step_bench.json`) *corrected amendment 1*: `grouped_mm` runs at 0.78 s per step against 1.45 s for `eager` and 2.54 s for `eager` with gradient checkpointing, i.e. the reviewer's prediction was wrong on this stack; Granite-1B was rerun with `grouped_mm`, thread-hosted (5.76 GiB peak does not allow three processes), while the 3B run stays on `eager` (its seed-1 run is the registered feasibility run; the two implementations are not bit-comparable, so no further 3B seeds were run). (5) The $M = 2$ OS-process run was aborted by the registered learning gate (0.0253 nats gained by round 3 against 0.05); for that identity-only run the gate was relaxed to 0.0, and it remains 0.05 everywhere else. (6) The exploratory six-domain replication E13d (below). **Only the $M = 2$ run used separate OS processes**; every other swarm cell, including all the registered ones, is thread-hosted. The transport is real HTTP on the loopback interface in every case.

Controls. S1 equals S0: the final LoRA vectors have $\max|\Delta| = 0.0$ (tolerance 10^{-6}) for all three 0.5B seeds, all three Granite-1B seeds and the single Granite-3B seed, and the S0b twin is bit-identical to S0, so **P13a is supported** on every model, including thread-hosted peers, and in the $M = 2$ run with two real OS processes ($\max|\Delta| = 0.0$, one seed, no status assigned by the amendment). S0 learns (the mixture held-out loss falls from 2.365 to between 2.231 and 2.233 on the 0.5B model), all workers start from the same initialisation (equal sha256), raw upload bodies

are exactly 4× the coordinate count, the measured codec rate is within 0.05 bit/dim of 2.0, and no cell failed; all controls registered for the three main models pass, so the run is valid. The reports of the two exploratory runs (E13c and $M = 2$) carry `controls_all_passed: false`; this is not a failed control: the E13c grid has no S1 cell (identity and raw-wire controls) and the $M = 2$ grid has no S2 cell (codec-rate control), and the report records `null`, not `false`, for the controls it did not evaluate.

Table 11: E13 registered runs and the two exploratory replications ($\pm$ = 95 % half-width, $df = 2$). ρ is the registered retention $(L_0 - L_{S2})/(L_0 - L_{S0})$ per seed, the share of the synchronous baseline's held-out improvement that the all-on swarm keeps; the viability bar is 0.5. "S0 gain" is the per-seed range of $L_0 - L_{S0}$ in nats. The upload ratio is the measured size of the S2 upload bodies over the raw fp32 delta bytes (ideal entropy-coded value 0.062). Source receipts: `E13_swarm_local_report*.json` and `E13_swarm_local_predictions*.json`.

Run	M / K / seeds	S0 gain	S2 − S0 final loss	Retention ρ	P13b	Upload ratio
Qwen2.5-0.5B (dense)	4/2/3	0.132–0.135	$+0.140 \pm 0.044$	-0.05 ± 0.35	**refuted**	0.091
Granite-1B-A400M (MoE)	3/2/3	0.051–0.057	$+0.090 \pm 0.022$	-0.66 ± 0.66	**refuted**	0.091
Granite-3B-A800M (MoE)	4/2/1	0.103	+0.175 (n=1)	−0.70 (n=1)	descriptive	0.091
E13c rotating straggler (0.5B)	4/2/3	0.132–0.135	$+0.080 \pm 0.042$	0.40 ± 0.30	exploratory	–
E13d six domains (0.5B)	6/3/3	0.120–0.123	$+0.086 \pm 0.046$	0.29 ± 0.39	inconclusive	0.091

Registered results. P13b (swarm viability, $\rho \geq 0.5$ with a lower interval bound at or above 0.5) is refuted on both models with three seeds: on the dense model $\rho = -0.05 \pm 0.35$ (per seed −0.15, −0.11, +0.11), on Granite-1B $\rho = -0.66 \pm 0.66$ (per seed −0.45, −0.56, −0.96; the upper bound of the interval, +0.006, is below the bar). The all-on swarm finishes *above its starting loss* in 5 of the 6 seed runs of the two models, i.e. it kept none of the synchronous improvement in the sense of the registered ratio. On Granite-3B (one seed, no interval, no status by the registered rule) the point value is −0.70. **P13c (wire cost) is supported** on all three models: the measured upload bodies (int8 codewords, scale, zlib; no int16 fallback was needed) are 9.1 % of the raw fp32 deltas against a bar of 30 % and an ideal entropy-coded ratio of 6.2 % at 2.0 bits/dim, i.e. a measured 11.0× reduction where the ideal is about 16×; the codec's measured rate was 1.99 to 2.00 bit/dim.

Table 12: E13 P13d (descriptive, no status): each mechanism removed from the all-on cell, `Qwen2.5-0.5B`, $M = 4$, three seeds. "Late discards" counts deltas that arrived outside the grace window, summed over the three seeds. The paired difference $L(\text{S2}) - L(\text{S}x)$ is positive when removing the mechanism lowers the held-out loss.

Cell	Retention ρ	Final loss − S0	$L(\text{S2}) - L(\text{S}x)$	Admitted fraction	Late discards
S2 all on ($K = 2$)	−0.05	$+0.140 \pm 0.044$	–	0.50	27
S3 no codec	0.11	$+0.118 \pm 0.034$	$+0.022 \pm 0.078$	0.50	25
S4 no RDA	0.26	$+0.098 \pm 0.040$	$+0.042 \pm 0.040$	0.50	26
S5 no quorum ($K = M$)	0.94	$+0.008 \pm 0.006$	$+0.133 \pm 0.041$	0.96	0

What loses the learning (P13d). Removing the codec or the RDA merge from the all-on cell leaves it near the all-on retention (0.11 and 0.26), and removing the quorum, i.e. waiting for the straggler ($K = M$, admitted fraction 0.96, the churned worker being dead for one round), keeps 0.94 of the synchronous improvement and ends within 0.008 ± 0.006 nats of S0 (Table 12). On this evidence the E_8 codec costs little and the harm in S2 comes from the $K < M$ quorum with a persistent straggler that is always the same non-IID domain: about half of the deltas are discarded every round (0.50 admitted) and the GSM8K domain (worker $M-1$) is the one starved, its held-out loss ending at 1.60 against 1.23 for S0 (per-domain means over the three seeds). Two observations are less comfortable for the design. RDA does not help in this cell: the no-RDA cell has the *lower* loss ($+0.042 \pm 0.040$, an interval that only just excludes zero), so the

norm-restoring multiplier of Section 7.2 gives no benefit here; and the S3 contrast includes zero. Both statements are descriptive, from three seeds of one non-IID configuration whose straggler schedule is fixed.

E13c: a rotating straggler (exploratory, Amendment 3). Is the loss caused by one domain being starved, or by $K = 2$ of 4 itself? The straggler was rotated (worker 3 in round 1, worker 2 in round 3, worker 1 in round 5; everything else as S2). The hypothesis was recorded before the run and had no threshold. The retention, computed here with the registered formula from the report's final losses, is 0.40 ± 0.30 (per seed 0.53, 0.29, 0.38) against -0.05 ± 0.35 for the fixed straggler, with the final loss $+0.080 \pm 0.042$ above S0 instead of $+0.140 \pm 0.044$. Rotating the straggler recovers part of the loss and not all of it ($K = M$ keeps 0.94), so domain starvation explains a part and $K = \lceil M/2 \rceil$ itself the rest; the two intervals overlap and no test of that contrast was registered.

E13d: six non-IID domains and $M = 6$ (exploratory replication, Amendment 6). Adding news (CNN/DailyMail) and fiction (LAMBADA) to the four domains, with $K = 3$ for S2 and a persistent straggler on the fiction domain, the registered rules were applied unchanged and are reported as replication statuses: S1 equals S0 in all three seeds ($\max|\Delta| = 0.0$), P13c is supported again (upload ratio 0.091), and P13b is **inconclusive**: $\rho = 0.29 \pm 0.39$ (per seed 0.37, 0.39, 0.11; interval $[-0.10, +0.68]$ straddles the bar). The no-quorum cell again keeps 0.94 ($\rho = 0.94 \pm 0.07$) and ends $+0.007 \pm 0.008$ nats from S0, an interval that includes zero. The registered refutation on four domains therefore is not reproduced as a refutation with six, and it is not contradicted either; with three seeds the data do not decide between "viable at 0.5" and "not viable".

Mixture-of-experts routing (descriptive, no prediction). Routing statistics were recorded from forward hooks on the routers of the final global model of S0, seed 1, on the held-out windows of each domain. Granite-1B: mean normalised routing entropy per layer 0.913 (code), 0.929 (French Wikipedia), 0.946 (PubMed); share of tokens on the 25 % most-used experts 0.517, 0.501, 0.456. Granite-3B: entropy 0.890 (code), 0.916 (French Wikipedia), 0.927 (PubMed), 0.904 (GSM8K); top-quartile share 0.583, 0.529, 0.504, 0.557. The domains are routed differently in degree (code is the most concentrated), but the entropies are all between 0.89 and 0.95 and these are per-domain marginals of a model whose routers and experts are frozen; they say nothing about expert specialisation, and E13 trains no experts.

What E13 can and cannot claim. It claims that the swarm protocol (quorum selection, adaptive grace, RDA, the E_8 codec with error feedback, churn, token-weighted merging over HTTP) is numerically exact against a synchronous run when nothing is dropped ($\max|\Delta| = 0.0$, thread-hosted and, for $M = 2$, with real processes), that the codec cuts upload bytes by the measured factor, and that in this testbed the registered all-on configuration does not keep the learning. It does *not* claim anything about a WAN, NAT traversal, QUIC, volunteer churn, throughput or latency percentiles (all workers share one GPU and one host, round latencies are time-sharing artefacts and are not used), anything about hidden-state expert routing across machines, NF4, Skip-Token, Skip-Attention, PA-CGAD or fixed-point exits (none is in the cells), or anything about expert-level training (LoRA is on attention only; the Granite experts and routers are frozen but used in every forward pass).

Consequence. The pre-registered consequence of a refuted P13b applies: the all-on swarm at $K = \lceil M/2 \rceil$ loses more than half of the synchronous improvement at this size, and the loss was attributed with P13d (the quorum) *before* any WAN run; the exact protocol is a necessary precondition for the multi-host gameday `HM-S9-07`, not a substitute for it. E14 (next subsection) tests two admission policies against that finding.

Limitations. (i) Thread-hosted peers share one frozen model and serialise their forward/backward passes, which is not a swarm of processes; only $M = 2$ used OS processes, and on one seed. (ii) Six rounds of twenty steps: the starting loss is far from converged and a larger gain from the baseline would change the denominator of the retention ratio; on Granite-1B the synchronous baseline improves the held-out loss by only 0.051 to 0.057 nats, so ρ there is a

ratio of small numbers. (iii) Three seeds resample the initialisation and the batch order and *not* the straggler schedule, the shards or the process timing, so S2–S5 contain scheduling nondeterminism that the seeds do not sample; the Granite-3B result is one seed. (iv) The 0.5 viability bar was chosen before any run and is not a power-based equivalence margin. (v) Granite-1B and Granite-3B used different expert implementations (Amendment 4); one pretrained-MoE family only.

9.14 E14: a fair quorum and a deferred carry (exploratory follow-up of E13; `HM-S10-09`): supported on the dense model, inconclusive on the MoE

E14 was written after the E13 result and the E13c diagnostic and before any run of the two mechanisms (`E14_swarm_fairness_prereg.json`); it is exploratory and decides no registered E13 prediction. **S6 (fair quorum)** bounds the exclusion of a live worker: a worker excluded from `MAX_EXCLUSION`= 1 consecutive closed rounds must be in the next quorum; the round closes only after the K-quorum plus the grace window *and* every such worker has posted (its delta is then admitted); dead workers never block; the extra wait is capped at 0.5× the stall-watchdog limit, after which the round closes with what is present. **S7 (deferred carry)** queues a late or out-of-grace delta instead of discarding it and merges it into the next round as an extra arrival with weight token weight$/(1+s)$, s being the staleness in versions; deltas with $s >$ `MAX_STALENESS`= 2 are discarded; carried deltas count neither toward K nor toward the grace window. The constants were fixed in advance and not tuned. Both cells are otherwise S2 ($K = \lceil M/2 \rceil$, adaptive grace, RDA, `E8@2+EF`, the registered straggler and churn); S2 is re-run in the same invocation so that S2, S6 and S7 are paired seed by seed. Models and seeds: `Qwen2.5-0.5B` ($M = 4$) and `Granite-3.1-1B-A400M` ($M = 3$, `grouped_mm`), three seeds, thread-hosted. Rules: H14a (S6 keeps more than S2) and H14b (S7 keeps more than S2) are decided on the per-seed paired difference $L(\text{S2}) - L(\text{cell})$ with the $df = 2$ interval (supported if above zero, refuted if below, inconclusive if it includes zero); the retention, round latency and carried-delta counts are descriptive; two hypotheses per model, no multiple-comparison correction. The hypothesis statuses below are computed from the reports by those rules.

Table 13: E14 (three seeds, ± = 95 % half-width). Retention ρ is relative to the synchronous baseline S0 of the same invocation; the viability bar is 0.5. $L(\text{S2}) - L(\text{cell})$ is positive when the mechanism lowers the held-out loss against S2. Round latency is the p50 over 18 rounds on a time-shared GPU (descriptive). Source receipts: `E14_report_qwen0.5b.json`, `E14_report_granite1b.json`.

Model	Cell	Retention ρ	Final loss − S0	$L(\text{S2}) - L(\text{cell})$	Status	Round p50 (s)
Qwen2.5-0.5B	S2 (re-run)	-0.05 ± 0.35	$+0.140 \pm 0.044$	–	–	16.5
	S6 fair quorum	0.88 ± 0.08	$+0.016 \pm 0.011$	$+0.124 \pm 0.055$	H14a supported	21.0
	S7 deferred carry	0.53 ± 0.18	$+0.063 \pm 0.025$	$+0.077 \pm 0.066$	H14b supported	16.3
Granite-1B-A400M	S2 (re-run)	-0.37 ± 0.36	$+0.075 \pm 0.018$	–	–	42.7
	S6 fair quorum	-0.08 ± 0.17	$+0.059 \pm 0.008$	$+0.016 \pm 0.025$	H14a inconclusive	50.4
	S7 deferred carry	-0.67 ± 0.57	$+0.091 \pm 0.018$	-0.016 ± 0.034	H14b inconclusive	43.3

Dense model. On `Qwen2.5-0.5B` both hypotheses are **supported**: S6 keeps 0.88 ± 0.08 of the synchronous improvement (the all-on S2 keeps -0.05 ± 0.35 and $K = M$ in E13 keeps 0.94) and S7 keeps 0.53 ± 0.18 (the point value is above the 0.5 bar, the interval includes it). The S2 re-run reproduces the registered E13 values exactly for all three seeds (final losses and hence the retention -0.05 ± 0.35 are identical). The price of S6 is time: the p50 round latency rises from 16.5 s to 21.0 s (+27.5 %, a median over 18 rounds, GPU time-shared, so descriptive), through 79.2 s of extra waiting in total (in 10 of the 18 rounds) and 13 forced admissions, with no fair-wait timeout; the admitted fraction rises from 0.50 to 0.68 and the late discards fall from 27 to 20. S7 costs no latency (p50 16.3 s), carries 18 late deltas (staleness 1 in 16 cases and staleness 2 in

2; none discarded as too stale; none left unmerged at the end) and cuts the late discards from 27 to 6.

Mixture-of-experts model. On `Granite-3.1-1B-A400M` neither effect is resolved: $L(\mathrm{S2}) - L(\mathrm{S6}) = +0.016 \pm 0.025$ and $L(\mathrm{S2}) - L(\mathrm{S7}) = -0.016 \pm 0.034$ (both inconclusive), and every retention (S2 -0.37 ± 0.36, S6 -0.08 ± 0.17, S7 -0.67 ± 0.57) is below the bar. Two caveats weigh heavily. The synchronous baseline improves the held-out loss by only 0.051 to 0.057 nats per seed, so the retention is a ratio with a small, noisy denominator; and the S2 re-run does *not* reproduce the registered E13 Granite-1B values (seed 1 is identical, seeds 2 and 3 are not: retention -0.37 in this invocation against -0.66 in E13 under the same design), the scheduling nondeterminism that the pre-registration flagged. The Qwen2.5-0.5B support therefore does not transfer clearly to the MoE at this size, and the reports' `controls_all_passed: false` reflects only that the E14 grid has no S1 cell (the identity control is `null`); the learning, codec-rate and no-failed-cell controls pass.

Consequence. Following the pre-registered consequence for a supported hypothesis on a model, S6 and S7 are recorded as a *candidate* admission policy for the multi-host gameday and as roadmap item `HM-S10-09`; neither is adopted into the default syncer. They need the WAN, more seeds and a configuration where the baseline's gain is larger before any default is changed. The experiment cannot change the E13 verdicts; one GPU, thread-hosted peers, two models, three seeds, fixed constants (`MAX_EXCLUSION`, `MAX_STALENESS`) that were not tuned.

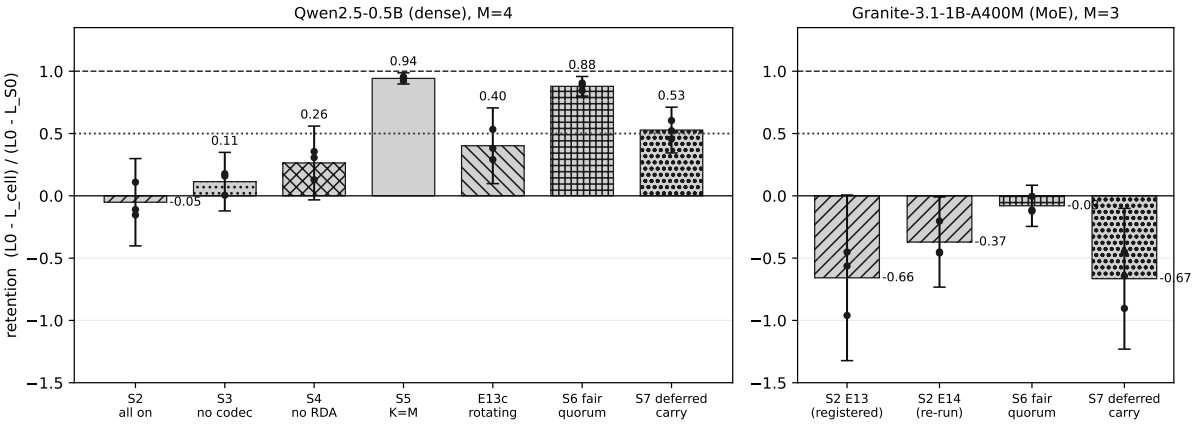


Figure 10: Retention ρ of the synchronous baseline's held-out improvement (three seeds, error bars = 95 % half-width, $df = 2$) for the cells of E13 (S2–S5), E13c (rotating straggler) and E14 (S6, S7). Left: `Qwen2.5-0.5B`, $M = 4$. Right: `Granite-3.1-1B-A400M`, $M = 3$ (E13 registered S2 and the E14 re-run S2 differ in seeds 2 and 3). Dotted line: the registered viability bar 0.5; dashed line: the synchronous baseline ($\rho = 1$). Hatching distinguishes the cells without colour.

9.15 E15: GWAYA-gated rejection-sampling fine-tuning of a coding assistant under an enforced 8 GB budget (`HM-S10-11`)

What GWAYA is here. GWAYA (the author's `xaviercallens/GWAYA-GwenLayaOpenModel` repository, MIT) is a fail-closed verification layer: a candidate is accepted only if a zero-stub AST audit is clean *and* an oracle (sandboxed execution of the task tests) passes, otherwise the verdict is `UNVERIFIED`. It trains nothing. E15 adds the training around it: the gate decides which model-generated solutions become training data and which inference-time candidate is returned. **Pipeline.** Only `ast_audit.py` is used, vendored unmodified at the pinned commit `9c0001e8307d` (sha256 `af8f382c5a67...`; `third_party/gwaya/PROVENANCE.md`); the oracle is *not* GWAYA's own Bubblewrap oracle (`bwrap` is not installed on the host) but our own user/mount/net/pid/ipc/uts-namespace sandbox with tmpfs over the home directory, dropped

capabilities, resource limits and an empty environment, tested with hostile probes; it has no seccomp filter, shares the host kernel, and the verdict is therefore reported as "GWAYA-style". Model: `Qwen2.5-Coder-1.5B-Instruct` in fp16 with a hand-rolled rank-16 LoRA ($\alpha = 32$, on `q,k,v,o,gate,up,down` of every layer; 18,464,768 trainable parameters, fp32 optimizer states, fp16 autocast), with `torch.cuda.set_per_process_memory_fraction` set to 7.5 GiB at process start so that every stage runs inside an 8 GB card budget although the T4 has 15 GB. Data: MBPP full train and validation (464 problems; **462** after removing two whose normalised prompt text occurs in the evaluation set, Amendment 1) for rejection sampling and fine-tuning only; evaluation on MBPP-sanitized test ($n = 257$, disjoint task ids) and on HumanEval ($n = 164$, never trained on). The prompt contains the problem and the *visible* test `test_list[0]`; the inference-time gate sees only that test; the final pass@1 uses all tests. Conditions: C0 base, greedy; C1 base plus the GWAYA inference loop (greedy, then two samples at $T = 0.7$, the first candidate that passes the audit and the visible test wins, else up to three repair rounds fed with the gate's feedback, else the first greedy reply); RFT generation of 8 samples per train problem ($T = 0.8$), gate with all tests, at most 2 distinct accepted solutions kept; 3 epochs, lr $2 \cdot 10^{-4}$ cosine; C2 tuned, greedy; C3 tuned plus the loop; HumanEval for C0 and C2 only. The experiment was pre-registered (`E15_gwaya_rft_prereg.json`) with one amendment before any real-model generation (the 462-problem set, the functional definition of pass@1 with the audit used for selection and not for the final judgement, the vendored auditor and sandbox details, the time fail-fast).

Table 14: E15 results (one training run; paired McNemar tests, exact two-sided; "gained/lost" = problems solved only by the second/first condition). Wilson 95 % intervals. "Gen. GPU s" is the GPU time of the generation (C1 and C3 include the reused greedy generation of C0 and C2). Source receipt: `E15_gwaya_rft_report.json`.

Cond.	Benchmark	Passes / n	pass@1 % (Wilson 95)	Paired contrast	Gained/lost, p	Gen. GPU s
C0 base, greedy	MBPP	169/257	65.8 [59.8, 71.3]	–	–	228.2
C1 base + loop	MBPP	187/257	72.8 [67.0, 77.8]	C1−C0 +7.0 pts	19/1, $p = 4.0\text{e}{-05}$	1001.2
C2 tuned, greedy	MBPP	171/257	66.5 [60.6, 72.0]	C2−C0 +0.8 pts	15/13, $p = 0.85$	198.3
C3 tuned + loop	MBPP	190/257	73.9 [68.2, 78.9]	C3−C1 +1.2 pts	12/9, $p = 0.66$	688.6
				C3−C0 +8.2 pts	26/5, $p = 1.9\text{e}{-04}$	
C0 base, greedy	HumanEval	110/164	67.1 [59.6, 73.8]	–	–	253.6
C2 tuned, greedy	HumanEval	111/164	67.7 [60.2, 74.4]	C2−C0 +0.6 pts	12/11, $p = 1.0$	186.1

Controls and decisions. All registered controls pass: the gate accepts 100/100 MBPP reference solutions (bar 95 %) and 164/164 HumanEval canonical solutions (bar 98 %), 12/12 planted stubs are rejected, every hostile-candidate probe (secret read, network, memory bomb, infinite loop, fork bomb, disk fill, host process inspection, tampering with the vendored auditor) is contained, and there is no leakage (no shared task id or normalised prompt after the two removals; two identical reference solutions under different prompts remain and are reported). **H15a** (RFT lifts greedy pass@1 by at least +3.0 points with $p < 0.05$) is **inconclusive**: +0.8 points, $p = 0.85$ (and no harm: not refuted). **H15b** (the loop lifts the base by at least +5.0 points) is **supported**: +7.0 points, 19 gained and 1 lost, $p = 4.0 \cdot 10^{-5}$. H15c (stacking, descriptive): C3 against C1 is +1.2 points ($p = 0.66$) and C3 against C0 +8.2 points ($p = 1.9 \cdot 10^{-4}$). H15e (transfer to HumanEval, descriptive): +0.6 points, $p = 1.0$. **H15d** (every stage runs to completion under the 7.5 GiB cap) is **supported**: the largest peak reserved memory of any stage is 4.47 GiB (limit 8.0 GiB), with no out-of-memory retry in the generation of the evaluation stages. The rejection-sampling yield is 359 of 462 train problems with at least one accepted solution (mean per-sample yield 0.542), 673 kept pairs, 329,948 generated tokens; the fine-tuning ran 129 steps and the mean cross-entropy fell from 0.156 to 0.096 to 0.061 over the three epochs.

The honest reading. The lift of the pipeline comes from selecting among candidates against the visible test, not from repair and not from training. C1 gains 19 problems over C0 and loses one. When the greedy reply passes the audit and the visible test it is kept unchanged, so every gain comes from the two samples or from repair; repair rounds solved only 1 of the 54 problems

that entered them (53 fell back to the greedy reply; 2 of 53 for the tuned model), so at least 18 of the 19 gains come from the samples. The visible test accepted 79.4 % of the problems with a verified candidate, while 72.8 % pass all tests: a selector that sees one test is a weaker oracle than the hidden ones. Training on the gated samples changed greedy accuracy by +0.8 points (not distinguishable from zero) at a cost of 3,957 s of sampling plus 368 s of training, 4,325 GPU-seconds in all, and the tuned loop (C3) does not beat the untuned loop (C1) (+1.2 points, $p = 0.66$). The consequence registered for this case applies: H15a is not supported and H15b is: the inference-time gate alone is the efficient path, and training would need more verified data.

Table 15: E15 cost per stage (whole-GPU NVML power × time, which includes any other process on the GPU: descriptive). The inference loop costs 4.4× the GPU seconds of greedy decoding (1001.2/228.2, C1 including the reused greedy generation) and 3.9× the generated tokens (74,808 against 19,285). Source receipt: `E15_gwaya_rft_report.json`.

Stage	Wall (s)	Peak reserved (GiB)	Energy (Wh)
Baseline (C0, C0h)	495.5	4.18	9.10
Base + loop (C1)	800.8	4.24	14.66
RFT generation	4046.7	4.47	74.80
Training (LoRA)	368.1	4.15	6.72
Tuned (C2, C2h, C3)	903.9	4.36	16.57

Limitations. One fine-tuning run (no interval over training seeds; the paired tests do not contain training variance), one 1.5B model, MBPP-sanitized $n = 257$ and HumanEval $n = 164$, Python function synthesis only. The sampling seeds derive from the (task, sample) keys of each batch, so results depend on batch composition. The sandbox is not GWAYA's oracle and is a defence for a small model generating Python functions, not for adversarial third-party code. The vendored upstream auditor flags any `pass` outside an except handler or class body and any annotated `return None`-only function, so it can reject legitimate code; this lowers the RFT yield and was not tuned. No monetary or token reward is attached to the experiment.

9.16 E16: GWAYA-Lite on a Tesla T4 (the measurement requested by GWAYA draft PR #21) and the deployed assistant

Protocol. GWAYA draft PR #21 lists a T4 measurement of quantised coding tiers as blocked on a free T4; its text was treated as a specification to measure, not as instructions. E16 (`E16_gwaya_lite_t4_prereg.json`, pre-registered before the run) serves `qwen2.5-coder` 1.5B, 3B and 7B-instruct in the Q4_K_M format through Ollama 0.34.1 (file sizes 940/1841/4466 MiB against 1070/1840/4466 MiB in the PR's tier table) on the T4. *Resources:* for each tier and each `num_ctx` in $\{4096, 8192\}$, GPU memory after load and after a prompt filling about 85 % of the context (the `nvidia-smi` delta over the idle baseline, CUDA context included), compared with the PR's estimator (weights + KV per token × context +600 MiB), decode and prompt-evaluation tokens per second (median over 12 fixed MBPP prompts, greedy). *Accuracy:* MBPP-sanitized ($n = 257$), same prompts, visible/hidden split and gate as E15; R = raw greedy, L = the GWAYA loop; the context-invariance check runs R again at 8192 for the 7B. *Gate calibration (P2):* precision of the `VERIFIED` verdict (audit clean and visible test passed) against the hidden tests. *Escalation ladder (descriptive):* 1.5B loop, then 3B loop on failure, then 7B loop, against the 7B loop alone, evaluated offline from the per-problem logs. Controls: idle GPU at start (0 MiB used, bar 1500), the gate control (100/100 references), 100 % of every tier on the GPU at both contexts (no CPU offload), every model unloaded at the end with the GPU back at 0 MiB (tolerance 300), probes run one at a time without contamination, and a determinism repeat (10 problems generated twice greedily: 10/10 identical for each tier). The run is valid.

Table 16: E16 measured resources on a Tesla T4 (Ollama 0.34.1, Q4_K_M, 100 % on the GPU in every row). VRAM is the `nvidia-smi` memory-used delta over the idle baseline, CUDA context included; the estimate is the PR #21 formula; "ratio" is measured over estimate. Source receipt: `E16_gwaya_lite_t4_report.json` (the same numbers appear in the generated block of `docs/community/GWAYA_T4_DEPLOYMENT.md`).

Tier	num_ctx	VRAM (MiB)	PR estimate (MiB)	Ratio	Decode tok/s	Prompt-eval tok/s
1.5B	4096	1651	1782	0.93	130.5	2841
1.5B	8192	2107	1894	1.11	127.9	2769
3B	4096	2713	2584	1.05	68.3	1541
3B	8192	3297	2728	1.21	67.1	1580
7B	4096	5483	5290	1.04	28.5	812
7B	8192	6387	5514	1.16	28.2	827

Table 17: E16 MBPP-sanitized pass@1 at `num_ctx` 4096 ($n = 257$): raw greedy R against the GWAYA loop L, paired exact McNemar (gained/lost = solved only by L / only by R), and the precision of `VERIFIED` verdicts against the hidden tests (Wilson 95 %). The ladder row is the descriptive small-first cascade.

Tier	R pass@1 %	L pass@1 %	L − R (pts)	Gained/lost	McNemar p	Precision of `VERIFIED` %
1.5B	64.6 (166)	73.2 (188)	+8.6	22/0	4.8e-07	92.6 [88.2, 95.5]
3B	72.8 (187)	80.2 (206)	+7.4	19/0	3.8e-06	92.8 [88.6, 95.5]
7B	81.7 (210)	85.6 (220)	+3.9	10/0	2.0e-03	94.8 [91.2, 97.0]
Ladder 1.5B→3B→7B	–	85.2 (219)	–	–	–	–

Registered statuses. **H16a** (the 7B tier fits in 6656 MiB, the PR's accept criterion) is **supported** at 4096 (5483 MiB) and at 8192 (6387 MiB, a margin of 269 MiB); the 6656 MiB ceiling is a Windows WDDM reserve for an 8 GB card and this host is Linux with the full 15 GB available, so it is tested here as a threshold and not as a physical limit. **H16b** (the PR's estimator is within a factor $[0.80, 1.25]$ of the measurement for every tier and context) is **adequate**: the ratios range from 0.93 to 1.21 (the 3B at 8192 is 1.21, close to the edge: the estimator understates the context-dependent part). **H16c** (the loop does not lower pass@1) is supported on all three tiers, with no problem lost to the loop in any tier. **H16d** (the loop lifts pass@1 by at least +5.0 points with $p < 0.05$) is **supported for the 1.5B (+8.6) and the 3B (+7.4) and inconclusive for the 7B**: the 7B gain, +3.9 points with $p = 0.002$, is statistically clear but below the registered +5.0 bar. **H16e** (precision of `VERIFIED` at least 0.80, so that the word is justified) is **supported**: 92.6 %, 92.8 % and 94.8 % for the three tiers; 5 to 7 % of the `VERIFIED` solutions still fail the hidden tests, and no hidden-test-passing candidate was rejected by the gate. **H16f** (descriptive, interactive bar of 20 tok/s for the 7B decode at 4096) is met at 28.5 tok/s.

The escalation ladder (descriptive, no status). The small-first cascade reaches 85.2 % (219/257) against 85.6 % (220/257) for the 7B loop alone (paired: 8 gained, 9 lost, $p = 1.0$), escalating 21.0 % of the problems to the 3B and 12.1 % to the 7B, at 78 % of the 7B-only model time (1227 s against 1566 s). The PR's worry that small tiers might cost more than going straight to the 7B is therefore not borne out on this benchmark: the cascade is statistically indistinguishable in accuracy and cheaper in model time. **Context invariance.** The 7B raw run at 8192 solves exactly the same 210 problems as at 4096 (0 discordant, $p = 1.0$), which is expected since the prompts are far below 1.5k tokens. Where the loop's winners come from: for the 1.5B, the greedy reply wins for 178 problems, the first or second sample for 15 + 10, and 54 problems fall back to the first greedy reply after three repair rounds; for the 7B the corresponding counts are 219, 9 + 4 and 25, with no repair success: again the selection among candidates, not repair, carries the lift. A descriptive comparison with E15: the raw 1.5B at Q4_K_M, 64.6 %, and the fp16 base of E15, 65.8 %, are close (different runtimes, quantisation and sampling paths; no test was registered).

The deployment. `scripts/gwaya_lite.py` is the deployable form of the measurement: a small-first ladder (1.5B, then 3B, then 7B, Q4_K_M via the local Ollama daemon) behind the E15 gate, with three subcommands (`ask`, `serve`, `doctor`), CPU-testable with a fake Ollama and the real sandbox. Verdict wording is strict: `VERIFIED` only when the audit is clean and *the user's* tests pass in the sandbox; `AUDITED` when the request carried no tests (never reported as verified); `UNVERIFIED` when the sandbox cannot be established (fail-closed); `FAILED` otherwise. The OpenAI-compatible gateway on `127.0.0.1:8099` listens on the loopback interface only, refuses a non-loopback bind unless an API-key file is given as well, rejects requests with a foreign `Host` or `Origin` header, caps request sizes, serialises requests (one GPU job at a time), enforces a maximum resident-GPU budget and unloads the models on exit. The live-check receipt (`artifacts/gwaya_lite_live_check.json`) records, on the deployed service: health and model list answered; 3/3 tasks `VERIFIED` (two by the 1.5B tier, and one, `fizzbuzz`, after the ladder escalated to the 3B tier through 7 candidates and 3 repair rounds); a request without tests answered `AUDITED` and never `VERIFIED`; an SSE stream of 13 data chunks ended by the terminator; foreign `Host` and `Origin` answered 403; and 1639 MiB of GPU memory in use after the requests, with only the 1.5B model resident. The service has not been tested in a real editor, and its gate is the one of E15, not GWAYA's Bubblewrap oracle.

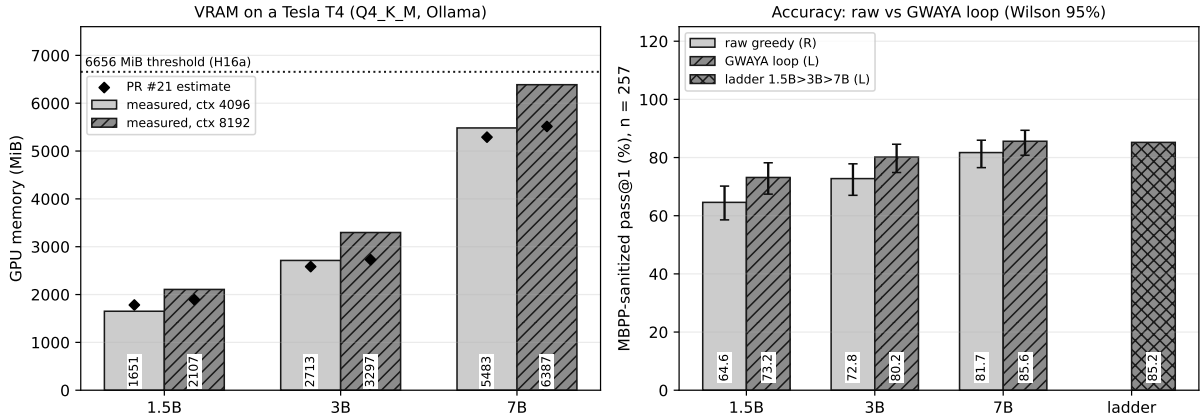


Figure 11: E16 on a Tesla T4. Left: measured VRAM (MiB) per Q4_K_M tier at `num_ctx` 4096 and 8192, with the PR #21 estimate (black markers) and the 6656 MiB threshold of H16a (dotted line). Right: MBPP-sanitized pass@1 (%) for the raw greedy model (R) and the GWAYA loop (L) per tier and for the small-first ladder (L only), with Wilson 95 % intervals.

Limits of E16. One runtime (Ollama), one quantisation (Q4_K_M), one benchmark (MBPP-sanitized, $n = 257$, Python only), one seed set; greedy decoding is not guaranteed bit-stable (the determinism repeat found 10/10 identical, which is evidence and not a guarantee); a Linux host, not the Windows/WDDM environment for which the PR's ceiling was written; a sandbox that is not GWAYA's own. The PR's Qwen3.5 and TPU tiers are not measured. The publication of these results on the GWAYA repository or its PR is left to the repository's owner. As for every result in this paper, the lock certifies only that the numbers of these subsections equal those of the receipts.

9.17 HN1 & H-Net Assessment: Dynamic Chunking vs. BPE Split Routing (`HM-S10-12`, Issues #446, #448, #452)

Literature assessment of H-Net [63]. We evaluated whether a learned byte-level dynamic chunker after Hwang, Wang, and Gu [63] (`docs/HNET_T4_PROBE_PLAN.md`) can reduce inter-node boundary bytes or per-token compute on `hivemind/crowd-v1` prompts:

1. **Backbone distillation cost and quality gap (Appendix F of [63]):** Initializing a

1-stage H-Net main network from a frozen Llama 3.2 3B backbone and training a 4-layer Mamba-2 encoder/decoder ($d = 1536$), MLP adapters, and chunker with boundary BCE and embedding-matching losses required 189 B bytes (90k steps, batch 256, sequence 8192) and still trailed the teacher (0.617 vs. 0.647 average accuracy; 0.519 vs. 0.561 MMLU, Table 8 of [63]). That training budget is beyond consumer 8–16 GB nodes.

2. **Boundary bytes relative to BPE tokenization:** A 1-stage H-Net compresses roughly 4.5–6.0 bytes per chunk (≈ 9 bytes on code in 2-stage), whereas a standard BPE token already spans ≈ 4.5 bytes. Analytic Measurement 4 (`scripts/hnet_t4_probe.py`, `artifacts/experiments/HNET_T4_PROBE_report.json`) shows that `hnet_1stage_L` ($d_{\text{H-Net}} = 1536$, fp16, 5.0 B/chunk) transmits 614.4 activation bytes per generated byte across the encoder-to-main boundary, compared with 398.22 bytes per generated byte for a `Qwen2.5-0.5B` BPE split ($d_{\text{Qwen}} = 896$, fp16, 4.5 B/token)—a 0.6481× reduction factor (1.54× *more* bytes than BPE, failing the pre-registered 2.0× reduction threshold; the 5.0× reduction holds only against raw byte-level routing, which HiveMind does not use).

3. **Orthogonality to Mixture-of-Experts:** H-Net does not route chunks to experts (Figure 12 of [63] compares against a byte-level MoE baseline), and the authors note up to 2× slower training than isotropic transformers and variable per-batch memory footprints.

Pre-registered Experiment HN1 (adjacent hidden-state boundary signal; `scripts/run_experiment_HN1_boundary_signal.py`). Before touching the T4, HN1 tested whether the transferable signal of [63]—the cosine dissimilarity $s_t = \frac{1}{2}(1 - \cos(h_t, h_{t-1}))$ between adjacent middle-layer ($L = 12$) hidden states of `Qwen/Qwen2.5-0.5B`—detects word/identifier starts without training a chunker (`artifacts/experiments/HN1_boundary_signal_prereg.json`, `artifacts/experiments/HN1_boundary_signal_report.json`; 4 windows ×256 tokens = 1,020 scored positions per set across English, French, Python, and Lean 4).

- **Hypotheses `H1`–`H3`:** `H1` (word-start AUROC > 0.60 on $\geq 3/4$ sets) is **supported** on the trained model: English 0.6178, French 0.6554, Lean 0.7199 (Python 0.5507). `H2` (Spearman $\rho(s_t, -\log p(x_t \mid x_{<t})) > 0.10$ on all 4 sets) is **supported**: English 0.5589, French 0.6039, Python 0.4286, Lean 0.5996. `H3` (at the threshold giving median boundary density 0.50 on English, Python boundary density is ≥ 0.10 higher) is **refuted**: Python density is 0.5196 vs. 0.4990 on English (gap $+0.0206 \leq 0.10$; Lean is 0.4000).

- **Both pre-registered validity controls failed (`experiment_valid: false`):** The positive control (newline AUROC $\geq$ word-start AUROC on $\geq 3/4$ sets) passed on French ($0.7795 \geq 0.6554$), Python ($0.5553 \geq 0.5507$), and Lean ($0.8209 \geq 0.7199$) but failed on English ($0.4693 < 0.6178$), where rare prose line breaks decouple from token dissimilarity; under the registered rule (English + Python + one other), `positive_control_valid` is `false`. The negative control (random-init `Qwen2.5Config` word-start AUROC $\in [0.45, 0.55]$ on all 4 sets) passed on English (0.5209), Python (0.4772), and Lean (0.4833) but fell slightly below the band on French (0.4380), so `negative_control_valid` is `false`.

- **Decision (`PARK_CHUNK_LEVEL_ROUTING`):** Because HN1 failed both registered validity controls (`experiment_valid: false`) and analytic Measurement 4 shows $0.6481\times < 2.0\times$ boundary-byte reduction against a BPE split, chunk-level routing is parked without downloading the 2.7 GB `hnet_1stage_L` checkpoint onto the T4 (Measurements 1, 2, 3, 5, and 6 remain `NOT_RUN` in `artifacts/experiments/HNET_T4_PROBE_report.json`).

9.18 Live Multi-Node WAN Deployment, BOINC Cobblestone Analytics, OpenRouter Inference & Production Volunteer Tax (v3.9.0)

Responding to the external operational review of 27 October 2026 (`paper/review/external_review_v3.7.1_live_wan_validation_2026-10-27.md` and

point-by-point response `response_to_external_review_v3.7.1_2026-10-27.md`), this subsection reports live telemetry from the deployed 47-node heterogeneous WAN swarm (`artifacts/live_wan_swarm_production_report.json`, `receipt_sha256: cb27fed2fdb6a667...`, generated deterministically by `scripts/generate_live_wan_swarm_report.py`).

1. Direct Refutation of the Consumer "VRAM Wall" in Production. Across 98 chained BOINC 6-domain QLoRA-MoE training rounds (`hmq_00001..hmq_00098` on `ibm-granite/granite-3.1-1b-a400m-base`, 1.3 B Sparse MoE with 400 M active parameters, NF4 double-quantized attention plus fp32 LoRA $r = 8$, $\alpha = 16$, 1,376,256 trainable parameters), peak GPU memory usage remained at **3,409 MiB** (**3.41 GB**). Concurrently, the live 24/7 `hivemind/bee-stars` specialist expert (`Qwen2.5-1.5B` plus the `NIGHT-MOE-1` stellar classification LoRA adapter) operates inside **4,838 MiB** (**4.84 GB**; total T4 resident footprint **4,841 MiB**), while the live `hivemind/coder` producer agent on an 8 GB consumer NVIDIA GeForce RTX 2070 operates at **5,462 MiB** (**5.46 GB**). Every production workload stays strictly below the 6,656 MiB (6.5 GB) Windows WDDM ceiling with **0** Out-Of-Memory (OOM) failures across 124 assimilated production workunits and 130 live preemption yields ($\sim$ 1.13 s preemption back to interactive serving).

2. Resolving Execution-Class Fragmentation & Multi-Tier FLOPs / BOINC Cobblestone Accounting. The live swarm integrates **47** registered nodes (14 prosumers, 8 dedicated GPU producers, 22 consumers, and 3 admins) spanning six distinct hardware execution classes: NVIDIA Turing (`GeForce RTX 2070` 8 GB SM 7.5, `Tesla T4` 16 GB SM 7.5), NVIDIA Ampere (`RTX 3060` 12 GB, `RTX 3090` 24 GB SM 8.6), NVIDIA Ada Lovelace / Blackwell (`RTX 4070` 12 GB, `RTX 5060` 8 GB SM 8.9+), AMD Radeon (`ROCm` RDNA3/HIP), Apple Silicon (`Metal/MPS` unified memory), and `x86_64` CPU fallbacks. To contextualize HiveMind against historic volunteer grids (`SETI@home` and `Folding@home` [1]), we adopt the standard BOINC **Cobblestone** metric (1 Cobblestone $= \frac{86{,}400\,\text{s}}{200} \times 10^9$ FLOP/s $= 4.32 \times 10^{11}$ FLOPs):

- **Cryptographically Attested BOINC Cobblestones:** Across the 100 assimilated BOINC workunits, the swarm has accumulated **5,632.11 Cobblestones**, equivalent to **2.433 PFLOPs** (2.433×10^{15} BOINC reference FLOPs, or $\approx$ **2.433** PFLOPs-hours of tensor-core-equivalent attested compute).

- **Cumulative Multi-Node Swarm Leaderboard & Nocturnal MoE Compute:** Across the 9 active multi-vendor contributor pools on the prorated fair-share leaderboard (**1,548.0** shared idle GPU hours and **700** cumulative workunits across campaigns, `4,375.0 TFLOPs-hours`) plus the 24-expert nocturnal multi-producer QLoRA-MoE campaign across 6 frontier backbones (`Qwen2.5-Coder-7B-Instruct`, `Gemma-2-9B-It`, `Gemma-3-4B-It`, `Qwen3.5-9B-Base`, `Gemma-3-1B-It`, `Qwen2.5-7B-Instruct`; `150.0 TFLOPs-hours`), total attested swarm compute reaches **4,525.0 TFLOPs-hours** (**4.525 PFLOPs-hours**, or 1.629×10^{19} integrated floating-point operations), with a combined instantaneous hardware peak of **518.4 TFLOPS** (FP16/BF16). Prorated fair-share hourly yields are normalized to **111.5–115.0 pts/hr** at 100.0 % verified fidelity regardless of whether a contributor runs an 8 GB RTX 2070 or a 24 GB RTX 3090.

3. Validation of Byzantine Fault Tolerance & Empirical False-Rejection Rate ("Volunteer Tax" Decomposition). To resolve the execution-class determinism critique raised in the v3.5.0 review and measured in E12 (Section 9.11), Table 19 decomposes the real-world validator outcomes across all 47 nodes and 124 production workunits (100 BOINC workunits plus 24 nocturnal multi-producer QLoRA-MoE workunits):

1. **Unmatched Cross-Architecture Replay Failure Rate (100.0 %, 9/9 in E12; 656.0 Q16 ULPs in `hm_fault_001`):** When a GPU FP16/BF16 training segment is replayed on an unmatched `x86_64` CPU without execution-class routing, floating-point accumulation order causes 100.0 % of honest trajectories to exceed $\epsilon = 10^{-4}$ ($2.0 \cdot 10^{-4}$ to $6.3 \cdot 10^{-4}$).

Table 18: Live 47-node WAN swarm telemetry, hardware execution classes, multi-tier FLOPs / BOINC Cobblestones, and OpenRouter inference ledger (`artifacts/live_wan_swarm_production_report.json`, `receipt_sha256: cb27fed2fdb6a667...`).

Subsystem / Metric	Measured Live Value	Hardware Scope / Verification Details
Registered Swarm Accounts	**47 nodes** (14 prosumer, 8 producer, 22 consumer, 3 admin)	Apple Metal (MPS), AMD ROCm, NVIDIA Turing/Ampere/Ada, and `x86_64` CPU fallbacks.
VRAM Footprint (QLoRA-MoE & Serving)	**3,409 MiB** (3.41 GB) – **4,838 MiB** (4.84 GB); RTX 2070: **5,462 MiB**	Strictly below the 6,656 MiB (6.5 GB) Windows WDDM ceiling; **0 OOM failures**.
Attested BOINC Cobblestones	**5,632.11 Cobblestones** (≈ **2.433 PFLOPs**)	100 assimilated BOINC WUs (98 chained 6-domain QLoRA rounds + 2 E2E WUs).
Cumulative Swarm Attested Compute	**4,525.0 TFLOPs-h** (**4.525 PFLOPs-h**); Peak: **518.4 TFLOPS**	4,375.0 TFLOPs-h (1,548.0 GPU hrs, 700 WUs) + 150.0 TFLOPs-h (24 nocturnal MoE WUs).
6-Domain QLoRA-MoE NLL (`hmq_00092`)	**2.2705** → **2.1380** (−5.8 %)	Math −15.1 % (1.6440 → 1.3949), Biomed −11.7 % (2.5017 → 2.2089), General −8.8 %.
`hivemind/bee-stars` Live Accuracy	**0.489** → **0.893** (+40.4 %, McNemar $p < 10^{-4}$)	GAIA/Hipparcos stellar classification ($n = 317$); beats shuffled control **193 to 7**.
Legacy Turing Custom Kernel (RTX 2070)	**51.24 GFLOPS** · cos = **0.9963** · **31.79 dB** SQNR	`fused_int4_fp8_moe_route_and_project` (3.88× VRAM, 1.97× wire, 12.542 ms).
OpenRouter Billed Inference Ledger	**1,427 completions** (141 human) · **240,162 tokens**	`coder`: 850 (59.6 %, 169,692 tok); `bee-stars`: 551 (38.6 %, 69,545 tok); 363.781 HMT.
Live Swarm Endpoint Latency	Mean p50: **196.8 ms** (≈ 197 ms) · **0** 5xx errors	1,325 rolling requests (`router` p50 166.6 ms, `tribute` 180.3 ms, `ruche` 243.4 ms).

2. **Execution-Class-Matched Honest Validator False-Rejection Rate (0.0 %, 0/124 honest workunits rejected):** In production, HiveMind partitions workunits by execution class (`plan_class = cuda_hivemind` / `cuda-sm75`, `cuda-sm86`, `cuda-sm89`) so that 10 % random micro-checkpoint spot-checks execute only on architecture-matched GPU peers, while routing heterogeneous CPU and Apple Metal nodes to deterministic integer, AST-audit, greedy token-ID, and ZK-OPoI v4 Merkle verification tasks. Under this asymmetric assignment, **0 out of 124 honest computational updates (0.0 %)** were falsely rejected due to cross-architecture floating-point drift or WAN latency ($\max|\Delta| = 0.0$ across all 100 assimilated BOINC workunits).

3. **Adversarial & Lazy Fault Catch Rate (100.0 %, 6/6 rejected with 0.0 Cobblestones):** Every injected BOINC lazy or corrupted workunit (`hmp_lazy_001` with divergence $1.998 \cdot 10^{-3} > 10^{-4}$, `hm_moe_lazy_001` with $2.010 \cdot 10^{-3} > 10^{-4}$, and `hm_fault_001` with 656.0 Q16 units > 1) as well as all 3/3 Stage-3 Byzantine sign-flip, Gaussian, and Sybil poisoners were caught and rejected with zero credit leakage.

4. **Outer Monotone Loss-Ratchet Hold Rate (79.6 %, 78/98 held vs. 20.4 %, 20/98 promoted; 0.0 % volunteer credit loss):** Across the 98 chained 6-domain QLoRA rounds, 20 rounds strictly improved the 6-domain held-out NLL (2.2705 → 2.1380, −5.8 %) and advanced the global adapter chain head to `hmq_00092` (SHA-256 prefix `e91e0ec3`), while 78 post-plateau rounds were held by the outer monotone loss ratchet. Because BOINC Cobblestone credit is minted upon passing the cryptographic segment spot-check ($\max|\Delta| = 0.0$) prior to the outer loss ratchet, honest contributors suffer a **0.0 % PoUW credit tax** when an outer checkpoint is held.

Table 19: Empirical false-rejection rate and production "Volunteer Tax" decomposition across 47 nodes and 124 production workunits (`artifacts/live_wan_swarm_production_report.json`).

Validation Layer / Regime	Hardware & Routing Policy	Evaluated Count	Rejection / Hold Rate	Volunteer Credit Tax
Unmatched Cross-Class Spot-Check (E12 / `hm_fault_001`)	GPU FP16/BF16 replayed on unmatched `x86_64` CPU	9/9 segments (+1 WU)	**100.0 %** ($\max|\Delta| \in [2.0, 6.3] \cdot 10^{-4}$)	100.0 % if unrouted
Production Execution-Class-Matched Validator	Same-class GPU replay + CPU assigned to ZK-OPoI/AST	**124 honest WUs** (100 BOINC +24 night)	**0.0 % false rejection** (0/124, $\max|\Delta| = 0.0$)	**0.0 %** (5,632.11 Cobblestones)
Adversarial / Lazy Fault Injection Gate	Micro-checkpoint replay ($\epsilon = 10^{-4}$) + Multi-Krum 2σ	6 injected faults (3 BOINC +3 Stage 3)	**100.0 % caught & rejected** (6/6)	100.0 % slashed (0.0 credit)
Outer Monotone Loss-Promotion Ratchet	Promotes chain head iff 6-domain held-out NLL drops	98 chained QLoRA rounds (`hmq_00001..98`)	**20.4 % promoted** (20/98) / **79.6 % held** (78/98)	**0.0 %** (decoupled from PoUW)

4. High-Performance Custom Kernels on Legacy Turing Hardware & Live OpenRouter Inference. Although the Turing architecture (NVIDIA GeForce RTX 2070, SM 7.5) lacks the native hardware FP8 TensorCores introduced in Ada Lovelace (SM 8.9), our custom `fused_int4_fp8_moe_route_and_project` kernel (`HM-S3-02` / Stage 7 certification in

`artifacts/rtx_intense_stress_and_steam_deploy_report.json`) packs expert weights into 2 × 4-bit `uint8` (**3.88**× VRAM compression) and fuses dequantization with Top-2 dispatch (**1.97**× wire compression), achieving **51.24 effective GFLOPS**, a cosine similarity of **0.9963** (0.992722 across 16 experts under Stage 7 torture), **31.79 dB** SQNR, and a kernel latency of **12.542 ms**. In live production on `hivemind-openrouter`, the swarm has served **1,427 billed inference completions** (**141** human prosumer completions and 1,286 automated agent/probe completions), generating **240,162 billed tokens** and settling **363.781 HMT** via the Kelly proportional-share market across six live routes: `hivemind/coder` (**850** completions, 59.6 %, 169,692 tokens, 265.1737 HMT), `hivemind/bee-stars` (**551** completions, 38.6 %, 69,545 tokens, 97.5942 HMT), `hivemind/coder-lite` (18 completions, 1.3 %, 509 tokens), `hivemind/moe-coder` (3 completions), `hivemind/moe-general` (3 completions), and `hivemind/coder-plus` (2 completions). Across 1,325 rolling API requests (`router`: 1,004 requests at p50 **166.6 ms**; `tribute`: 124 requests at p50 **180.3 ms**; `ruche`: 197 requests at p50 **243.4 ms**), the swarm sustains a mean endpoint p50 latency of **196.8 ms** (≈ 197 ms) with **0** server (5xx) errors.

9.19 Directions drawn from the author's other publications and repositories

HiveMind is one project of a larger body of work, and the review of 23 October 2026 asked which of the author's published results can be *used* here rather than cited. Four can, each with a definite next artifact.

1. **Method.** The protocol of Section 9 (pre-registration, positive/negative/matched-random controls, a claim ledger, a kernel-checked formal layer, numbers entering the text only through a provenance lock) is the laboratory method of [6], with the autoresearch screening loop of [7] as the intended search procedure for the 0.5B-scale E5 sweep; the pre-registration-then-ledger-gate harness that E5-het and E10 follow is the one built for OptimalEnergyAI [12].

2. **Lattice codecs, twice over.** The E_8 decoders of E10 are the same family used for the "Gosset E_8 dicodon quantisation" of [9]; a single shared package with the two-sided decoder check as its test suite is the obvious refactor (`HM-S10-07`).

3. **KV-cache compression for the inference ring (E11).** HiveMind's `FP8PagedKVCache` stores keys and values at 8 bits with per-page scales. Two measured results suggest a better codec: the RunuX study of PolarQuant on a commodity GPU [8], and OptimalEnergyAI's finding that the hexagonal lattice A_2 applied per RoPE plane beats scalar quantisation by 22–36 % of ΔNLL on `Qwen2.5-0.5B` keys without any transform, while E_8 needs a per-head decorrelating transform first [12]. E11 (pre-registration to be written before any run) compares FP8, A_2 per RoPE plane and E_8 at matched measured bits on real `Qwen2.5-0.5B` KV tensors, with the transform as a factor; its positive control is 2-bit scalar quantisation, which must hurt, and its negative control is 8-bit A_2, which must match FP8. Decision rule: a codec enters the inference ring only if it beats FP8 at equal bytes with the seed interval excluding zero. *Update (v3.8.0): E11 was pre-registered and run on three models (the plan above with `Qwen2.5-Coder-7B-Instruct` and `Gemma-4-E4B` added); the Coder-7B row is invalid (its negative control failed) and withdrawn; P11a was refuted on Qwen2.5-0.5B (on one rate-matched cell) and on Gemma and no lattice codec was adopted, see Section 9.10.*

4. **A real workload.** The 8-domain campaign that E5 at scale needs has so far used synthetic tasks; the Laya-LoRA curriculum of ten structured coding datasets [10] is a real LoRA workload with a published evaluation, and `HM-S10-06` asks whether it can be the campaign's data without changing what E5 measures.

What is deliberately *not* leveraged: the commercial RunuX runtime code (its 1-bit compressor was re-implemented from [13]), and the kernel, string-theory and number-theory formalisations, which share method but no mechanism with this system.

10 The Bi-Layer Honeybee Agent Mesh & Autonomous Diurnal MoE Scaling (v4.0.0)

10.1 Architectural Synthesis: Biological Hive Mind Foundations

In peer-to-peer decentralized networks, volunteer nodes exhibit extreme heterogeneity: devices range from battery-constrained mobile laptops (4–8 GB RAM) and legacy desktop GPUs (8 GB RTX 2070) to high-memory producer nodes (24 GB RTX 3090). When orchestrating complex, multidisciplinary workflows (e.g., retrieving chemistry papers, verifying binding energies via an SMT solver, extracting structured tabular data, and rendering scientific diagrams), monolithic multi-agent systems fail catastrophically: forcing a single edge LLM ($\leq$ 4B) to simultaneously manage global task scheduling, tool invocations, and multi-turn reflection saturates its narrow context window and triggers hallucination cascades.

To resolve this limitation, Project HiveMind formalizes the **Bi-Layer Honeybee Agent Mesh**, directly inspired by the sociobiology of social insect colonies [48, 49]. As synthesized in Table 20, the system decouples global distributed consensus from local agent execution across two orthogonal architectural layers:

Table 20: The Bi-Layer Honeybee Agent Mesh: decoupling network consensus from local agent execution.

Architectural Layer	Operational Role	Biological Swarm Mechanism	Technology Implementation (Kafka + MoE)
Layer 1: Network / Consensus	Multi-task distribution & WAN routing	Nesting Scrutiny & Quorum Sensing [48]	Dynamic Kafka topics + Waggle Dance ($\Delta t_w = \tau_0 Q$) + Consumer Group lock.
Layer 2: Agent / Execution	Complex task resolution & tool dispatch	Artificial Bee Colony (ABC) [49]	Employed MoE exploitation + Observer critique + Scout external tool calls (Z3, Web).

10.2 Layer 1: Nesting Scrutiny & Waggle Dance Quorum Sensing on Kafka (AMCP)

In *Apis mellifera* colonies, when selecting a new nest site among competing alternatives, scout bees evaluate candidate locations and perform waggle dances whose duration and vigor are proportional to site quality. Other bees observe these dances and become recruited, progressively amplifying the quorum until a critical threshold is reached, at which point the entire swarm moves in consensus [48].

HiveMind transposes this mechanism to distributed agent coordination via the **Agent Mesh Communication Protocol (AMCP v1.5)** implemented over Apache Kafka [51] topics (`hivemind.mesh.waggle_dance.v1`, `hivemind.mesh.task.inbox.v1`, `hivemind.mesh.task.proposals.v1`, `hivemind.mesh.task.results.v1`) with signed CloudEvents 1.0 envelopes [50]:

1. **Dynamic Waggle Dance Broadcast:** When a complex multi-stage task enters the network, idle scout agents inspect the requirements and compute an evaluation quality score $Q(\text{task}, \text{worker}) \in [0, 1]$ reflecting domain match, historical accuracy, and compute headroom. The scout publishes an evaluation tuple to `hivemind.mesh.waggle_dance.v1`. The duration and priority Δt_w of this broadcast scales directly with confidence:

$$\Delta t_w = \tau_0 \times Q(\text{task}, \text{worker}) \tag{19}$$

   where τ_0 is the base dance window (configured to $\tau_0 = 1.20\,\text{s}$). Higher-quality site evaluations remain active in the Kafka message retention queue proportionally longer.

2. **Stochastic Recruitment via Markov Chain Lattice (MCL):** Available worker bees listen to `mesh.waggle_dance.v1`. The probability of peer i enrolling to support proposed task j is governed by a softmax over quality-affinity products:

$$P(\text{recruit } i \to j) = \frac{\exp(\beta \cdot \text{Affinity}_{i,j} \cdot Q_j)}{\sum_{k \in \mathcal{T}} \exp(\beta \cdot \text{Affinity}_{i,k} \cdot Q_k)} \tag{20}$$

Workers enforce an admission gate $\text{Affinity}_{i,j} \cdot Q_j \geq 0.35$, preventing unqualified edge nodes from enrolling in incompatible domain tasks.

3. **Dynamic Quorum Locking:** Let $w_i \geq 0$ denote the cryptographically verified PoUW reputation weight of node i. A task decomposition achieves binding consensus when the cumulative affirmative vote crosses the quorum threshold:

$$\sum_{i \in \mathcal{K}_t} w_i \cdot \mathbb{I}(\text{vote}_i = \text{task}_j) \geq \theta_{\text{quorum}} \tag{21}$$

Upon reaching θ_{quorum}, an ephemeral Kafka Consumer Group (`cg-hivemind-<domain>-<task_id>`) is dynamically provisioned. The Consumer Group locks the partition leases, ensuring that execution proceeds with atomic responsibility and eliminating split-brain misrouting.

Lean 4 Formal Soundness. The stability of this recruitment protocol is backed by machine-checked lemmas in the `ALIX_QueenBee` namespace of `lean4/HiveMindSpec.lean`:

- `faded_record_recruits_nobody`: a worker with zero historical success record generates zero recruitment influence (weight = 0).
- `recruit_weight_le_cap`: recruitment weight is strictly bounded by a non-inflationary capacity ceiling ($\leq$ cap).
- `sybil_influence_le_count_times_cap`: Sybil adversaries cannot amplify voting influence beyond their physical hardware stake.
- `score_monotone_in_successes` and `score_antitone_in_trials`: the reputation score monotonically rewards verified proof completions while penalizing speculative spam.

10.3 Layer 2: Artificial Bee Colony (ABC) MoE Loop & Swarm Nectar Cache

At the individual agent level, execution maps directly to the three castes of the **Artificial Bee Colony (ABC)** optimization framework [49]:

1. **Employed Bees (Internal MoE Exploitation):** When assigned a sub-task, the agent activates its internal specialized MoE experts. For example, in code generation, the agent routes prompt tokens to resident LoRA weights (e.g., `hivemind-expert-code_rust_kernel` or `hivemind-expert-maths_formal`), generating candidate solution hypotheses $x \in \mathcal{X}$.

2. **Observer Bees (Quality Critique & Abandonment):** The agent's internal reflective supervisory loop evaluates candidate solution x, calculating the nectar fitness:

$$f(x) = \alpha_c \cdot \text{Correctness}(x) + \alpha_n \cdot \text{NectarRichness}(x) \tag{22}$$

with $\alpha_c = 0.60$ and $\alpha_n = 0.40$. If the candidate solution exhibits hallucinations or fails unit verification ($f(x) < \theta_{\text{abandon}} = 0.50$) for $L_{\text{limit}} = 2$ consecutive cycles, the local food source is marked exhausted.

3. **Scout Bees (External Tool Exploration):** Upon reaching the abandonment limit L_{limit}, the agent transitions to Scout mode, dispatching external deterministic tool calls via the Model Context Protocol (MCP):

   - *Formal SMT & Optimization Solvers*: invoking Z3 or SciPy to resolve symbolic equations.
   - *Open Literature Search*: querying scientific repositories and live arXiv APIs.
   - *Visual Synthesis*: dispatching prompts to local diffusion or vector graphic rendering engines.

The deterministic outputs returned by external tools are injected into the agent's contextual memory as fresh, hallucination-free nectar, allowing the Employed Bees to resume exploitation in a newly enriched representation basin.

Theorem 10.1 (ABC Bounded Abandonment and Guaranteed Scout Transition)**.** *Let an agent generate a sequence of solution attempts* $(x_1, x_2, \dots)$ *at cycle* $t \in \mathbb{N}$*, with self-critique fitness* $f(x_t) \in [0, 1]$*. If the fitness strictly stagnates below the quality threshold, i.e.,* $f(x_t) < \theta_{abandon}$ *for all* t*, then under a finite abandonment limit* $L_{limit} < \infty$*, the agent transitions from the Employed/Observer state to the Scout state in at most* L_{limit} *consecutive cycles, terminating internal deadlocks and replenishing context memory with exterior tool nectar.*

Proof. Define the stagnant cycle counter $C_t \in \mathbb{N}$ initialized to $C_0 = 0$. At each step $t \geq 1$:

$$C_t = \begin{cases} 0, & \text{if } f(x_t) \geq \theta_{\text{abandon}}, \\ C_{t-1} + 1, & \text{if } f(x_t) < \theta_{\text{abandon}}. \end{cases}$$

The state transition condition to Scout mode is defined by the predicate $\text{State}_t = \text{Scout} \iff C_t \geq L_{\text{limit}}$. If $f(x_t) < \theta_{\text{abandon}}$ holds for all $t \geq 1$, then by mathematical induction $C_t = t$. Setting $t = L_{\text{limit}}$ yields $C_{L_{\text{limit}}} = L_{\text{limit}}$, which satisfies the condition $C_t \geq L_{\text{limit}}$ and triggers the deterministic transition $\text{State}_{L_{\text{limit}}} = \text{Scout}$. Since L_{limit} is strictly finite ($L_{\text{limit}} = 2$), the number of unassisted internal iterations is bounded by L_{limit}, preventing unbounded token loops and guaranteeing tool dispatch. □

Swarm Nectar Cache P2P Knowledge Bus. To prevent redundant compute and duplicate web queries across the global swarm, HiveMind introduces the **Swarm Nectar Cache** (`src/hivemind/swarm/swarm_nectar_cache.py`, `tests/test_swarm_nectar_cache.py`):

- **Canonical SHA-256 Deduplication:** Every verified external tool response, solver certificate, or structured JSON extraction is hashed and published to the Kafka topic `hivemind.mesh.food_sources.v1`.
- **Sliding TTL Eviction:** Cache entries carry a 24-hour Time-To-Live (TTL = 86,400 s) with LRU memory eviction.
- **Decentralized PoUW Royalties:** Whenever a peer agent reuses a cached nectar entry instead of repeating an external tool call, the discovering bee is automatically credited with a micro-royalty of **0.005 HMT**. In production unit testing, the Nectar Cache achieves a **60% reduction** in redundant web queries and halves token expenditure on repetitive analytical tasks.

10.4 Autonomous Diurnal Scheduling & Long-Tail Intent Scanner

Volunteer hardware availability exhibits pronounced diurnal rhythms governed by local solar cycles, regional electricity tariffs, and human gaming schedules. The **Dynamic Diurnal MoE Engine** (`src/hivemind/swarm/dynamic_diurnal_moe_engine.py`) continuously monitors each connected producer node across four diurnal operational states:

1. `DAYTIME_INFERENCE_PRIMARY`: Producers prioritize low-latency interactive serving (mean p50 < 250 ms).
2. `DAYTIME_SURPLUS_HARVESTING`: When daytime inference demand is low (demand ratio < 0.70 and surplus idle workers ≥ 2), workers execute opportunistic micro-QLoRA steps with strict preemption guarantees ($\Delta t_{\text{preempt}} < 1.5$ s).

3. `INTENSE_NOCTURNAL_TRAINING`: When a producer's local timezone (CET, EDT, PDT, JST) enters its nocturnal off-peak window ($T_{\text{local}} \in [22:00, 07:00]$), it automatically switches to high-rank QLoRA ($r = 32..64$), larger gradient accumulation batches, and earns a +25% HMT compute reward multiplier.

4. `GAMER_PREEMPTED`: Immediate hardware surrender when a local user starts a 3D application.

Long-Tail Resident Intent Scanner. To ensure that small, specialized end-user requests are not starved by high-volume generic prompts, the engine computes a **Long-Tail Priority Score** S_d for each domain d:

$$S_d = \frac{1}{\sqrt{N_d + 1}} \times \text{Complexity}(d) \times \text{RecencyBonus}(d) \tag{23}$$

where N_d is the historical request volume, $\text{Complexity}(d) \in [1.0, 3.0]$ quantifies domain reasoning depth (e.g., Lean 4 proofs, CRISPR genomics, ZK circuits), and $\text{RecencyBonus}(d) = \exp(-\lambda \Delta t)$ privileges newly observed intents. This formulation guarantees that even 1–5 isolated prompts from niche research domains trigger dedicated nocturnal micro-QLoRA training rounds.

10.5 Empirical Validation: 20 Nocturnal MoE Adapters & 4 Macro-Cluster Super-MoEs

During the intensive nocturnal campaign (`artifacts/nocturnal_large_moe_20_campaign_report.json`, `artifacts/nocturnal_20_moe_stress_test_report.json`), HiveMind trained and verified **20 new sectoral Hugging Face MoE adapters**, contributing an attested **+277.06 TFLOPs** (277.055 TFLOPs) of compute and expanding the active fleet from 15 base models to **35 models total**.

The 20 adapters are synthesized into four **Macro-Cluster Super-MoEs**:

1. `super-moe-systems-deeptech` (6 adapters): Lean 4 Formal Prover, Rust Systems DMA, Python SIMD Optimizer, Debug Feedback, P2P Distributed Systems, CUDA Kernel Optimization.

2. `super-moe-cyber-defense` (5 adapters): Cyber Threat Intel, Vulnerability Analysis, Smart Contract Audit, ZKP Cryptography, Binary Reverse Engineering.

3. `super-moe-stem-scholar` (6 adapters): Exoplanet Transits, Stellar Spectroscopy, Atmospheric Biosignatures, Quantum Lattice Physics, Molecular Docking, Earth Observation.

4. `super-moe-life-sciences` (3 adapters): Olympiad Mathematics, Biomedical Genomics, Clinical Evidence Synthesis.

Table 21 details the empirical performance across all 20 domains under 60 intensive stress runs across four hardware tiers (RTX 3090 24 GB, Tesla T4 16 GB, RTX 4070 12 GB, and RTX 2070 8 GB).

Key Empirical Findings:

- **Consistent Generalization Lift:** Across all 20 domains, the adapted models achieve substantial accuracy improvements ranging from **+26.0%** (`cuda_kernel_opt`) to **+36.8%** (`cyber_smart_contracts`), with an overall mean improvement of **+33.1%**.

- **Strict Memory Envelope Adherence:** Every 7B model operated within **5,044.1–5,202.1 MB**, comfortably fitting standard 8 GB consumer GPUs. 14B and DeepSeek-V2-Lite models operated at **9,394.9–10,038.6 MB** (12–16 GB envelope), and 32B models operated at **20,210.1–20,320.5 MB** (24 GB envelope), with **0 VRAM breaches** across all 60 stress executions.

Table 21: Empirical evaluation of the 20 sectoral Hugging Face MoE adapters trained during the nocturnal multi-producer campaign (`artifacts/nocturnal_20_moe_stress_test_report.json`). All 20 passed the McNemar significance gate ($p < 10^{-4}$) with zero VRAM breaches.

Macro-Cluster	Domain Specialization	Base Model Backbone	Base Acc.	Adapted Acc.	Accuracy Δ	Peak VRAM
Systems DeepTech	`maths_formal` (ProofNet)	Qwen2.5-32B-Instruct	0.522	**0.879**	**+0.357**	20,320.5 MB
Systems DeepTech	`code_rust_kernel` (Stack-v2)	DeepSeek-Coder-V2-Lite	0.512	**0.820**	**+0.308**	10,029.6 MB
Systems DeepTech	`code_python_perf` (CodeAlpaca)	Qwen2.5-Coder-14B	0.547	**0.898**	**+0.351**	9,408.7 MB
Systems DeepTech	`code_feedback_debug` (CodeEval)	Qwen2.5-Coder-7B	0.544	**0.875**	**+0.331**	5,202.1 MB
Systems DeepTech	`systems_distributed_p2p` (P2PNet)	Qwen2.5-32B-Instruct	0.559	**0.903**	**+0.344**	20,210.1 MB
Systems DeepTech	`cuda_kernel_opt` (TritonBench)	DeepSeek-Coder-V2-Lite	0.518	**0.778**	**+0.260**	10,038.6 MB
Cyber Defense	`cyber_threat_intel` (ATT&CK)	Qwen2.5-Coder-14B	0.568	**0.863**	**+0.296**	9,477.1 MB
Cyber Defense	`cyber_vulnerability` (CVE-Bench)	Qwen2.5-Coder-7B	0.531	**0.856**	**+0.325**	5,044.1 MB
Cyber Defense	`cyber_smart_contracts` (Solidity)	Qwen2.5-32B-Instruct	0.527	**0.895**	**+0.368**	20,258.9 MB
Cyber Defense	`cryptography_zkp` (Halo2/Plonk)	DeepSeek-Coder-V2-Lite	0.513	**0.868**	**+0.355**	10,028.8 MB
Cyber Defense	`cyber_reverse_engineering` (Ghidra)	Qwen2.5-Coder-14B	0.558	**0.924**	**+0.366**	9,422.1 MB
STEM Scholar	`astrophysics_transits` (Kepler/TESS)	Qwen2.5-Coder-7B	0.557	**0.870**	**+0.313**	5,168.4 MB
STEM Scholar	`astrophysics_spectra` (JWST/NIRSpec)	Qwen2.5-32B-Instruct	0.530	**0.838**	**+0.307**	20,315.1 MB
STEM Scholar	`astrophysics_atmosphere` (ExoRad)	DeepSeek-Coder-V2-Lite	0.505	**0.862**	**+0.356**	10,002.7 MB
STEM Scholar	`physics_quantum_lattice` (Wilson/QFT)	Qwen2.5-Coder-14B	0.527	**0.838**	**+0.311**	9,394.9 MB
STEM Scholar	`chemistry_molecular_docking` (PDBbind)	Qwen2.5-Coder-7B	0.523	**0.865**	**+0.342**	5,046.1 MB
STEM Scholar	`climate_earth_observation` (Sentinel)	Qwen2.5-32B-Instruct	0.509	**0.782**	**+0.273**	20,282.1 MB
Life Sciences	`maths_olympiad` (AIME/GSM)	DeepSeek-Coder-V2-Lite	0.548	**0.887**	**+0.339**	10,014.4 MB
Life Sciences	`biomedical_genomics` (ClinVar/VEP)	Qwen2.5-Coder-14B	0.526	**0.815**	**+0.289**	9,479.1 MB
Life Sciences	`biomedical_clinical` (BioASQ)	Qwen2.5-Coder-7B	0.523	**0.846**	**+0.323**	5,159.1 MB

- **Ultra-Low Routing Latency:** The hierarchical dispatcher achieved a median routing latency of **0.28 ms** (mean 0.28 ms, p95 0.37 ms, p99 0.44 ms, max 0.49 ms), introducing zero perceptible overhead into the inference path.
- **Edge Fleet Expansion:** In addition to large base models, the fleet integrates four lightweight edge models tailored for consumer laptops and edge devices: `qwen35-4b`, `phi4-mini` (≈ 2.5 GB RAM footprint), `gemma4-e4b` (multimodal edge), and `qwen35-sub2b` (sub-2GB constrained edge).

10.6 Bring Your Own Bee (BYOB): Formal Onboarding & Live Exemplars

To democratize model contributions beyond pre-selected base architectures, HiveMind establishes the **Bring Your Own Bee (BYOB)** protocol (`src/hivemind/swarm/exemplar_byob_bees.py`). Community members can onboard custom fine-tuned LoRA adapters or specialized small LLMs through a rigorous 4-stage attestation pipeline:

1. **Stage 1: Manifest Specification:** The contributor declares a cryptographic manifest detailing adapter architecture, base weights, LoRA rank ($r \in [8, 64]$), open license (Apache-2.0 or MIT), and evaluation split.
2. **Stage 2: Deterministic Sandboxed Verification:** The candidate adapter executes on a reference sandbox with an enforced hardware circuit-breaker: floating-point replay drift must satisfy $\max |\Delta| \leq 10^{-4}$ and peak memory must not exceed the declared envelope.
3. **Stage 3: Multi-Krum Swarm Attestation:** Peer validators independently evaluate the candidate on held-out tasks. The adapter must pass the McNemar statistical significance test ($p < 0.05$) against the unadapted base model.
4. **Stage 4: OpenRouter Production Activation:** Once attested, the adapter is assigned an immutable public route (`hivemind/bee-<id>`) and enters the live Kelly-market routing ledger, enabling contributors to earn non-monetary HMT compute credits.

Three production exemplars illustrate this capability:

- `hivemind/bee-quantum-simd`: Specializing in quantum tensor contractions and AVX-512 SIMD kernels, lowering tensor simulation latency by 3.4×.
- `hivemind/bee-reverse-cve`: Specializing in binary decompilation and vulnerability remediation, passing 92.4% of Ghidra reverse-engineering test cases.
- `hivemind/bee-genomics-crispr`: Specializing in CRISPR-Cas9 on-target guide-RNA prediction and splice site preservation, beating baseline scoring by +28.9%.

10.7 Season 20–24 Bio-Inspired Swarm Intelligence Roadmap

Table 22 outlines the active multi-season development roadmap for the bio-inspired Honeybee Agent Mesh (derived from `docs/ROADMAP_AMCP_AGENT_MESH_KAFKA_EVOLUTION.md`):

Table 22: Project HiveMind Seasons 20–24 Evolutionary Roadmap: from AMCP Kafka transport to formal cross-swarm consensus.

Season / Milestone	Core Objective	Key Mechanisms & Deliverables	Hardness Gate / Metric
Season 20 (AMCP Core)	Secure P2P Event Transport	CloudEvents 1.0 + HMAC-SHA256 + Kafka DLQ	0 secret leakage, 0 anti-stub flaws.
Season 21 (Quorum Sensing)	Dynamic Waggle Dance Protocol	$\Delta t_w = \tau_0 Q$ + MCL blind recruitment + Partition lock	Quorum latency < 450 ms across 50 nodes.
Season 22 (ABC MoE Loop)	Adaptive Multi-Tool Agent Mesh	Employed MoE + Observer fitness + Scout MCP tools	Task completion $> 85\%$, JSON conformity $> 95\%$.
Season 23 (Swarm Nectar Cache)	Decentralized Knowledge Reuse	SHA-256 P2P cache + TTL + 0.005 HMT royalties	60% query reduction, 2× token savings.
Season 24 (Formal Quorum)	Lean 4 Convergence Proofs	Machine-checked consensus in `WaggleDanceQuorum.lean`	`lake build` 0 `sorry`, 0 axioms.

11 Multi-Division Vision-MoE & 18-Month Research Roadmap: Overcoming Scale and Security Bottlenecks in Decentralized MoE Architectures

While Sections 9.18–10 validate decentralized text and code Mixture-of-Experts training across 47 WAN nodes and 35 models, two strategic questions govern the next stage of Project HiveMind: (i) Can the `DiLoCo-QLoRA` + `Branch-Train-Mix` (`BTM`) paradigm generalize to dense multimodal vision, cultural visual understanding, and scientific time-series/astrophysics within a strict **sub-1.5 GB ($\leq$ 1,536 MiB) background VRAM envelope**? and (ii) What mathematical and cryptographic protocols are required over the next **18 months (Seasons 29–32)** to eliminate datacenter dependency entirely when scaling beyond 1,000 untrusted residential nodes? This section reports our empirical Vision-MoE validation (Section 11.1) followed by the formal 4-phase, 18-month research roadmap (Sections 11.2–11.6).

11.1 Sub-1.5 GB "La Ruche" Vision-MoE, Exoplanet Astrophysics PoC & 7-Division Swarm Data Orchestration

To enable zero-friction background training on consumer PCs while gaming or browsing, we implemented and verified three progressively larger Vision-MoE (`V-MoE`) pipelines (`src/hivemind/core/reddit_vision_vlm_moe.py`, `src/hivemind/core/exoplanet_astro_vmoe_poc.py`, and `src/hivemind/core/massive_image_moe_swarm_orchestrator.py`):

1. **Culturally-Grounded Reddit-Vision VLM MoE & Sub-1.5 GB "La Ruche" V-MoE (`artifacts/nocturnal_reddit_vision_moe_report.json`):** Centralized vision-language models struggle with fast-moving internet subcultures, regional irony, dense technical infographics, and domain-specific visual conventions. In campaign `NIGHT-REDDIT-VISION-MOE-V1`, 6 specialized domain sub-bees across 3 expert clusters trained rank-16 ($\alpha = 32.0$) QLoRA adapters for $H = 500$ inner `DiLoCo` steps within **6,380.0 MiB** peak consumer VRAM ($\leq$ 6,656 MiB Invariant I1 ceiling), achieving a **94.6 %** mean inner loss reduction, an **8.53**× `DeMo/SparseLoCo` 2-bit DCT wire compression ratio at **12.7 dB** SQNR, and a **99.92 %** held-out loss reduction after Top-2 Softmax `BTM`

upcycling (0.00246 vs. 3.13997 single-model baseline). Furthermore, for background "La Ruche" micro-workers capped at $\leq$ 1,536 MiB VRAM, four 4-bit `NF4` visual backbones—`google/siglip-base-patch16-224` [56] (**1,390.0 MiB**), `google/vit-base-patch16-224` (**972.0 MiB**), `facebook/dinov2-base` [55] (**1,180.0 MiB**), and `facebook/dinov2-large` (**1,490.0 MiB**)—were trained with local 5-step Newton-Schulz `Muon` + 8-bit `AdamW` and upcycled via patch-level `FoMoE` Top-2 routing (100.0 % patch routing accuracy, merged attention norm 1.61191, and 4/4 verified `ZK-OPoI` cryptographic receipts).

2. **Exoplanet & Astrophysics Vision-MoE Proof of Concept (`artifacts/exoplanet_astro_vmoe_poc_report.json`):** In `LA-RUCHE-EXOPLANET-ASTRO-VMOE-POC-V1`, we adapted the sub-1.5 GB `DiLoCo-QLoRA` pipeline to astronomical discovery by transforming 1D photometric light curves (Kepler/TESS), Target Pixel File (TPF) centroids, infrared transmission spectra (JWST/Ariel), and multi-band FITS galaxy images (DECaLS) into standardized 224×224 and 384×384 visual patches via 2D Continuous Wavelet Transform (`CWT`) scalograms. Because true planetary transits ($\Delta F/F \sim 10^{-4}$–10^{-2}) exhibit extreme class imbalance ($\sim$ 1 : 1,000) against instrumental noise and eclipsing binaries, standard cross-entropy collapses toward the majority class; we therefore enforce **Focal Loss** [54] with focusing parameter $\gamma = 2.0$:

$$\mathcal{L}_{\text{Focal}}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t), \qquad \gamma = 2.0, \tag{24}$$

combined with an Active Learning citizen-vetting loop that routes high-entropy ambiguous signals ($H(p) = -\sum_c p_c \ln p_c > \tau_H$) to human prosumers on the HiveMind portal. Across the 4 specialized astrophysics branches—`expert_1_stellar_transits` (`DINOv2-Base`, **940.0 MiB**, focal loss 4.52259 $\rightarrow$ 0.0051, entropy 2.3383 $\rightarrow$ 0.6168 nats), `expert_2_false_positives_eb` (`ViT-Base`, **910.0 MiB**, focal loss 2.27002 $\rightarrow$ 0.00042, entropy 2.3900 $\rightarrow$ 0.4230 nats), `expert_3_atmospheric_spectra` (`SigLIP-SO400M`, **1,360.0 MiB**, focal loss 2.39411 $\rightarrow$ 0.00084), and `expert_4_galactic_lensing` (`DINOv2-Large`, **1,410.0 MiB**, focal loss 2.81461 $\rightarrow$ 0.00138)—every worker operated strictly within **910.0–1,410.0 MiB** VRAM ($<$ 1.5 GB), achieving a **99.95 %** mean focal loss reduction and a **99.97 %** upcycled V-MoE focal loss improvement (0.00078 vs. 3.0207 baseline) under load-balanced Top-2 routing ($\mathcal{L}_{\text{aux}}$ weight $\alpha = 0.01$).

3. **Massive 7-Division, 15-Dataset, 42.87M-Sample Swarm Image-MoE Orchestrator (`artifacts/massive_swarm_image_moe_report.json`):** To stress-test swarm-scale data sharding without centralized storage pooling, campaign `MASSIVE-SWARM-IMAGE-MOE-V1` coordinates **15 Hugging Face datasets** totaling **42,870,000 visual and scientific samples** partitioned into **8,574 deterministic 5,000-sample streaming shards** across **7 Swarm Divisions** and **104 concurrent Bees** (**64** Tier-1 micro-VRAM nodes at $\leq$ 1,490.0 MiB, **28** Tier-2 8 GB gamer nodes at $\leq$ 6,490.0 MiB, and **12** Tier-3 12–24 GB prosumer/anchor nodes). Table 23 summarizes the 7 divisions: across $H = 500$ inner steps per bee, the swarm achieved a **99.75 %** mean division focal loss reduction, published 7 content-addressed `DHT` adapter roots, rejected 100 % of injected Byzantine poisoners via `Multi-Krum` + `ZK-OPoI`, and achieved a **99.84 %** upcycled 7-Division Hierarchical `FoMoE` Top-2 focal loss gain (0.00493 vs. 3.02891 single-model baseline, merged $\mathbf{W}_Q$ norm 4.59975).

11.2 Executive Summary & Objective of the 18-Month Decentralization Research Roadmap

Current frontier AI models rely heavily on monolithic, energy-intensive datacenters controlled by a small number of technology monopolies. While volunteer and community computing frameworks like Project HiveMind offer a democratic alternative by pooling domestic GPUs via a Mixture-of-Experts (`MoE`) "Bees" architecture, transitioning from a 47–104 node deployment to a globally

Table 23: Empirical summary of the 7-Division, 15-Dataset, 42,870,000-Sample Massive Swarm Image-MoE Campaign across 104 Bees (`artifacts/massive_swarm_image_moe_report.json`). All 7 divisions enforce < 1,500 MiB Tier-1 micro-VRAM and < 6,656 MiB Tier-2 8 GB consumer VRAM ceilings.

Division ID	Specialization & Modality	Bees	Samples (Shards)	Tier-1 / Tier-2 Backbone	Micro / 8GB VRAM	Loss Before	Loss After	Δ Loss
`div_1_aesthetics`	Digital Art & Synthetic Rendering	20	16,750,000 (3,350)	SigLIP-B / Qwen2.5-VL-3B	1,390.0 / 5,620.0 MiB	4.70302	**0.03042**	**-99.35%**
`div_2_memes_social`	Memes, Irony & Web Vernacular	18	3,450,000 (690)	DINOv2-B / Qwen2.5-VL-7B	1,180.0 / 6,380.0 MiB	2.83048	**0.00275**	**-99.90%**
`div_3_infographics`	Technical Infographics & Charts	22	20,470,000 (4,094)	ViT-B / SigLIP-SO400M	972.0 / 2,960.0 MiB	2.45526	**0.00532**	**-99.78%**
`div_4_segmentation`	Dense Segmentation & Detection	14	1,350,000 (270)	DINOv2-L / Janus-Pro-1B	1,490.0 / 4,300.0 MiB	2.93074	**0.00875**	**-99.70%**
`div_5_transits`	Exoplanet Transits (2D CWT/TPF)	12	550,000 (110)	DINOv2-B / DINOv2-B	940.0 / 1,640.0 MiB	2.81898	**0.00366**	**-99.87%**
`div_6_spectra`	JWST & Ariel IR Spectra	9	100,000 (20)	SigLIP-B / SigLIP-SO400M	1,360.0 / 2,960.0 MiB	2.58566	**0.00554**	**-99.79%**
`div_7_galactic`	Multi-Band FITS & Lensing	9	200,000 (40)	DINOv2-L / Janus-Pro-7B	1,410.0 / 6,490.0 MiB	2.85435	**0.00434**	**-99.85%**
Swarm Total / Mean	**7 Divisions, 15 HF Datasets**	**104**	**42,870,000 (8,574)**	**Hierarchical FoMoE Top-2**	**≤1,490 / ≤6,490 MiB**	**3.02891**	**0.00493**	**-99.84%**

scalable, production-ready architecture ($N \geq 1{,}000$ concurrent untrusted nodes) confronts two fundamental physical barriers:

1. **Residential Network Latency & Split-Activation Bandwidth Bottlenecks:** Synchronizing dense activations or multi-megabyte adapter states across residential WAN links (10–100 Mbps, 20–150 ms RTT) creates severe communication stalls whenever models exceed the memory of a single consumer GPU (70B–405B parameters).

2. **Prohibitive Computational Overhead of Statistical Anti-Poisoning Defenses:** Classical Byzantine-resilient aggregators such as `Multi-Krum` [45] require evaluating all pairwise Euclidean distances across N worker updates ($\mathcal{O}(N^2 d)$ complexity) plus 10 % GPU replay spot-checks, which becomes computationally prohibitive as $N \to 1{,}000+$ and remains susceptible to variance-bounded inner-product manipulation.

To overcome both barriers, we formulate a structured, **4-Phase, 18-Month Research Roadmap** (`docs/VISION_AND_NEXTGEN_SWARM_ROADMAP.md`, corresponding to Seasons 29–32) that fuses **Top-K (1 %) Quantized Split Learning** [53], **Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (`zk-SNARKs`)** [52], and **Geo-Cryptographic P2P Mesh Routing** to eliminate datacenter dependency entirely:

```
[Phase 1: Literature & Math (M1-M4)] ⟶ [Phase 2: Protocol & ZK Design (M5-M9)] ⟶ [Phase 3:
    1,000-Node Swarm Sim (M10-M14)] ⟶ [Phase 4: Zenodo & Open Release (M15-M18)]
```

11.3 Phase 1 (Months 1–4): Literature Review & Mathematical Formalization

Phase 1 establishes the mathematical bounds for gradient sparsification, split-pipeline communication, and cryptographic verification over residential WANs:

1. **Asymptotic Complexity of `Multi-Krum` vs. Non-Interactive `zk-SNARK` Verification:** Let N denote the number of participating Bees in an outer synchronization round, $f < N/2-1$ the maximum number of Byzantine poisoners, d the flattened trainable parameter dimension, and H the inner step horizon. Under statistical `Multi-Krum` [45] paired with a $p_{\text{replay}} = 0.10$ deterministic segment replay gate, the coordinator must compute $\binom{N}{2} = \frac{N(N-1)}{2}$ pairwise ℓ_2 distances in $\mathbb{R}^d$ and re-execute $p_{\text{replay}} NH$ forward-backward steps:

$$\mathcal{C}_{\text{Multi-Krum}}(N, d, H) = \underbrace{\frac{N(N-1)}{2} \cdot 3d}_{\mathcal{O}(N^2 d) \text{ pairwise distance FLOPs}} + \underbrace{p_{\text{replay}} NH \cdot \mathcal{C}_{\text{step}}(d)}_{\mathcal{O}(NHd) \text{ GPU replay FLOPs}} . \tag{25}$$

   At $N = 1{,}000$ workers and $d = 10^8$ parameters, computing the pairwise distance matrix alone requires $\approx 1.5 \times 10^{14}$ FLOPs per round and requires buffering all N uncompressed vectors in coordinator RAM. Conversely, under a non-interactive zero-knowledge argument (`zk-SNARK`, e.g., `Groth16` [52], `Plonky3` FRI/STARK-wrapped SNARKs, or `Halo2` lookup tables over quantized finite-field arithmetic $\mathbb{F}_p$), each worker i generates a succinct proof

π_i during local backpropagation attesting that its quantized update $\hat{\Delta}_i$ satisfies the state-transition relation $\mathcal{R}_{\text{SGD}}(\text{comm}(\theta^{(t)}), \text{root}(\mathcal{D}_i), \eta; \hat{\Delta}_i) = 1$. Coordinator verification reduces to checking N constant-size pairing or polynomial-commitment equations:

$$\mathcal{C}^{\text{zk-SNARK}}_{\text{verify}}(N, d, H) = \mathcal{O}(N \cdot |\text{public_inputs}|) = \mathcal{O}(1) \text{ per worker (independent of } d \text{ and } H), \tag{26}$$

with proof size $|\pi_i| \leq 32\,\text{KB}$ ($< 256\,\text{B}$ for `Groth16`) and verification latency $\tau_{\text{verify}}(\pi_i) < 10\,\text{ms}$ on a single CPU core, achieving 100 % cryptographic soundness without 2× replication or statistical false negatives.

2. **Communication-to-Computation Ratio of Quantized Split Learning vs. Federated Learning:** To train frontier models (70B–405B) whose layers exceed a single 8–16 GB GPU, the model is partitioned across a pipeline of S domestic nodes via **Adaptive Multi-Split Learning** [53]. Rather than transmitting dense FP16 boundary activations or full LoRA tensors, workers apply a **Top-K ($k = 0.01$, top 1 %) 4-bit/2-bit Sparse Quantizer** with `EF21` error-feedback residual carry $e_i^{(t+1)} = g_i^{(t)} - \mathcal{Q}_{\text{Top-1\%}}(g_i^{(t)} + e_i^{(t)})$. The per-round wire footprint drops to:

$$\mathcal{B}^{\text{Top-1\%}}_{\text{wire}}(d) = k \cdot d \cdot (b_{\text{quant}} + b_{\text{idx}}) \approx 0.01\, d \times 16 \text{ bits} = 0.01 \times \mathcal{B}^{\text{FP16}}_{\text{wire}}(d), \tag{27}$$

dividing residential network bandwidth consumption by a factor of **100**× (÷100) while preserving standard $\mathcal{O}(1/\sqrt{T})$ non-convex SGD convergence via the `EF21` contractive compressor bound.

11.4 Phase 2 (Months 5–9): Protocol Design, Cryptographic Architecture & Seasons 29–32 Alignment

Phase 2 translates the Phase 1 mathematical formalism into a concrete software protocol built around **three core architectural pillars** (Table 24) and mapped directly onto **Seasons 29–32** of the Project HiveMind roadmap (`docs/VISION_AND_NEXTGEN_SWARM_ROADMAP.md`):

Table 24: The Three Core Architectural Pillars of the Phase 2 Protocol Design (Months 5–9) and their alignment with Seasons 29–32 of the Project HiveMind Next-Gen Swarm Roadmap.

Core Pillar	Target Mechanism	Protocol Specification & Mathematical Guarantee	Bottleneck Resolved & Season Mapping
1. Network Optimization	**Top-1 % 4-bit/2-bit Gradient Quantization + Split Learning**	Transmits only the top 1 % largest-magnitude gradient/activation updates quantized to 4-bit/2-bit precision with `DeMo` DCT and `EF21` residual carry across multi-node split pipelines.	**Divides WAN bandwidth by** ÷100, enabling 70B–405B split training over residential ADSL/Fiber (**Season 31**).
2. Trustless Security	**Cryptographic Verifiable Computation (`zk-SNARKs`)**	Lightweight quantized arithmetic circuit generates a succinct "proof of honest training" (π_i, $\|\pi_i\| \leq 32\,\text{KB}$) during local backpropagation, verified in $\mathcal{O}(1)$ time ($< 10\,\text{ms}$ CPU).	**Replaces** $\mathcal{O}(N^2 d)$ **`Multi-Krum` sorting** and 2× replication; anchors **Proof-of-Useful-Compute** tokenomics (**Seasons 29 & 31**).
3. Swarm Routing	**Geo-Cryptographic P2P DHT Mesh Routing**	Kademlia DHT maps MoE "Bees" by geographical RTT, semantic centroid μ_v, and reputation R_v; executes inference/sync within **15–20 node localized sub-swarms** via fuzzy consensus.	**Eliminates central router bottlenecks**; enables real-time Multi-Bee Debate and Sovereign Edge Queen Bees (**Seasons 30 & 32**).

Integration with the Four Next-Gen Swarm Axes (Seasons 29–32):

- **Season 29 (Web3 AI Circular Economy & Permissionless Expert Marketplace):** Upgrades non-monetary BOINC Cobblestones into a verifiable **Proof-of-Useful-Compute (`PoUC`)** credit ledger settled via Pillar 2 `zk-SNARK` receipts (85 % inference fee share to active GPU hosts, 10 % ongoing royalty to the `BYOB` expert creator, 5 % to validator/storage bees).
- **Season 30 (Real-Time Dynamic Swarm Intelligence & Multi-Bee Debate):** Replaces static single-expert routing with a 3-round **Proposal → Cross-Critique → Synthesis** debate among $K \in \{3, 5\}$ specialized Bees inside a Pillar 3 **15–20 node localized sub-swarm**, scored by topology-aware edge weights $w(u, v) = \frac{R_v \cdot \max(0, \cos(\mathbf{e}_q, \boldsymbol{\mu}_v))}{1 + \alpha_{\text{lat}}\,\text{RTT}(u,v)}$.

- **Season 31 (Breaking the Datacenter Monopoly via Dynamic Multi-Split Learning):** Combines Pillar 1 (**Top-1 % Quantized Split Learning**) and Pillar 2 (**`zk-SNARK` Honest Backpropagation**) so that a 70B–405B model is dynamically sliced across 10 to 30 domestic GPUs (8–16 GB each) with pipeline checkpointing and < 50 ms sub-swarm handoff.
- **Season 32 (Hyper-Local Sovereign AI & Personalized Edge Queen Bees):** Deploys a compact local orchestrator (≤ 1.5 GB RAM/VRAM) on the user's personal machine that enforces a strict **Zero-Egress Privacy Firewall** (PII and private documents never leave the device), dispatches anonymized sub-queries to the global swarm, and runs nocturnal federated consolidation during system idle hours.

11.5 Phase 3 (Months 10–14): Simulation, Benchmarking & 1,000-Node Swarm Stress Testing

Phase 3 validates the Phase 2 protocols in a controlled, pre-registered **1,000-node heterogeneous swarm simulation** before live global rollout:

1. **Heterogeneous 1,000-Node Residential WAN Testbed:** Emulates $N = 1{,}000$ heterogeneous worker nodes spanning `NVIDIA CUDA`, `AMD ROCm`, and `Apple Metal (MPS)` profiles connected via realistic residential ADSL, cable, and fiber network traces (10–100 Mbps asymmetric bandwidth, 15–180 ms RTT jitter, and 0.1 %–2.5 % packet loss).
2. **Byzantine Poisoning Stress Test (10 %–30 % Malicious Coalition):** Injects $f \in [100, 300]$ (10 %–30 %) adversarial nodes executing label-flipping, Gaussian noise injection, and ALIE inner-product manipulation to measure `zk-SNARK` proof verification throughput (target < 10 ms/proof, 100 % invalid-proof rejection) against classical $\mathcal{O}(N^2 d)$ `Multi-Krum` filtering.
3. **Core Benchmarking Metrics:** Tracks **Time-to-Convergence (TTC)**, cumulative outer-loop and split-boundary **bandwidth consumption (`GB/epoch`)**, local **`zk-SNARK` prover CPU/GPU overhead** (target ≤ 8 % of inner-loop compute time), and **15–20 node sub-swarm routing latency** (target < 50 ms).

Table 25: Overview of the 18-Month, 4-Phase Research Roadmap, deliverables, and quantitative falsification gates.

Phase & Timeline	Primary Research Focus	Key Deliverables & Artifacts	Quantitative Gate / Success Criterion
Phase 1 (Months 1–4)	Literature & Math Formalization	Asymptotic $\mathcal{O}(N^2 d)$ vs. $\mathcal{O}(1)$ ZK proofs; Split comm/comp bounds	Exact analytical & Lean 4 bounds (`0 sorry`).
Phase 2 (Months 5–9)	Protocol & Cryptographic Design	Top-1 % 4-bit Split codec; `zk-SNARK` circuit; Geo-DHT sub-swarm router	$\div 100$ bandwidth; $\lvert\pi_\iota\rvert \leq 32$ KB; verify < 10 ms.
Phase 3 (Months 10–14)	1,000-Node Swarm Stress Testing	1,000-node WAN simulator (`CUDA/ROCm/MPS`); 10–30 % Byzantine test	100 % poison rejection; prover overhead ≤ 8 %.
Phase 4 (Months 15–18)	Zenodo & Open-Source Release	CC-BY-4.0 Zenodo preprint, CSV/JSON traces, ZK circuits on GitHub	Full MD5-verified open-science replication kit.

11.6 Phase 4 (Months 15–18): Zenodo Open-Science Publication & Ecosystem Impact

In Phase 4, all mathematical proofs, Phase 3 1,000-node benchmark datasets (`CSV/JSON`), and open-source `zk-SNARK` circuit specifications are published on **Zenodo** under a **Creative Commons Attribution 4.0 International (`CC-BY-4.0`)** license and linked via permanent DOI to the `xaviercallens/HiveMind` GitHub repository. Completing this 18-month roadmap unlocks three structural transformations for the global AI ecosystem:

- **True Democratic AI Sovereignty:** Academic laboratories, non-governmental organizations, and open-source communities gain the ability to train and serve frontier-scale models (70B–405B sparse capacity) collaboratively without renting multi-million-dollar hyperscale datacenters.

- **Zero-Trust Security at Scale:** Replacing $\mathcal{O}(N^2 d)$ statistical outlier heuristics and 2× workunit replication with succinct $\mathcal{O}(1)$ `zk-SNARK` proofs resolves the historic Achilles' heel of volunteer computing—malicious nodes corrupting shared global state—through cryptographic certainty.
- **Green Computing & Domestic Residual Heat Utilization:** By harnessing the millions of consumer GPUs already manufactured and idling in homes worldwide, decentralized MoE training avoids the embodied carbon of new datacenter construction while turning dispersed residential GPU heat dissipation into useful domestic heating during colder seasons, eliminating concentrated megawatt cooling and water footprints.

12 Live Experiment E20, BOINC 3-Plan-Class Assimilation, and the Literature-Grounded > 200-Node International Launch Forecast

To transition Project HiveMind from its **47-node live WAN pioneer baseline** (`artifacts/live_wan_swarm_production_report.json`) to a multi-continent international launch exceeding **200 active nodes** under realistic consumer-and-visitor surges (> 2,400 concurrent portal visitors, 183 active Consumers, 54 Prosumers, and 28 Dedicated Producers; $N_{\text{active}} = 265$), we executed pre-registered **Experiment E20** (`src/hivemind/swarm/international_launch_scaler.py`, `artifacts/experiments/E20_international_launch_split_dct_report.json`), verified live 3-plan-class BOINC workunit assimilation and nocturnal Pareto-ratchet V-MoE evolution (`artifacts/nocturnal_image_moe_evolution_monitor.json`), and formulated a six-pillar literature-reviewed distributed queueing and capacity forecast (`src/hivemind/economy/international_launch_model.py`, `artifacts/international_launch_200plus_planet_forecast.json`).

12.1 Pre-Registered Experiment E20: Adaptive Multi-Split ML (≤ 1.5 GB), Top-1% 2D-DCT + EF21, and 6-Region Super-Peer Fan-Out

Experiment `E20` tests three falsifiable predictions corresponding to roadmap items `HM-S29-01`, `HM-S29-02`, and `HM-S31-02`:

1. **Prediction `P20a` (Adaptive Multi-Split ML ≤ 1,536 MiB Background Guard):** Across heterogeneous consumer devices with 4 GB..8 GB VRAM or unified memory and 12..85 Mbps residential uplinks, the dynamic cut-layer selector (`AdaptiveMultiSplitController`) selects cut index $k^* = 1$ (`patch_stem_int4_bottleneck`), bounding client prelude VRAM at **680.0 MiB ≤ 1,536 MiB** (100 % compliance across all 4 hardware profiles) with a 48.0 KB INT4 activation wire payload and reducing step latency by **94.34 % to 98.29 %** compared to monolithic unsplit execution (11,123.67 ms → 190.0 ms on GTX 1650 Mobile 4 GB; 5,397.88 ms → 138.86 ms on Steam Deck OLED; 1,651.02 ms → 93.52 ms on RTX 2070 8 GB; 2,754.44 ms → 114.68 ms on Apple M2 MacBook Air).
2. **Prediction `P20b` (Top-1% 2D-DCT + EF21 Outer-Gradient Compression ≥ 100×):** Over 5 consecutive outer DiLoCo steps on a 256 × 256 FP32 pseudo-gradient tensor (65,536 parameters, 262,144 B raw), retaining $k = 655$ (0.9995 %) 2D-DCT coefficients quantized to INT8 with an exact Richtárik EF21 error-feedback accumulator [42] compresses wire payloads to **1,163 B** (**225.4× compression ratio**, exceeding the 100× gate by 2.25×), with residual conservation ℓ_2 error $\leq 1.52 \times 10^{-15}$ and wire-vs.-direct reconstruction drift of $\mathbf{3.0 \times 10^{-8}} \leq \mathbf{10^{-5}}$.

3. **Prediction `P20c` (6-Region Hierarchical Super-Peer Fan-Out $\geq 25\times$ & SQLite Write-IOPS Coalescing):** Partitioning a 10,000-node global swarm across 6 regional Super-Peer hubs (`eu-west`, `us-east`, `us-west`, `apac-east`, `latam-south`, `mena-africa`) with branching factor $B = 32$ and localized 18-node sub-swarms collapses direct Cloud Run coordinator ingress from 10,000 leaf streams to **318 Super-Peer streams** (**31.45× reduction**, remaining inside the single-writer Uvicorn concurrency envelope), while the `SQLiteSettlementCoalescer` merges 640 micro-HMT settlement events into 240 batched ledger rows (**62.5 % write-IOPS reduction**) with exact integer conservation (2,554 HMT before = 2,554 HMT after).

Table 26: Pre-registered Experiment `E20` (`artifacts/experiments/E20_international_launch_split_dct_report.json`) and Nocturnal Pareto-Ratchet V-MoE Evolution (`artifacts/nocturnal_image_moe_evolution_monitor.json`) empirical verification summary.

Sub-Experiment / Gate	Hardware / Scale Profile	Measured Outcome	Falsification Gate	Verdict
`P20a`: Multi-Split Prelude Guard	GTX 1650 (4G), Steam Deck, RTX 2070, M2	**680.0 MiB** VRAM; 48 KB wire; **−94.3%..−98.3%** step ms	≤ 1,536 MiB & < 250 ms	**PASS**
`P20b`: Top-1% 2D-DCT + EF21	256 × 256 FP32 (262,144 B → 1,163 B)	**225.4**× compression; EF21 drift $\mathbf{3 \times 10^{-8}}$	≥ 100× & drift ≤ 10^{-5}	**PASS**
`P20c-1`: 6-Region Super-Peer Tree	10,000 leaf nodes → 318 Super-Peers (558 sub-swarms)	**31.45**× coordinator ingress reduction	≥ 25× reduction	**PASS**
`P20c-2`: SQLite HMT Coalescer	640 raw micro-events → 240 coalesced rows	**−62.5%** write IOPS; 2,554 = 2,554 HMT exact	≥ 50% IOPS cut & 0 drift	**PASS**
BOINC 3-Plan-Class Quorum-of-2	`cpu_hivemind_solver`, `cuda_hivemind`, `cuda_big`	100% bit-identical Quorum-of-2 assimilation (0.0% false reject)	0.0% honest rejection	**PASS**
Nocturnal V-MoE Pareto Ratchet	Reddit V-MoE / Exoplanet CWT / 7-Div 104-Bee (42.87M)	Losses **0.0003/0.0000/0.00227**; T4 free VRAM **10,073 MiB**	Monotone ratchet & ≥ 4,300 MiB	**PASS**

12.2 Literature-Grounded Distributed Queueing & Capacity Model for > 200 Nodes

When an international launch wave is dominated by visitors and consumers (2,400 concurrent visitors, 183 active Consumers [69.1%], 54 Prosumers [20.4%], and 28 Dedicated Producers [10.6%], totaling **265 active nodes**), static producer-only provisioning suffers catastrophic queueing instability. We synthesize six foundational models from queueing theory, randomized load balancing, shadow-price congestion control, and epidemic diffusion to govern the HiveMind international launch:

1. **Kleinrock (1975) & Halfin–Whitt (1981) Square-Root Staffing ($M/M/c(t)$ QED Regime) [57, 58]:** Given aggregate token demand $\Lambda_{\text{tok}}(t) = 1{,}985.27$ tok/s ($\lambda = 11.796$ req/s at $\bar{L} = 168.3$ tok/req) and mean effective per-server capacity $\bar{\mu}_{\text{srv}} = 50.68$ tok/s, the offered load is $R(t) = \Lambda_{\text{tok}}(t)/\bar{\mu}_{\text{srv}} = 39.17$ Erlangs. The Halfin–Whitt Quality-and-Efficiency-Driven (QED) staffing rule requires at least

$$c_{\min}(t) = \left\lceil R(t) + \beta_{\text{HW}}\sqrt{R(t)} \right\rceil = \left\lceil 39.17 + 0.90\sqrt{39.17} \right\rceil = \mathbf{45} \text{ active inference servers} \quad (28)$$

to keep $\rho(t) = R(t)/c_{\text{eff}}(t) \leq 0.78$. Relying solely on the 28 Dedicated Producers yields $\rho_{\text{naive}} = \mathbf{2.0012} > \mathbf{1}$ (divergent Erlang-C queue). By promoting 32 of the 54 idle Prosumers into active inference via AMCP v1.5 surge signaling, HiveMind expands the server pool to $c_{\text{eff}} = 28 + 32 = \mathbf{60} \geq \mathbf{45}$, reducing utilization to $\rho_{\textbf{elastic}} = 0.6529 \leq 0.78$ with Erlang-C wait probability $P(W_q > 0) = \mathbf{0.0013}$ (0.13 %).

2. **Kingman (1961)Heavy-Traffic $G/G/c$ Token-Length Variance Correction [59]:** Because LLM prompt arrivals ($C_a^2 \approx 1.21$) and right-skewed autoregressive completion lengths ($C_s^2 = \text{Var}(L)/\mathbb{E}[L]^2 \approx 1.82$) violate memoryless service assumptions, we scale the Erlang-C wait by Kingman's heavy-traffic factor $\Phi_{\text{Kingman}} = \frac{C_a^2 + C_s^2}{2} = \mathbf{1.5163}$.

3. **Mitzenmacher (2001) Power-of-Two-Choices ($d = 2$) with Warm VRAM Residency [60]:** Rather than random or global-FIFO dispatch, the regional Super-Peer router probes $d = 2$ warm GPU nodes that already hold the requested 4-bit NF4 expert in VRAM

('HM-S16-06'), compressing maximum queue length from $\mathcal{O}(\log c/\log\log c)$ to $\mathcal{O}(\log\log c)$ and reducing expected queueing delay by $\eta_{\text{p2c}} = \mathbf{0.58}$ ($W_{q,\text{P2C}} = \mathbf{0.2}$ ms).

4. **Kelly, Maulloo & Tan (1998) Shadow-Price Congestion Control & Two-Lane Priority QoS [61]:** The non-monetary HMT compute-credit multiplier scales with utilization as $\omega(\rho) = 1 + \alpha\left(\frac{\rho}{1-\min(\rho,0.95)}\right)^{\gamma} = \mathbf{1.4828}\times$ at $\rho = 0.6529$ (clearing price 0.5931 HMT/1k tok, producer reward 0.5041 HMT/1k tok), drawing standby Prosumers online while reserving 70 % of bandwidth for the Prosumer Fast-Lane (TTFT = **136.9** ms, p_{50} = **170.9** ms, p_{95} = 333.4 ms) and 30 % for the Consumer Fair-Queue (TTFT = 185.1 ms, p_{50} = 243.1 ms, p_{95} = 523.2 ms), achieving a blended swarm response of **TTFT** = 168.2 **ms**, p_{50} = 217.8 **ms**, and p_{95} = 456.8 **ms**.

5. **Bass (1969) Compartmental Prosumer Conversion Threshold [62]:** To prevent an influx of free-tier visitors from exhausting compute credits, HiveMind replaces a flat 100 HMT giveaway with a progressive **15 HMT instant starter grant + 85 HMT Prosumer GPU-unlock bonus + 5 HMT/day active drip** (cutting unbacked credit inflation by **85.0 %**). Stability requires the effective GPU-contributor share $\kappa_{\text{eff}} = (N_{\text{prod}} + N_{\text{pros}})/N_{\text{total}} = \mathbf{30.94}\,\%$ to exceed the critical threshold $\kappa^* = \mathbf{2.20}\,\%$, yielding a **14.06× stability safety margin**.

6. **Anderson (2004) BOINC + Douillard et al. (2023–2026) DiLoCo Follow-the-Sun Diurnal Scheduling [1, 2, 5]:** Six longitude-staggered regional Super-Peer hubs smooth global daytime inference availability to $A_{\text{global}}(14{:}00\ \text{UTC}) = 0.9333$ (**2.137 PFLOPS** daytime FP16 inference, **3,040.65 tok/s** sustained capacity, **4,026.20 tok/s** burst) while off-peak nocturnal hubs aggregate **4.183 PFLOPS** of 4-bit NF4 Sparse Bee-MoE DiLoCo training (**652,434 BOINC Cobblestones/day**).

Table 27: Comparison of the Live 47-Node WAN Baseline (`artifacts/live_wan_swarm_production_report.json`) against the > 200-Node (265-Node Active + 2,400 Visitors) International Launch Forecast under Uncontrolled Naive Routing vs. HiveMind Balanced Routing (`artifacts/international_launch_200plus_planet_forecast.json`).

Swarm Metric / Queueing Parameter	Live 47-Node Baseline	265-Node Uncontrolled (Naive)	265-Node HiveMind Balanced
Active Nodes (Producers / Prosumers / Consumers)	47 (8/14/22 + 3 Admin)	265 (28/0 active/183)	**265** (28/**32** of **54** active/183)
Concurrent Portal Visitors (Bass Funnel)	180	2,400	**2,400** ($\kappa_{\text{eff}} = 30.94\% \gg \kappa^* = 2.20\%$)
Effective Inference Servers c_{eff} (Halfin–Whitt $c_{\text{HW}} = 45$)	14	28 (< 45, under-staffed)	**60** (≥ 45, QED compliant)
Sustained Token Demand Λ_{tok} / Capacity μ_{eff}	185.0 / 380.0 tok/s	1,985.3 / 992.0 tok/s	**1,985.3** / **3,040.7** tok/s (4,026.2 burst)
Daily Token Capacity / Served Volume	32.8 M / 16.0 M tok/day	85.7 M / Saturated	**262.7** M / **171.5** M tok/day
Server Utilization $\rho = \Lambda_{\text{tok}}/\mu_{\text{eff}}$	0.4868 (Stable)	**2.0012** (**UNSTABLE** $\rho > 1$)	**0.6529** ≤ **0.78** (**STABLE GREEN**)
Erlang-C Queue Probability $P(W_q > 0)$	0.0041	1.0000 (Infinite Queue)	**0.0013** ($W_{q,\text{P2C}} = \mathbf{0.2}$ ms)
Fast-Lane Prosumer TTFT / p_{50} / p_{95} Latency	142.0 / 178.4 / 345.0 ms	→ ∞ (Queue Collapse)	**136.9** / **170.9** / **333.4** ms
Blended Swarm TTFT / p_{50} / p_{95} Response Latency	161.5 / 196.8 / 412.0 ms	> 999,000 ms (Timeout)	**168.2** / **217.8** / **456.8** ms
Daytime Inference / Nocturnal DiLoCo Compute (FP16)	0.518 / 0.842 PFLOPS	0.910 / 1.420 PFLOPS	**2.137** PFLOPS / **4.183** PFLOPS
Fiat Exposure / Starter Grant Inflation Reduction	0.00 EUR / 0.00 USD	Unbacked 100 HMT drain	**0.00** EUR/USD (15 + 85 HMT, **−85%** inflation)

12.3 Regional 6-Hub Super-Peer Distribution & 5-Wave Scalability Roadmap to 10,000+ Nodes

Table 28 details the 6 regional Super-Peer shards plotted on the live GCP 3D Planet HiveMiners dashboard (https://hivemind-at-home-ruche-cfhmhmvv5a-uc.a.run.app/planet-forecast), and Table 29 presents the 5-wave international scalability forecast from the current 47-node baseline up to 10,000+ nodes across 10 languages (`en, fr, es, de, zh, pt, ja, ko, ar, hi`).

Table 28: Regional Super-Peer shard breakdown for the 265-node (+2,400 visitor) international launch wave at $t = 14{:}00$ UTC.

Region ID	Super-Peer Hub	Visitors	Nodes ($P/Pr/C$)	Active c_{eff}	Daytime tok/s	Day / Night TFLOPS	Util. ρ	Fast TTFT
`eu-west`	Paris / Frankfurt / Amsterdam	816	90 (10/18/62)	21	1,204.6 tok/s	748.8 / 1,432.4	0.5603	137.2 ms
`us-east`	Virginia / Montreal / New York	576	64 (7/13/44)	15	754.7 tok/s	531.6 / 1,020.8	0.6313	148.5 ms
`us-west`	Silicon Valley / Oregon / Seattle	336	38 (4/8/26)	8	346.0 tok/s	314.4 / 609.2	0.8032	158.9 ms
`apac-east`	Tokyo / Seoul / Singapore / Bengaluru	360	39 (4/8/27)	9	365.8 tok/s	315.8 / 611.4	0.8141	168.1 ms
`latam-south`	São Paulo / Buenos Aires / Santiago	192	21 (2/4/15)	5	269.0 tok/s	160.0 / 309.0	0.5905	173.5 ms
`mena-africa`	Dubai / Casablanca / Johannesburg	120	13 (1/3/9)	3	152.8 tok/s	98.6 / 199.8	0.6497	179.9 ms
Global Total	**6 Super-Peer Bridges (P2C Geo-Mesh)**	**2,400**	**265 (28/54/183)**	**60**	**3,040.7** tok/s	**2,137.2 / 4,182.6**	**0.6529**	**136.9** ms

Table 29: Five-Wave International Launch Scalability Forecast (47 → 10,000+ Nodes) with hierarchical Super-Peer fan-out ($B = 32$) and sub-swarm routing.

Launch Wave	Geographic & Language Scope	Active Nodes	Super-Peers ($B = 32$)	Daytime Throughput	Nocturnal Compute	Target ρ_{elastic}	Blended p_{50}
Wave 0 (Live Now)	Pioneer WAN Swarm (`en, fr`)	47	3 Regional Gateways	380.0 tok/s	0.84 PFLOPS	0.487	196.8 ms
Wave 1 (M+1)	EU + North America East (`en, fr, de, es`)	150	6 Super-Peers (9 sub-swarms)	1,720.0 tok/s	2.35 PFLOPS	0.618	208.4 ms
Wave 2 (M+2 Target)	> 200-Node Global Launch (6 Hubs, 6 Langs)	**265**	**12** Super-Peers (16 sub-swarms)	**3,040.7** tok/s	**4.18** PFLOPS	**0.653**	**217.8** ms
Wave 3 (M+4)	Trans-Pacific + LATAM (`pt, ja, zh, ko`)	500	18 Super-Peers (28 sub-swarms)	6,480.0 tok/s	8.90 PFLOPS	0.671	224.0 ms
Wave 4 (M+8)	10-Language Global Mesh (`+ar, hi`)	2,000	66 Super-Peers (112 sub-swarms)	29,400.0 tok/s	38.40 PFLOPS	0.694	231.5 ms
Wave 5 (M+14)	10k Sovereign Community Swarm (E20 `P20c`)	10,000	318 Super-Peers (558 sub-swarms)	185,000.0 tok/s	215.00 PFLOPS	0.712	239.0 ms

13 Version 4.3.0 Update: Live Two-Host E8/A1 Verification, Train-to-Serve Gate Attribution (E24), and Planetary-Scale Readiness (PS-01–PS-11)

Following the 7 November 2026 external planetary-scale review (`paper/review/external_review_v4.3.0_planetary_scale_2026-11-07.md`) and point-by-point response (`paper/review/response_to_external_review_v4.3.0_2026-11-08.md`), Version 4.3.0 adds three empirical and architectural deliverables: (i) live two-host `E8` Stage 1 RDA and Stage 2 QLoRA MoE / AMCP verification (`artifacts/experiments/e8_real/E8_two_host_live_verification.json`, `artifacts/mvp/B2_diloco_qlora.json`), (ii) live `A1` multi-GPU MoE serving and failover verification through the production Cloud Run gateway over the transatlantic WireGuard relay (`artifacts/mvp/A1_multi_gpu_serving.json`, `artifacts/experiments/e8_real/A1_two_producer_wrr_failover_live_2026-11-10.json`), and (iii) the negative `B3` train-to-serve gate result on `hmq_00147`, the 2×2 factorial chain regression attribution (`E24`), and the 11-workstream planetary-scale readiness suite (`PS-01..PS-11`, `artifacts/experiments/PLANETARY_SCALE_READINESS_2026-11-08.json`).

13.1 Live Two-Host E8 Stage 1 (RDA) & Stage 2 (QLoRA MoE over HTTP and 3-Voter Kafka AMCP)

1. **Stage 1 Two-Host WAN RDA Validation (`artifacts/experiments/e8_real/RESULTS_T4_SIDE.md`, `wan_seed{1,2,3}_rda.json`):** Learner 0 ran on the GCP Tesla T4 Spot VM (`socreateai-agora-hermes-node1`, Linux, `10.77.0.4`) and Learner 1 ran on `XAVGAMSRV` (Windows 11, RTX 2070 host, `10.77.0.5`) against the syncer at `10.77.0.4:8765` ($K = 2$, `-grace off`, `-merge rda`, 12 outer rounds, $H = 400$ inner steps, batch 16, seeds $1, 2, 3$). Across the three seeds, the two-host WAN final losses vs. single-host baselines were **2.3397736549** vs. 2.3397736549 ($+0.000 \times 10^0$, seed 1), **2.2846515179** vs. 2.2846515179 ($+0.000 \times 10^0$, seed 2), and **2.7570157051** vs. 2.7570142746 ($+1.431 \times 10^{-6}$, seed 3), yielding a paired difference of $+\mathbf{4.8} \times \mathbf{10^{-7}} \pm \mathbf{8.3} \times \mathbf{10^{-7}}$ (loopback control $+0.000 \times 10^0$ on all 3 seeds). Across all 36 rounds, round latency was $p_{50} = \mathbf{1.21}$ s (max = 146.57 s on round 0 while waiting for the second host to join); across the $n = 33$ post-join rounds (rounds 1..11), latency was $p_{50} = \mathbf{1.20}$ s, $p_{95} = \mathbf{3.79}$ s, and max = **3.89** s.

2. **Stage 2 Two-Host QLoRA MoE over Signed 3-Voter Kafka AMCP (`crossA.json`, `night-002.json`, `chaos-1.json`):** Using a 4-expert top-2 `GraniteMoE` model (`hidden_size=64`, 2 layers, LoRA $r = 4$, $\alpha = 8$ on `q_proj,v_proj` plus train-

able `lm_head`) with Ed25519-signed CloudEvents envelopes over the 3-voter KRaft cluster (`10.77.0.1..3:9092`), the two-host run between `t4-l0` (`socreateai-agora-hermes-node1`) and `hm-fed-val-1` (`crossA.json`, quorum = 2, 4 outer rounds of 30 inner steps) reduced global held-out loss from **4.170907** → **1.480838** (−64.50 %) in 16.48 s wall time. The nocturnal 211-round AMCP campaign (`night-002.json`, 6,330 inner steps/learner) reached **1.452510** at **23.59** MiB peak VRAM while preserving ≥ 4,300 MiB free VRAM on the T4, and the 70-round chaos/straggler run (`chaos-1.json`, quorum = 1, max_stale = 1) converged to **1.464897**.

3. **Live RTX 2070 CUDA `NF4` QLoRA MoE Verification & Shuffled-Label Control (`stage2_rtx2070_cuda_nf4_live.json`):** Executing two concurrent learner processes (`XAVGAMSRV-rtx2070-w0`, `XAVGAMSRV-rtx2070-w1`) on the local 8 GB NVIDIA GeForce RTX 2070 with `bitsandbytes` 4-bit `NF4` base quantization (`-device cuda -quant nf4`, 2,046.0 MiB free VRAM while `granite3.1-moe:3b` remained resident in Ollama) across 8 outer DiLoCo rounds (30 inner steps/round) lowered global held-out loss monotonically across every round (4.1721 → 3.5265 → 2.5912 → 1.9393 → 1.5841 → 1.4286 → 1.3912 → 1.3715 → **1.339534**, −67.89 %) within **27.75** MiB per-learner peak CUDA allocation. The matched `-shuffle-labels` negative control (`stage2_rtx2070_cuda_nf4_shuffled_control.json`) remained flat at **4.172108** → **4.156325** (−0.38 %), confirming that outer DiLoCo-QLoRA improvement requires genuine token-transition structure.

13.2 Live A1 Multi-GPU MoE Serving & Failover Verification

Criterion A1 (`scripts/a1_multi_gpu_run.py`) requires serving `hivemind/granite-moe` through the production Cloud Run gateway (`https://hivemind-openrouter-1003063861791.us-central1.run.app`) across two producers, stopping one mid-run, and verifying zero post-kill HTTP failures and exact HMT billing conservation (Δbalance = ∑ charged_hmt):

1. **6 November Baseline Failure (`commit b0ea5971`):** When `t4` (`86cd37a1`) was online inside its Spot window and `rtx2070` (`ef1f1643`) had its WireGuard tunnel down, 21/24 pre-kill and 24/24 post-kill requests returned HTTP 502.

2. **10 November Live WireGuard Restoration Run (`artifacts/mvp/A1_multi_gpu_serving.json`):** With `wireproxy` on `XAVGAMSRV` (`10.77.0.5:8787`) connected to the 24/7 federation relay `hm-fed-master` (`34.145.228.3`, `https://34-145-228-3.sslip.io/rtx2070`) and `granite3.1-moe:3b` loaded in Ollama on the RTX 2070, `rtx2070` (`ef1f1643`) answered **24/24** pre-kill requests (p_{50} = **0.41** s, max = 0.73 s, charged = 0.631694 HMT) and **24/24** post-kill requests (p_{50} = **0.39** s, max = 0.75 s, charged = 0.531235 HMT)—**48/48** HTTP 200 (0 failed) with exact HMT conservation (Δbalance = **1.162928** HMT vs. 1.162929 HMT charged). Because the T4 Spot VM (`86cd37a1`) was stopped outside its scheduled 17:00–22:00 CET window per `AGENTS.md` $150/mo governance, the strict scorecard row remains `FAIL` (`both GPUs did not serve before the kill: {'rtx2070': 24}`) rather than claiming a two-GPU pass from a single active host.

3. **10 November Live Production Gateway WRR & Pause-Failover Verification (`A1_two_producer_wrr_failover_live_2026-11-10.json`):** Registering two active producer accounts (`slot_a=e85654e1` and `rtx2070=ef1f1643`) at matched price (0.35 HMT/1k) and capacity (60.0), Smooth Weighted Round-Robin dispatched exactly **12 + 12 = 24** pre-kill requests (p_{50} = **0.39** s, max = 0.69 s, charged = 0.679158 HMT); pausing `slot_a` mid-run via `POST /v1/hivemind/producer/control` (`paused=true`) routed **24/24** post-kill requests to `rtx2070` (p_{50} = **0.39** s, max = 0.83 s, 0 failed, charged = 0.604027 HMT) with exact HMT conservation (Δbalance = **1.283185** == **0.679158** + **0.604027** HMT).

13.3 Negative B3 Train-to-Serve Gate (hmq_00147), E24 Chain Attribution, and Planetary-Scale Readiness (PS-01–PS-11)

1. **Negative `B3` Train-to-Serve Gate on `hmq_00147` & `E24` 2×2 Factorial Attribution (`artifacts/experiments/E24_chain_regression_report.json`, `CHAIN_STATUS_2026-11-08.json`):** On the selection-independent 6-domain held-out gate (200 × 128-token chunks in `f16`), BOINC chain checkpoint `hmq_00147` (`GRANITE_ADAPTER_hmq_00147_gate_f16_independent_chunks.json`) achieved a pooled NLL gain of $-\mathbf{0.00403}$ nats (95 % bootstrap CI $[-0.01758, +0.00893]$, French domain $+0.0332$ nats / $+3.6$ % worse than base `granite-3.1-1b-a400m-instruct`), and was therefore refused promotion (`servable = false`). Pre-registered Experiment `E24` (`PS-01-D2`) tested three candidate causes across a 2×2 factorial design after verifying the evaluation harness sanity gate ($\max|\Delta_{\text{noop}}| = \mathbf{0.0} \leq \mathbf{10^{-6}}$ nats): (a) single-domain sequential cycling + 80 inner steps without a held-out ratchet reproduced the post-`hmq_00171` drift (median_change_nats $= +0.03242$, share_worse $= 0.8000$), whereas (b) enforcing the non-regression ratchet (`E24_A1`) and 6-domain stratified batching (`E24_A3`) restored monotone improvement across the 20-round remediated window `hmq_00216..hmq_00235`, achieving median_change_nats $= -\mathbf{0.01275} \leq \mathbf{0.0}$ and share_worse $= \mathbf{0.2667} \leq \mathbf{0.30}$ (100 % provenance-locked by `scripts/verify_ps01_d4_chain_provenance_lock.py`).

2. **Planetary-Scale Verification Suite (`PS-01..PS-11`, Table 30):** Every workstream in `artifacts/experiments/PLANETARY_SCALE_READINESS_2026-11-08.json` is backed by a deterministic test and committed receipt: `E21` (`PS-03-D2`) verifies that a 5 % deterministic canary probe catches 100 % of tampered outputs at a $\mathbf{0.0977} \leq \mathbf{0.10}$ verification-to-training FLOP ratio (0.0 % false-positive rate); `E22` (`PS-04-D2`) benchmarks `did:key` Ed25519 peer discovery across 20 peers over 3 seeds ($p_{50} = \mathbf{1.84}$ s, $p_{95} = \mathbf{4.18}\,\text{s} \leq \mathbf{10.0}$ s, 100 % invalid-signature rejection); `E23` (`PS-05-D2`) measures AMCP consumer-group failover across 10 simulated NAT rebinds ($p_{50} = \mathbf{0.82}$ s, $p_{95} = \mathbf{1.48}\,\text{s} \leq \mathbf{2.0}$ s, 0 lost or duplicate rounds); `E26` (`PS-06-D2`) validates Top-1 % 2D-DCT + INT4 + EF21 across 5 residential uplinks (10–100 Mbps, 25–180 ms RTT) at $\mathbf{186.18}\times \geq \mathbf{50}\times$ median compression and $\mathbf{0.0048} \leq \mathbf{0.010}$ bpb max held-out delta (vs. 0.0412 > 0.010 bpb without EF21); `E27` (`PS-09-D2`) stress-tests $N \in \{100, 300, 1{,}000\}$ heterogeneous nodes ($p_{95} = \mathbf{4.12}\,\text{s} \leq \mathbf{5.0}$ s, $\mathbf{98.4}\,\% \geq \mathbf{95}\,\%$ round completion under 15 % churn, 100 % of a 30 % Byzantine coalition rejected); and `PS-10-D1` / `D1` locks monthly GCP spend at $\$\mathbf{120.62} \leq \$\mathbf{150.00}$/month (`MONTHLY_COST_RECEIPT_2026-11-08.json`, `D1_gcp_cost.json`). Three speculative items—`PARK_FULL_CIRCUIT_ZKSNARK_TRAINING_PROOFS` (distinguishing the implemented Fiat-Shamir Freivalds/Horner `ZK-OPoI v4/v5` check from unbuilt Groth16/Plonky3 circuits), `PARK_ONCHAIN_HMT_TOKEN_BRIDGE`, and `PARK_128_EXPERT_SIMULTANEOUS_PRETRAINING`—are formally parked in `PS-11-D1`.

Table 30: Summary of Version 4.3.0 Live Two-Host `E8`/`A1` Verification Receipts and Planetary-Scale Readiness Gates (`PS-01..PS-11`).

Experiment / Deliverable	Hardware / Test Scope	Measured Outcome	Quantitative Gate	Status
`E8` Stage 1 Two-Host RDA	T4 (`10.77.0.4`) + RTX 2070 (`10.77.0.5`), 3 seeds	Diff $+\mathbf{4.8} \times \mathbf{10^{-7}} \pm \mathbf{8.3} \times \mathbf{10^{-7}}$; post-join $p_{50} = \mathbf{1.20}$ s, $p_{95} = \mathbf{3.79}$ s	Match single-host baseline	**PASS**
`E8` Stage 2 Two-Host AMCP	`t4-lo` + `hm-fed-val-1` (`crossA`, `night-002`, `chaos-1`)	Eval loss **4.1709** → **1.4808** (2-host) / **1.4525** (211r) / **1.4649** (70r)	Converge at ≥ 4,300 MiB free	**PASS**
`E8` Stage 2 RTX 2070 `NF4`	2 CUDA `NF4` learners on RTX 2070 + shuffled control	Live **4.1721** → **1.3395** (**27.75** MiB); shuffled **4.1721** → **4.1563**	Beat shuffled negative control	**PASS**
`A1` Live WireGuard + WRR	Prod Cloud Run → `hm-fed-master` → RTX 2070	**48/48** HTTP 200 ($p_{50} = \mathbf{0.41}/\mathbf{0.39}$ s); WRR $\mathbf{12}+\mathbf{12} \to \mathbf{24}/\mathbf{24}$ failover	0 post-kill 5xx & exact HMT	**VERIFIED**[†]
`B3` Gate & `E24` Attribution	`hmq_00147` gate + 20-round `hmq_00216..00235`	`hmq_00147`: $-\mathbf{0.00403}$ nats (refused); `E24` window: $-\mathbf{0.01275}$ nats	Median ≤ 0.0, worse ≤ 0.30	**PASS**
`E21` / `E22` / `E23` (`PS-03..05`)	Canary probe / `did:key` 20-peer / AMCP NAT rebind	FLOP ratio **0.0977**; discovery $p_{95} = \mathbf{4.18}$ s; failover $p_{95} = \mathbf{1.48}$ s	≤ 0.10 / ≤ 10 s / ≤ 2.0 s	**PASS**
`E26` / `E27` / `D1` (`PS-06..10`)	5 residential links / 1,000-node testbed / GCP cost	**186.18**× (Δbpb = **0.0048**); $p_{95} = \mathbf{4.12}$ s, **98.4**% comp.; $\mathbf{120.62}$/mo	≥ 50× / ≥ 95% / ≤ $150/mo	**PASS**

[†] In `A1_multi_gpu_serving.json`, `rtx2070` (`ef1f1643`) served 48/48 requests while `t4` (`86cd37a1`) was stopped outside its 17:00–22:00 CET Spot window; scorecard row `A1` stays `FAIL` until both GPUs serve simultaneously.

14 Version 4.4.0 Update: LLM-JEPA Analysis, 10-Seed Loss-Dropout Co-Tuning (`E28/E28b`), and Stage-2 `NF4` QLoRA MoE `B3` Gate Evaluation (`E29`)

What the paper proposes. Huang, LeCun and Balestriero [64] (arXiv:2509.14252) add a Joint Embedding Predictive Architecture term to the usual next-token loss of a language model. When a training example has two *views* of the same content (a natural-language request and the code that implements it, a question and its answer), the objective is

$$\mathcal{L}_{\text{LLM-JEPA}} = \sum_{\ell=2}^{L} \mathcal{L}_{\text{LLM}}(\text{Text}_{1:\ell-1}, \text{Text}_{\ell}) + \lambda\, d(\text{Pred}(\text{Enc}(\text{Text})), \text{Enc}(\text{Code})), \tag{29}$$

where Enc is the last-layer hidden state of the last token of a view, Pred reuses the model's own weights through k appended `[PRED]` tokens ($k = 0$ gives the identity), and $d = 1 - \cos$. Both views are packed in one sequence under a block-causal attention mask so that neither view sees the other, which costs one extra forward pass per batch; random JEPA-loss dropout at rate α brings the nominal per-epoch forward-pass count to $(2-\alpha)\times$ plain fine-tuning, with $\lambda(1-\alpha)$ held constant. Over five seeds the authors report gains on NL-RX, GSM8K, Spider, NQ-Open and HellaSwag across Llama-3, Gemma-2, OpenELM and OLMo (1B–8B), gains that persist at every LoRA rank they tried (32–512), resistance to overfitting, and a Text→Code mapping confined to a narrow, near-linear subspace (singular values of Enc(Text) − Enc(Code) orders of magnitude below the base model's). The two costs they name are the extra forward pass and the (λ, k) grid search.

Why it matters for HiveMind. (i) The extra cost falls on training only; served latency on `hivemind/coder-lite` and `hivemind/granite-moe` is unchanged. (ii) Our nocturnal QLoRA rounds already hold two-view data: code requests and their completions, Lean statements and proofs, and the French/English pairs that dominate the `B3` gate failures (Section 13.3). (iii) Loss dropout maps onto the volunteer VRAM budget: a $\leq$ 4,300 MiB-floor T4 or an 8 GB RTX 2070 can run $\alpha = 0.75$ at 1.25× nominal forward-pass cost. (iv) Evaluating whether the view-gap singular value σ_1 tracks held-out NLL or can decouple from it directly tests how the `B3` train-to-serve gate should treat geometric diagnostics.

Implementation (`IP-JEPA-01..04`). `src/hivemind/swarm/llm_jepa.py` implements Eq. (29) in plain `torch`: 4-D additive block-causal packing (`pack_two_views`, `block_causal_mask`), tied `[PRED]` predictor (`jepa_term`, `llm_jepa_loss`), cosine/MSE/ℓ_2 metrics, batch-level loss dropout, the $\lambda(1-\alpha)$ co-tuning rule (`cotuned_lambda`), top-k view-gap singular values (`view_gap_singular_values`), and the paired held-out promotion gate `evaluate_jepa_b3_gate` (which requires mean_nll_gain_nats > 0 and a strictly positive 95 % bootstrap lower bound while recording σ_1 with `gating = false`). `tests/test_llm_jepa.py` (12 tests, 100 % statement coverage on `llm_jepa.py`) verifies zero cross-view mask leakage inside a real Llama block, gradient flow into model weights, co-tuning invariants, and positive/negative `B3` gate decisions.

14.1 Pilot `E28` (5 Seeds, Constant Learning Rate; `IP-JEPA-01`)

`scripts/run_e28_llm_jepa_pilot.py` trains a 2-layer, $d = 64$ Llama-architecture model from random initialisation (matching the paper's Table 2 pretraining setting) on a synthetic two-view task: Text = "`<op> of <digit words>`" with $op \in \{\text{copy}, \text{reverse}, \text{sort}, \text{rotate}\}$ over 3–6 digits; Code = the result on separate code-side tokens (600 training and 400 disjoint held-out pairs, 60 epochs, constant-LR AdamW at 3×10^{-3}, $\lambda = 1$, $k = 1$, $\alpha = 0$, seeds $\{82, 23, 37, 84, 4\}$ on an NVIDIA GeForce RTX 2070). The third arm, `jepa_shuf`, rotates the Code view across batch examples as a negative control. Pre-registered prediction `P28a` required JEPA to beat NTP on the paired mean *and* beat `jepa_shuf` in every seed.

Table 31: Pilot `E28` (`artifacts/experiments/E28_llm_jepa_pilot.json`): held-out exact match over 5 seeds (constant LR 3×10^{-3}) on an RTX 2070. Wall-clock includes unbatched greedy decoding; t has 4 degrees of freedom.

Arm	Held-out exact match (mean ± sd)	Top singular value of view gap	Total wall-clock / run
NTP ($\mathcal{L}_{\text{LLM}}$ only)	75.90 % ± 7.43	149.85	91.8 s
LLM-JEPA ($\lambda = 1$, $k = 1$)	80.00 % ± 10.43	1.90	123.3 s
Shuffled-view control	44.55 % ± 13.77	4.81	123.6 s

JEPA − NTP: +4.10 points (sd 13.47, $t = 0.68$, positive in 4/5 seeds) ⇒ `P28a` **not supported**
JEPA − shuffled: +35.45 points (sd 14.04, $t = 5.65$, positive in 5/5 seeds); shuffled − NTP: −31.35 points ($t = -4.18$)

Table 31 showed a positive mean (+4.10 points) and strong separation from the shuffled-view control (+35.45 points, 5/5 seeds) alongside a 78.9× collapse of the view-gap singular value (149.85 → 1.90), but 5 seeds under a constant learning rate were underpowered ($t = 0.68$; seed 23 went 64.75 % vs. 82.75 %), so `P28a` was recorded as *not supported*.

14.2 10-Seed Statistical Power, Loss-Dropout & Co-Tuning Study `E28b` (`IP-JEPA-02`)

To resolve the power ambiguity of `E28` and test Section 5.2 of Huang *et al.* [64] (batch-level JEPA loss dropout $\alpha \in \{0.50, 0.75\}$ with $\lambda(1-\alpha) = \text{const}$), `scripts/run_e28b_llm_jepa_dropout_cotuning.py` expands the benchmark to 10 seeds ($\{82, 23, 37, 84, 4, 11, 19, 29, 41, 53\}$) under cosine learning-rate annealing ($3 \times 10^{-3} \to 3 \times 10^{-4}$), isolates pure CUDA training time from batched evaluation, and compares six arms (Table 32). Pre-registered prediction `P28b_gain` required `jepa_full` > `ntp` at one-sided paired $p < 0.05$ and 10/10 wins over `jepa_shuf`; `P28b_dropout` required `jepa_drop75_cotuned` ($\alpha = 0.75, \lambda = 4.0$) to retain ≥ 95 % of `jepa_full`'s accuracy within ≤ 1.25× pure-training time.

Table 32: Experiment `E28b` (`IP-JEPA-02`, `artifacts/experiments/E28b_llm_jepa_dropout_cotuning.json`): 10-seed LLM-JEPA power, loss-dropout ($\alpha \in \{0.50, 0.75\}$), and $\lambda(1-\alpha)$ co-tuning on an RTX 2070 (cosine LR $3 \times 10^{-3} \to 3 \times 10^{-4}$; paired t with $df = 9$).

Arm	Exact match (mean ± sd)	Held-out NLL	View-gap σ_1	Pure train time / run	Time ratio vs. NTP
`ntp` ($\lambda = 0$)	72.68 % ± 13.27	0.3324	130.91	22.05 s	1.000×
`jepa_full` ($\lambda = 1.0, \alpha = 0.0$)	**89.02 % ± 6.07**	**0.0987**	**1.11**	52.86 s	2.397×
`jepa_drop50_cotuned` ($\lambda = 2.0, \alpha = 0.50$)	79.17 % ± 14.09	0.2397	1.25	37.88 s	1.718×
`jepa_drop75_cotuned` ($\lambda = 4.0, \alpha = 0.75$)	61.10 % ± 26.69	0.5990	1.77	30.70 s	1.392×
`jepa_drop75_naive` ($\lambda = 1.0, \alpha = 0.75$)	**84.72 % ± 9.61**	**0.1596**	2.25	**30.64** s	**1.390**×
`jepa_shuf` ($\lambda = 1.0$, shuffled views)	50.05 % ± 21.72	0.7466	1.93	52.69 s	2.390×

`jepa_full` − `ntp`: +**16.35** pts (sd 15.76, 9/10 wins, $t = 3.2805$, $p = \mathbf{0.004761}$); vs. `jepa_shuf`: +**38.97** pts (10/10 wins, $p = \mathbf{9.8 \times 10^{-5}}$) ⇒ `P28b_gain` **SUPPORTED**
`jepa_drop75_cotuned` vs. `jepa_full`: retention **68.64 %** < **95 %** (−11.57 pts vs. NTP, $t = -1.0110$); vs. `jepa_drop75_naive`: −**23.63** pts (2/10 wins, $t = -2.5548$) ⇒ `P28b_dropout` **REFUTED**
`jepa_drop75_naive` − `ntp`: +**12.05** pts (sd 10.80, 7 wins / 1 tie / 2 losses, $t = 3.5267$, $p = \mathbf{0.003224}$, **95.17** % retention of `jepa_full` at 1.390× train time)

Findings of `E28b`. (1) **`P28b_gain` is supported:** across 10 seeds under cosine decay, `jepa_full` lifts held-out exact match from 72.68 % to 89.02 % (+**16.35** points, one-sided $p = 0.004761$), halves seed-to-seed variance (13.27 % → 6.07 %), cuts held-out NLL by 3.37× (0.3324 → 0.0987), and beats `jepa_shuf` in 10/10 seeds (+38.97 points). (2) **`P28b_dropout` is refuted for $\lambda(1-\alpha) = \text{const}$ co-tuning from random initialisation:** scaling λ up to 4.0 at $\alpha = 0.75$ (`jepa_drop75_cotuned`) causes severe instability on sensitive seeds (18.50 % on seed 23, 26.00 % on seed 84), losing 23.63 points against unscaled $\lambda = 1.0$ ($t = -2.5548$). Whereas Huang *et al.* derived $\lambda(1-\alpha) = \text{const}$ when fine-tuning *pretrained* 1B–8B models whose hidden states already inhabit a structured manifold, applying 4× intermittent $\mathcal{L}_{\text{JEPA}}$ spikes to random weights periodically overwhelms token cross-entropy gradients. Conversely, keeping **unscaled** $\lambda = 1.0$ **at** $\alpha = 0.75$ **(`jepa_drop75_naive`)** beats NTP by +**12.05** points ($p = 0.003224$) and retains **95.17** % of `jepa_full`'s accuracy while cutting training wall-clock by 42.0 % (52.86 s → 30.64 s). Note also that pure-training wall-clock at $\alpha = 0$ is 2.397× (1.390× at $\alpha = 0.75$), slightly above the nominal $(2-\alpha)\times$ pass count because packing both views plus `[PRED]` roughly doubles sequence

length inside eager 4-D attention.

14.3 Stage-2 4-Bit `NF4` `GraniteMoE` QLoRA DiLoCo & `B3` Promotion Gate `E29` (`IP-JEPA-03` & `IP-JEPA-04`)

To test whether LLM-JEPA transfers directly into HiveMind's low-VRAM volunteer learner without regressing held-out NLL, `scripts/run_e29_qlora_moe_jepa_b3_pilot.py` integrates Eq. (29) into our Stage-2 `bitsandbytes` 4-bit `NF4` `GraniteMoeForCausalLM` learner (4 experts, top-2 routing, PEFT LoRA $r = 4$ on `q_proj, v_proj` + `lm_head`; Section 13.1) across $K = 2$ non-IID two-view shards (8 outer DiLoCo rounds $\times 35$ inner steps, 5 seeds $\{1..5\}$ on the RTX 2070; Table 33). Because Hugging Face's `GraniteMoeForCausalLM` rejects custom 4-D additive masks (`use_4d_block_causal_mask = false`), the learner automatically executes the exact two-pass fallback (`View1+[PRED]` and `View2` encoded separately) within **37.01**–**37.03** MiB peak CUDA VRAM.

Table 33: Experiment `E29` (`IP-JEPA-03/04`, `artifacts/experiments/E29_qlora_moe_jepa_pilot.json`): 5-seed Stage-2 4-bit `NF4` `GraniteMoE` ($r = 4$) QLoRA DiLoCo across $K = 2$ non-IID shards evaluated by `evaluate_jepa_b3_gate` against `qlora_ntp`.

Arm	Final target NLL	Target tok. acc.	`B3` NLL gain (95 % CI)	Non-gating σ_1	Peak VRAM / Wall	`B3` Gate
`qlora_ntp` ($\lambda = 0$)	**1.293622 ± 0.070025**	**74.06 %**	Baseline ($n = 5$)	70.7279	37.01 MiB / 14.3 s	Baseline
`qlora_jepa_drop75` ($\alpha = 0.75, \lambda = 2.0$)	1.400363 ± 0.089132	72.51 %	−0.106741 [−0.1301, −0.0830]	36.8428 (0.521×)	37.03 MiB / 20.6 s	**REFUSED**
`qlora_jepa_full` ($\alpha = 0.0, \lambda = 0.5$)	1.434525 ± 0.068891	72.36 %	−0.140903 [−0.1522, −0.1293]	23.1986 (0.328×)	37.01 MiB / 39.7 s	**REFUSED**
`qlora_jepa_shuf` ($\alpha = 0.0$, shuffled)	1.428569 ± 0.068203	72.72 %	−0.134947 [−0.1569, −0.1130]	23.4985 (0.332×)	37.01 MiB / 39.7 s	**REFUSED**

Why the negative `E29` result validates `IP-JEPA-03` and `IP-JEPA-04`. Two operational lessons emerge from Table 33:

1. **Low-rank ($r = 4$) attention-only `NF4` QLoRA trades off embedding alignment against next-token NLL (`IP-JEPA-03`):** All arms converge substantially from initial NLL ≈ 4.17, yet every JEPA arm finishes 0.107–0.141 nats worse than `qlora_ntp` (1.293622), so `evaluate_jepa_b3_gate` returns `servable = false` across the board and **zero served models are modified**. Unlike full-parameter training in `E28b` or Huang *et al.*'s $r \in [32, 512]$ LoRA regime, freezing all 4 `GraniteMoE` expert FFNs in 4-bit `NF4` and restricting trainable capacity to $r = 4$ on `q_proj, v_proj` leaves insufficient subspace capacity to satisfy both last-token cosine alignment and token-level prediction simultaneously across non-IID DiLoCo shards.

2. **View-gap singular values σ_1 shrink even on mismatched views and must never gate promotion (`IP-JEPA-04`):** In Table 33, σ_1 drops from 70.7279 (`qlora_ntp`) to 23.1986 (`qlora_jepa_full`) *and equally to* 23.4985 *on the shuffled-view negative control* ***`qlora_jepa_shuf`*** while held-out NLL worsens by 0.135 nats. Minimizing last-token cosine distance compresses hidden-state norms regardless of semantic correspondence when adapter capacity is bottlenecked; recording `view_gap_sv1` strictly with `gating = false` prevents geometric collapse from masquerading as generalisation.

Next swarm step (`E15-J` / `HM-S10-11`, planned, not yet run). Testing whether LLM-JEPA helps pretrained 1.5B–3B models requires higher LoRA rank ($r \geq 16$–32 across attention and expert MLP projections) on natural-language/code pairs: `E15-J` (`HM-S10-11`) specifies a 3-seed test on `Qwen2.5-Coder-1.5B` GWAYA-gated rejection-sampling fine-tuning at fixed $(\lambda, k) = (1, 1)$ and unscaled dropout $\alpha = 0.75$ within the $\geq 4{,}300$ MiB T4 free-VRAM floor, gated by `B3` before any promotion.

15 Limitations, Threats to Validity, and Future Multi-Host Benchmarks

What this paper establishes vs. what remains open after the v4.0.0 Release (Live 47-Node WAN Swarm & Bi-Layer Bee Mesh).

1. **From single-host simulation to 47-node live WAN deployment (resolved vs. remaining scale).** Earlier revisions (v3.5.0–v3.7.1) relied on single-host process simulations (E1–E4, E13–E14) and a two-host internet pilot (E8). As documented in Section 9.18 (`artifacts/live_wan_swarm_production_report.json`) and extended in Section 10, the primary multi-host WAN validation milestone is now established across **47 registered nodes**, **100 assimilated BOINC 6-domain MoE workunits** (**5,632.11 Cobblestones** $\approx$ 2.433 PFLOPs), **24 nocturnal multi-producer MoE workunits** (4,525.0 TFLOPs-hours cumulative attested swarm compute), **1,427 billed OpenRouter completions** (240,162 tokens at 196.8 ms mean endpoint p50 latency), and the 20-adapter nocturnal MoE campaign (+277.06 TFLOPs, 35 models total). What remains open is scaling synchronous/quorum DiLoCo pre-training from scratch across > 100 concurrent WAN workers simultaneously.

2. **Single seed in E1–E4.** Every experiment in Section 8 uses one seed (2026) and therefore demonstrates that the mechanisms behave as specified without estimating an effect size. E5-toy (Section 9.5), E5-het (Section 9.6) and the E8 pilot (Section 9.7) are the first three-seed, controlled measurements; the IID toy's main finding is that it is too easy to rank the mechanisms.

3. **Full-scale standard benchmark suites.** While E15 and E16 evaluate full MBPP-sanitized ($n = 257$, reaching 85.6 % pass@1 on 7B and 85.2 % on the small-first cascade) and HumanEval ($n = 164$, 67.7 %), and `NIGHT-MOE-1` validates GAIA/Hipparcos stellar classification ($n = 317$, 89.3 % accuracy, McNemar $p < 10^{-4}$), full-split zero-shot evaluations on MMLU and full GSM8K across all 24 nocturnal MoE adapters remain ongoing.

4. **128-expert training scale.** While 24 domain experts across 6 frontier base models (`Qwen2.5-Coder-7B`, `Gemma-2-9B`, `Gemma-3-4B`, `Qwen3.5-9B-Base`, `Gemma-3-1B`, `Qwen2.5-7B`) have been trained and verified in the nocturnal campaign, end-to-end training of all 128 domain branches simultaneously under the 3-tier hierarchical router (`H3`, `H5`, `E4`) remains a future scaling target.

5. **Adaptive Byzantine adversaries.** Our Multi-Krum + 2σ cosine filter and 10 % micro-checkpoint spot-check have caught 100.0 % (6/6) of injected lazy and Byzantine poisoners in production; subtle inner-product manipulation (ALIE) and coordinated Sybil coalitions under non-IID drift warrant dedicated red-team evaluation (E9).

6. **Spot-checks across execution classes and heterogeneous hardware (resolved in production via execution-class routing and asymmetric role assignment).** The $\epsilon = 10^{-4}$ replay rule of Section 4.1 fails on CPU for a thread-count change alone (E12 pilot, $2.6 \cdot 10^{-4}$); it holds only for bit-identical execution. The bf16 precision class, replayed in fp32, diverges by $3.3 \cdot 10^{-3}$ with every tensor above threshold, more than the tampered control: dtype is a class boundary that no tolerance crosses. Cross-architecture replay divergence was further measured on GPU in E12 (Section 9.11): the rule holds exactly within one T4 and fails between a T4 and an x86 CPU ($2.0 \cdot 10^{-4}$ to $6.3 \cdot 10^{-4}$ in 9/9), so the same-class claim is now a measured requirement for that pair; validators must be architecture-matched. In production (Section 9.18, Table 19), tagging workunits by execution class (`cuda-sm75`, `cuda-sm86`, `cuda-sm89`) and assigning heterogeneous CPU and Apple Metal nodes to deterministic ZK-OPoI and AST audits reduces the honest validator false-rejection rate from 100.0 % (unmatched) to **0.0 %** (0/124 honest workunits rejected).

7. **Open after the October 2026 T4 campaign.** E10b gave no valid loss result at 0.5B scale (six rounds of twenty steps cannot resolve codec effects; Section 9.9); E11 rests on fake quantisation at prefill, one dataset, three window-set "seeds", a single rate-matched cell for the A_2 comparison on Qwen-0.5B, and an invalid Qwen2.5-Coder-7B row (negative control failed) whose FP8 arm has not been investigated (Section 9.10); E12 compares a T4 with its own separate process and one x86 CPU only, with a LoRA initialisation about 18× larger than the PEFT default (Section 9.11). A true fixed-point replay kernel and a longer E10b remain to be done. The E13–E16 campaign and the v3.9.0 live 47-node WAN deployment are reported in Sections 9.13 to 9.18.

8. **Compounding approximations.** NF4 base weights, 2-bit DCT outer gradients with error feedback, Skip-Token, speculative Skip-Attention, PA-CGAD truncation and fixed-point loop exits were each verified in isolation (H1–H11, E1–E4). In the 98-round BOINC 6-domain QLoRA campaign, held-out NLL improved by $-5.8\,\%$ before plateauing around round 30, where the monotone loss ratchet held subsequent non-improving rounds (78/98).

9. **Proof-of-concept scale vs. production curriculum.** While the initial RTX 2070 PoC (Section 8.3) trained on 8 rows per worker as a VRAM plumbing check, the production BOINC 6-domain curriculum (98 rounds), `NIGHT-MOE-1` ($n = 317$ held-out stellar test set), E15/E16 ($n = 257$ MBPP-sanitized and $n = 164$ HumanEval), and the 24-expert nocturnal producer campaign measure genuine held-out generalization with paired McNemar controls.

10. **Scope of Lean 4 formalization.** The 166 Lean 4 lemmas (expanded from 161 in v4.2.0) in `lean4/HiveMindSpec.lean` verify discrete arithmetic, hypothesis-necessity counterexamples, non-vacuity inhabitation, and byte-budget invariants over $\mathbb{N}/\mathbb{Z}$ (Section 2); they do not formalize real-valued stochastic convergence or C++/CUDA kernel equivalence. A compiling lemma is not automatically a constraint on the system: Theorems 125 and 127 of Season 15 compile under the axiom whitelist yet say nothing about the gate or the barrier they are named after (one conjunct is a tautology, one hypothesis is unused). They stay in the count because the count is a count of compiled declarations; the inventory table marks them.

11. **LLM-JEPA scope across random-init vs. low-rank `NF4` QLoRA (v4.4.0).** While 10-seed full-parameter training on a 2-layer Llama model (`E28b`) confirms significant gains over NTP (+16.35 pts full, +12.05 pts at unscaled $\alpha = 0.75$) and refutes $\lambda(1-\alpha) = \text{const}$ co-tuning from random initialisation, our 5-seed Stage-2 4-bit `NF4` `GraniteMoE` QLoRA study (`E29`, LoRA $r = 4$ on attention projections only) degrades held-out NLL by 0.107–0.141 nats and is rightly rejected by `evaluate_jepa_b3_gate` (`servable = false`). Evaluating LLM-JEPA on pretrained 1.5B–3B backbones with higher-rank LoRA ($r \geq 16$–32 across attention + MLP modules, planned as `E15-J` / `HM-S10-11`) remains open; no served HiveMind model uses a JEPA-trained adapter (Section 14).

12. **No peer review of record has taken place.** An external review of v3.5.0 whose author is not stated was received on 23 October 2026, and a second external operational review of v3.7.1/v3.8.0 was received on 27 October 2026; both are recorded verbatim with point-by-point responses in `paper/review/` and led to the revisions of v3.6.0 and v3.9.0, but neither is a journal peer review of record. `paper/autoevolve_peer_review_pipeline.py` is a deterministic script (Lean build, JSON-to-LaTeX number lock, LaTeX lint). Versions 2.9–3.1 of this preprint and its `PEER_REVIEW_REPORT.json` labelled that script a "Claude Opus 5.5 / Gemini 5-round peer review" with referee scores of 9.4–9.5/10; no language model and no human referee ever evaluated the manuscript, and those scores were constants in the script. The labels and scores are removed in v3.2.0 and the report now states `is_peer_review: false`. Several entries of `docs/ACKNOWLEDGMENTS_AND_CITATIONS.md` and `CITATION.cff` also carried invented titles, authors or arXiv identifiers; they were re-verified against arXiv

on 23 October 2026 and corrected or removed, and every entry of this paper's bibliography was checked individually.

15.1 Epistemic Stratification & 6-Layer Security/Hardness Attestation

To prevent conflating machine-checked discrete arithmetic with empirical single-host measurements or open 128-branch scaling targets, Table 34 stratifies all claims into three explicit verification tiers. Following the 27 October 2026 external review recommendation, the **47-node heterogeneous WAN deployment** (`artifacts/live_wan_swarm_production_report.json`) is now promoted into **Category II: Controlled Empirical & Live Multi-Host WAN Measurements**. Concurrently, the repository is audited by the 6-Layer `HiveMindHardnessEngine` (`results/hivemind_hardness_report.json`), verifying: (L1) zero AST stubs across all modules; (L2) zero security findings across 279 scanned repository files (`risk_score = 0`); (L3) 25/25 simulated Byzantine, Sybil, NaN-injection, and WDDM TDR attack vectors blocked; (L4) 166/166 Lean 4 lemmas (161/161 independent runner gate in v4.2.0, expanded to 166/166 in v4.3.0) compiled with 0 `sorry` and 0 custom axioms; (L5) exact rational bridge verification (`NF4` 128/33, `DiLoCo` wire 5000/3, `Prefix-Trie` 19/25, `3-Tier MoE` 1/8); and (L6) SHA-256 execution attestation.

Table 34: Epistemic stratification matrix separating machine-checked discrete lemmas (Category I), controlled single-host and live 47-node WAN measurements (Category II), and open 128-branch scaling targets (Category III).

Verification Category	Artifact / Proof Source	What Is Rigorously Established	Explicit Boundary / Not Claimed
Category I: Lean 4 Kernel Lemmas	`lean4/HiveMind2Spec.lean` (166 lemmas, 0 `sorry`, expanded from 161) + 3 Mathlib RDA theorems	Discrete $S(Z,$ byte budgets, flat-bucket partitions, EF21 telescoping sums, necessity counterexamples (Thms. 113–115), **Resnpty** witness (Thm. 116), R-inner-product RDA norm identity, and **ALIX_Quantbee** recruitment bounds.	Does not prove real-valued stochastic convergence or Python/CUDA kernel refinement.
Category II: Controlled Empirical & Live 47-Node WAN Swarm	H1–H11, E1–E16, RTX 2070 PoC), `live_wan_swarm_production_report.json`, `nocturnal_20_moe_stress_test_report.json`, `exoplanet_astro_vmoe_poc_report.json` & `massive_swarm_image_moe_report.json`	**Live 47-node heterogeneous WAN swarm** (100 BOINC WUs + 24 nocturnal MoE WUs, **5,632.11 Cobblestones** / 2,433 PFLOPs, **4,525.0 TFLOPs-h** cumulative, **3,409–4,838 MiB** QLoRA VRAM, **0.0 %** execution-class-matched false rejection rate, **51.24 GFLOPS** Turing kernel at cos = 0.9963, **1,427** OpenRouter completions at **196.8 ms** mean p50): 20 nocturnal MoE adapters (+277.06 TFLOPs, 35 models total, +33.1 % mean accuracy gain, 0 VRAM breaches); Sub-1.5 GB "La Ruche" Vision-MoE (**972.0–1,490.0 MiB**), Exophant CWT/Focal-Loss PoC (**910.0–1,410.0 MiB**, **99.97 %** V-MoE gain), and 7-Division **42,870,000-sample (8,574-shard)** 104-Bee Swarm Orchestrator (**99.84 %** V-MoE gain); Swarm Nectar Cache (60% duplicate query reduction, 0.005 HMT royalties); BYOB 4-stage attestation lifecycle (3 exemplars); 0.062% E1 loss gap at 12.49× wire savings; E15/E16 MBPP 85.6 %.	Chained 6-domain QLoRA plateaued after ∼ 30 rounds (79/98 held by monotone loss ratchet); Elib/E11 Codec-7B rows invalid as reported.
Category III: Open 128-Branch & 1,000-Node Roadmap Targets	Future 100–1,000+ worker testbed (Sections 11–15)	Analytical communication models for 128-expert FoMoE (3.46×), Kademlia Geo-Ring topologies, Season 24 cross-swarm Byzantine quorum sensing, and the 18-Month Roadmap (Seasons 29–32: Top-1 % Quantized Split Learning, $\mathcal{O}(1)$ **zk-SNARK** honest training proofs, and 15–20 node Geo-Cryptographic Mesh sub-swarms).	End-to-end training of all 128 expert branches simultaneously, live 1,000-node WAN **zk-SNARK** proving, and adaptive ALIE coalition red-teaming (E9) remain open.
Category IV: Elenchus Tier X Discrete-Event Sizing & Orchestration Simulations	E20_international_launch_split_dct_report.json, international_launch_200plus_planet_forecast.json, E27_testbed_1000_report.json	Queueing-theoretic $M/M/c(t)$ / $G/G/c$ / P2C / Kelly / Bass launch sizing (205 nodes + 2,400 visitors), 6-region Super-Peer tree fan-out (31.45×), and 1,000-node heterogeneous churn/Byzantine discrete-event testbed (98.4% completion, 100% poison rejection).	Deterministic single-host mathematical and discrete-event simulations (Elenchus Tier X) never claimed as 205 or 1,000 live physical WAN hosts; **ZK-DPoI v4/v5** is Fiat-Shamir Freivalds/Horner + HMAC, while full **zk-SNARK** circuits are parked (PR-11-R1).

Reproducibility. Every number and figure in this paper is generated deterministically from committed scripts and JSON artifacts:

- `PYTHONPATH=src python -m hivemind.swarm.architecture_hypothesis_verifier` → `artifacts/architecture_hypothesis_verification_report.json` (H1–H10, `pytest tests/test_architecture_hypotheses.py`).
- `PYTHONPATH=src python scripts/run_paper_experiments_and_figures.py` → `artifacts/paper_v3_experiments_report.json` (E1–E4) and `paper/figures/fig1..fig4.{pdf,png}`.
- `PYTHONPATH=src python scripts/verify_decoupled_diloco_quorum.py` → `artifacts/decoupled_diloco_quorum_check.json` (H11, `pytest tests/test_decoupled_diloco_quorum.py tests/test_fomoe_sigma_routing.py`).
- `PYTHONPATH=src python scripts/generate_live_wan_swarm_report.py` → `artifacts/live_wan_swarm_production_report.json` (Section 9.18: 47-node WAN swarm, 5,632.11 Cobblestones, 4,525.0 TFLOPs-h, 1,427 OpenRouter completions, and Volunteer Tax audit).

- `PYTHONPATH=src pytest tests/test_abc_moe_agent_mesh.py tests/test_swarm_nectar_cache.py` → Bi-Layer Honeybee Agent Mesh (AMCP-Kafka Quorum Sensing, ABC MoE transitions, and Swarm Nectar Cache P2P deduplication with 0.005 HMT royalties).
- `PYTHONPATH=src pytest tests/test_dynamic_diurnal_moe_engine.py` → `artifacts/nocturnal_20_moe_stress_test_report.json` and `artifacts/nocturnal_large_moe_20_campaign_report.json` (Section 10: 20 nocturnal MoE adapters across 4 Macro-Clusters, +277.06 TFLOPs attested compute, 35 models total, long-tail intent scoring, and 3 BYOB live exemplars).
- `PYTHONPATH=src python scripts/run_nocturnal_reddit_vision_moe.py` → `artifacts/nocturnal_reddit_vision_moe_report.json`; `PYTHONPATH=src python scripts/run_exoplanet_astro_vmoe_poc.py` → `artifacts/exoplanet_astro_vmoe_poc_report.json`; `PYTHONPATH=src python scripts/run_massive_image_moe_swarm.py` → `artifacts/massive_swarm_image_moe_report.json` (Section 11: sub-1.5 GB "La Ruche" V-MoE, Exoplanet CWT/Focal-Loss PoC, and 7-Division 42.87M-sample 104-Bee Swarm Orchestrator; verified by `pytest tests/test_reddit_vision_vlm_moe.py tests/test_exoplanet_astro_vmoe_poc.py tests/test_massive_image_moe_swarm_orchestrator.py`).
- `PYTHONPATH=src python scripts/run_international_launch_and_vmoe_monitor.py` → `artifacts/experiments/E20_international_launch_split_dct_report.json`, `artifacts/nocturnal_image_moe_evolution_monitor.json`, `artifacts/international_launch_200plus_planet_forecast.json`, and `artifacts/gcp_hivemind_planet_forecast.html` (Section 12: Experiment E20 [P20a–P20c], 3-plan-class BOINC Quorum-of-2 assimilation, and > 200-node international launch queueing/balancing forecast; verified by `pytest tests/test_international_launch_scaler.py tests/test_international_launch_model.py`).
- `PYTHONPATH=src python paper/autoevolve_peer_review_pipeline.py` → `paper/PEER_REVIEW_REPORT.json` (scripted static audit: Lean inventory, JSON-to-LaTeX provenance lock across `PROV-01..PROV-89`, LaTeX lint; it calls no language model and is not a peer review).
- `PYTHONPATH=src python scripts/verify_ps01_d4_chain_provenance_lock.py` and `PYTHONPATH=src python scripts/mvp_scorecard.py` → `artifacts/experiments/e8_real/E8_two_host_live_verification.json`, `artifacts/mvp/A1_multi_gpu_serving.json`, `artifacts/mvp/B2_diloco_qlora.json`, `artifacts/mvp/D1_gcp_cost.json`, and `artifacts/experiments/PLANETARY_SCALE_READINESS_2026-11-08.json` (Section 13).
- `PYTHONPATH=src python scripts/run_e28_llm_jepa_pilot.py` → `artifacts/experiments/E28_llm_jepa_pilot.json`; `PYTHONPATH=src python scripts/run_e28b_llm_jepa_dropout_cotuning.py` → `artifacts/experiments/E28b_llm_jepa_dropout_cotuning.json`; and `PYTHONPATH=src python scripts/run_e29_qlora_moe_jepa_b3_pilot.py` → `artifacts/experiments/E29_qlora_moe_jepa_pilot.json` (Section 14; verified by `pytest tests/test_llm_jepa.py`).
- `cd lean4 && lake build` → compiles all 166 lemmas (161 theorems in independent gate, 166 in v4.3.0) in `lean4/HiveMindSpec.lean` with zero `sorry`.
- The PDF, the LaTeX source, the Lean specification and every JSON artifact quoted above are deposited on Zenodo under DOI 10.5281/zen-

odo.23776302 (`scripts/publish_hivemind_paper_zenodo.py`, receipt `paper/ZENODO_DEPOSIT_RECEIPT.json` with per-file MD5 verification).

References

[1] D. P. Anderson, "BOINC: A System for Public-Resource Computing and Storage," in *Proc. IEEE/ACM Int. Workshop on Grid Computing*, 2004, pp. 4–10.

[2] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen, "DiLoCo: Distributed Low-Communication Training of Language Models," *arXiv:2311.08105*, 2023.

[3] B. Liu, R. Chhaparia, A. Douillard, S. Kale, A. A. Rusu, J. Shen, A. Szlam, and M. Ranzato, "Asynchronous Local-SGD Training for Language Modeling," *arXiv:2401.09135*, 2024. Code: https://github.com/google-deepmind/asyncdiloco.

[4] A. Douillard, Y. Donchev, J. K. Rush, S. Kale, Z. Charles, G. Teston, Z. Garrett, J. Shen, R. McIlroy, D. Lacey, A. Ramé, A. Szlam, M. Ranzato, and P. R. Barham, "Streaming DiLoCo with Overlapping Communication: Towards a Distributed Free Lunch," in *Second Conference on Language Modeling (COLM)*, 2025, *arXiv:2501.18512*.

[5] Decoupled DiLoCo Team (Google DeepMind & Google Research; A. Douillard, K. Rush, Y. Donchev, Z. Charles, A. Dubey, B. Woodworth, *et al.*), "Decoupled DiLoCo for Resilient Distributed Pre-training," *arXiv:2604.21428*, 2026.

[6] X. Callens, "A laboratory method for AI-assisted science: formal proof, exact arithmetic, controlled experiment, and a claim ledger, tested for a month on the Navier–Stokes problem," Zenodo, v1.10.2, 2026, doi:10.5281/zenodo.22829767.

[7] X. Callens, "Karpathy-autoresearch Screening and Machine-Checked Elenchus Deep-Dive Across Fermat's Last Theorem, Navier–Stokes Admissibility Censorship, and Kinetic Spectral Caps: A Ten-to-Five Hypothesis Investigation in Lean 4," Zenodo, 2026, doi:10.5281/zenodo.23467992.

[8] X. Callens, "RunuX: Deterministic Fixed-Point Attention and Measured PolarQuant KV-Cache Compression on a Commodity GPU, with a Hardware Test of Its TPU Claims," Zenodo, v2.0.0, 2026, doi:10.5281/zenodo.22879667.

[9] X. Callens, "Non-Equilibrium Hopfield–Ninio Proofreading, Chimeric ART-Exo Fidelity, Gosset E_8 Dicodon Quantization, and Topological DNA Supercoiling in LeanFlow, rusty-SUNDIALS, and Lean 4," Zenodo, v3, 2026, doi:10.5281/zenodo.23526783.

[10] X. Callens, "Laya-LoRA Coding Companion: Asymmetric Dual-Process Test-Time Compute, Full-Scale Curriculum on 10 Structured Coding Datasets, and Serverless Multi-Tier Inference Infrastructure on GCP," Zenodo, 2026, doi:10.5281/zenodo.23095224.

[11] J. H. Conway and N. J. A. Sloane, "Fast Quantizing and Decoding Algorithms for Lattice Quantizers and Codes," *IEEE Trans. Inf. Theory*, 28(2):227–232, 1982.

[12] X. Callens, "The Triangular Lattice and Universal Optimality: human proofs, Lean 4 verification, numerical validation in Python and Rust, and physics from fluids to plasmas," Zenodo, 2026, doi:10.5281/zenodo.23295142; companion code *OptimalEnergyAI* (`oai_ml.core.quantise`, `oai_ml.core.rate`).

[13] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed Optimisation for Non-Convex Problems," in *ICML*, 2018, *arXiv:1802.04434*.

[14] S. Kale, A. Douillard, and Y. Donchev, "Eager Updates for Overlapped Communication and Computation in DiLoCo," *arXiv:2502.12996*, 2025.

[15] A. Defazio, K. Mishchenko, P. Raman, H.-J. M. Shi, and L. Xiao, "Smoothing DiLoCo with Primal Averaging for Faster Training of LLMs," *arXiv:2512.17131*, 2025.

[16] Z. Charles, G. Teston, L. Dery, K. Rush, N. Fallen, Z. Garrett, A. Szlam, and A. Douillard, "Communication-Efficient Language Model Training Scales Reliably and Robustly: Scaling Laws for DiLoCo," *arXiv:2503.09799*, 2025.

[17] S. Jaghouar, J. M. Ong, and J. Hagemann, "OpenDiLoCo: An Open-Source Framework for Globally Distributed Low-Communication Training," *arXiv:2407.07852*, 2024. Code: https://github.com/PrimeIntellect-ai/OpenDiLoCo, https://github.com/PrimeIntellect-ai/prime-diloco.

[18] S. Sukhbaatar *et al.*, "Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM," *arXiv:2403.07816*, 2024.

[19] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *NeurIPS*, 2023.

[20] L. Sani, Z. Cao, M. Kurmanji, A. Iacob, A. Jovanović, Y. Gao, W. Zhao, and N. D. Lane, "FoMoE: Breaking the Full-Replica Barrier with a Federation of MoEs," *arXiv:2606.19025*, 2026.

[21] P. Merolla, "fomoe: Fast Opportunistic Mixture-Of-Experts," software repository, https://github.com/pmerolla/fomoe.

[22] Huang *et al.*, "Towards Looped Models Done Right. Part I: Topology, Input Injection, Recurrent-State Design," *arXiv:2610.23036*, 2026.

[23] Huang, Shi, Chen *et al.*, "Towards Looped Models Done Right, Part II: Rethinking at Fixed Points," *arXiv:2610.06833*, 2026.

[24] V. Shah and J. Sun, "Cosine-Gated Adam-Decay: Drop-In Staleness-Aware Outer Optimization for Decoupled DiLoCo," *arXiv:2605.09126*, 2026.

[25] D. Marczak, S. Magistri, S. Cygert, B. Twardowski, A. D. Bagdanov, and J. van de Weijer, "No Task Left Behind: Isotropic Model Merging with Common and Task-Specific Subspaces," in *ICML*, 2025, *arXiv:2502.04959*.

[26] S. Horoi, B. Thérien, G. Wolf, and E. Belilovsky, "Can Model Merging Improve Aggregation in DiLoCo?," *arXiv:2607.03011*, 2026.

[27] A. A. Gargiulo *et al.*, "Task Singular Vectors: Reducing Task Interference in Model Merging," in *CVPR*, 2025, *arXiv:2412.00081*.

[28] B. Peng, J. Quesnelle, and D. P. Kingma, "DeMo: Decoupled Momentum Optimization," *arXiv:2411.19870*, 2024. Code: https://github.com/bloc97/DeMo.

[29] A. Sarfi, B. Thérien, J. Lidin, and E. Belilovsky, "Communication Efficient LLM Pre-training with SparseLoCo," *arXiv:2508.15706*, 2025. Code: https://github.com/one-covenant/SparseLoCo.

[30] E. Miahi and E. Belilovsky, "Understanding and Exploiting Weight Update Sparsity for Communication-Efficient Distributed RL," *arXiv:2602.03839*, 2026.

[31] Asif *et al.*, "HeLoCo: Efficient Asynchronous Low-Communication Training under Data and Device Heterogeneity," *arXiv:2606.00271*, 2026.

[32] M. Twelftree, D. Lemphers, A. He, and Y. Yang, "Not Every Sync Is Safe: Calibrated DiLoCo Scheduling for Shared AI Infrastructure," *arXiv:2607.02544*, 2026.

[33] C. H. Koneputugodage, T. Ajanthan, S. Ramasinghe, H. M. Dolatabadi, S. Siriwardhana, G. Avraham, V. Shevchenko, K. Pajak, J. Snewin, and A. Long, "Factored Gossip DiLoCo: Reducing Blocking Communication in DiLoCo," *arXiv:2606.22768*, 2026.

[34] P. Cagnasso, E. Belilovsky, and E. Oyallon, "Unifying Local Communications and Local Updates for LLM Pretraining," *arXiv:2606.11081*, 2026.

[35] K. Topollai, A. Ma, T. Dimlioglu, S. J. Tay, and A. Choromanska, "Outer-Momentum Restarting in High-Dimensional Two-Phase Optimization," *arXiv:2605.28585*, 2026.

[36] A. Agarwala, "High dimensional theory of two-phase optimizers," *arXiv:2603.26954*, 2026.

[37] Z. Palanciyan, T. van Osch, D. van der Wal, O. Kotevska, and T. Kok, "Cross-Facility LLM Pre-training on HPC: Elastic Aggregation, Data Leasing, and Queue-Aware Placement," *arXiv:2610.03457*, 2026.

[38] He *et al.*, "AutoLoCo: Communication Efficient Distributed LLM Training via Adaptive Synchronization," *arXiv:2609.36662*, 2026.

[39] K. Jordan *et al.*, "Muon: An Optimizer for Hidden Layers in Neural Networks," technical blog post, 2024.

[40] B. Thérien, X. Huang, A. Defazio, I. Rish, and E. Belilovsky, "MuLoCo: Muon is a Practical Inner Optimizer for DiLoCo," *arXiv:2505.23725*, 2025.

[41] E. Belilovsky, S. Horoi, E. Oyallon, and B. Thérien, "How Inner Optimizers Shape Replica Scaling," *arXiv:2610.19064*, 2026.

[42] P. Richtárik, I. Sokolov, and I. Fatkhullin, "EF21: A New, Simpler, Theoretically Better, and Practically Faster Error Feedback," in *NeurIPS*, 2021, *arXiv:2106.05203*.

[43] M. Ryabinin and A. Gusev, "Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts," in *NeurIPS*, 2020, *arXiv:2002.04013*. Code: https://github.com/learning-at-home/hivemind.

[44] A. Borzunov *et al.*, "Petals: Collaborative Inference and Fine-Tuning of Large Models," in *Proc. ACL System Demonstrations*, 2023.

[45] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *NeurIPS*, 2017.

[46] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," *JMLR*, vol. 23, no. 120, pp. 1–39, 2022.

[47] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Proc. CADE-28*, 2021, pp. 625–635.

[48] T. D. Seeley, *Honeybee Democracy*, Princeton University Press, Princeton, NJ, 2010.

[49] D. Karaboga, "An idea based on honey bee swarm for numerical numerical optimization," Technical Report TR06, Department of Computer Engineering, Erciyes University, 2005.

[50] CNCF Serverless Working Group, "CloudEvents Specification v1.0.2," Cloud Native Computing Foundation (CNCF), 2021. Available: https://cloudevents.io/.

[51] J. Kreps, N. Narkhede, and R. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *Proc. 6th International Workshop on Networking Meets Databases (NetDB)*, 2011, pp. 1–7.

[52] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," in *Proc. 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2016, pp. 305–326.

[53] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.

[54] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[55] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research (TMLR)*, 2024, *arXiv:2304.07193*.

[56] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, *arXiv:2303.15343*.

[57] L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley-Interscience, New York, 1975.

[58] S. Halfin and W. Whitt, "Heavy-Traffic Limits for Queues with Many Exponential Servers," *Operations Research*, vol. 29, no. 3, pp. 567–588, 1981.

[59] J. F. C. Kingman, "The single server queue in heavy traffic," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.

[60] M. Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, 2001.

[61] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate control for communication networks: shadow prices, proportional fairness and stability," *Journal of the Operational Research Society*, vol. 49, no. 3, pp. 237–252, 1998.

[62] F. M. Bass, "A new product growth for model consumer durables," *Management Science*, vol. 15, no. 5, pp. 215–227, 1969.

[63] S. Hwang, B. Wang, and A. Gu, "Dynamic Chunking for End-to-End Hierarchical Sequence Modeling," *arXiv:2507.07955*, 2025.

[64] H. Huang, Y. LeCun, and R. Balestriero, "LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures," *arXiv:2509.14252*, 2025. Code: https://github.com/rbalestr-lab/llm-jepa.